

Relazione Progetto PMCSN - Luca Cupellaro - 0365269 , Luigi Talamo - 0366950

Descrizione del sistema	1
Problematiche del sistema	2
Obiettivi	3
Modello concettuale	4
Eventi modellati	6
Descrizione degli eventi	6
Stato del Sistema	7
Modello delle specifiche	8
Distribuzioni	8
Modellazione dei centri	8
Modello computazionale	10
Algoritmo Next event simulation	10
Verifica	20
Verifica Analitica	20
Risultati ottenuti	22
Validazione	26
Progettazione degli esperimenti	29
Considerazioni e accorgimenti preliminari	29
Autocorrelazione	29
Calcolo degli intervalli di confidenza	30
Analisi del collo di bottiglia	33
Analisi del transitorio	33
Simulazione orizzonte finito	35
One-factor authentication orizzonte finito	37
Two-factor authentication orizzonte finito	43
Verifica Repliche	48
Modello Migliorativo	50
Limitazioni del modello base	52
Obiettivi	53
Modello Concettuale	54
Soluzione proposta	54
Politiche di scheduling dei job all'interno dei server	55
Eventi modellati	55
Stato del sistema	56
Modello delle specifiche	57
Distribuzioni	57
Modellazione dei centri	57
Modello Computazionale	61
Algoritmi per la next-event simulation	61
Scale up	61
Scale Down	62
Probabilità di perdita	63
Meccanismi adottati	63

Algoritmo di Load Balancing	65
Verifica	66
Validazione	74
Progettazione degli esperimenti	79
Analisi del collo di bottiglia	79
Esecuzione delle Simulazioni	82
Analisi del transitorio	82
RNG multi-stream: inizializzazione e utilizzo	82
Simulazioni orizzonte finito	86
One-factor authentication orizzonte finito	86
Two-factor authentication orizzonte finito	91
Conclusioni	96

Descrizione del sistema

Il sistema analizzato è una versione semplificata di un'applicazione e-commerce di una azienda di acquisti alimentari online. I servizi web della piattaforma software sono allocati su due potenti server principali dell'infrastruttura cloud privata, chiamati *Server A* e *Server B*.

Il Server A è un sistema multi-core che esegue vari servizi del Front-End, come:

- autenticazione del cliente
- processi amministrativi e CRM
- interazione con il servizio di pagamento
- operazioni di checkout con aggiornamento del database
- generazione di fatture
- servizi di spedizione e tracciamento degli ordini
- aggiornamento dei dati del cliente

Il Server B è un sistema blade multiprocessore, altamente scalabile, tollerante ai guasti, con configurazioni ridondanti per garantire una disponibilità continua ed equipaggiata con grandi quantità di memoria RAM e di storage SSD dedicato alla gestione dei database.

Tra i servizi più importanti ci sono:

- navigazione del catalogo
- elaborazione del carrello
- gestione del database di prodotti e clienti.

Per minimizzare il tempo di risposta verso i clienti, viene implementato un database in-memory utilizzato per memorizzare dinamicamente nella cache gli acquisti più recenti di ciascun cliente.

Un terzo server, il **Server P**, viene utilizzato per i servizi di pagamento. Esso è locato in un data center di un provider esterno.

A causa della complessità intrinseca nel mappare un workflow su un modello di rete di code, si riscontrano difficoltà quando una richiesta, una volta servita da una stazione, ritorna alla stessa e richiede tempi di servizio e routing molto diversi rispetto a quelli richiesti nelle interazioni precedenti. Per gestire correttamente questa eterogeneità comportamentale, ad ogni richiesta in esecuzione viene assegnato un *Class ID*. Questo identificativo consente di descriverne il comportamento e le caratteristiche, come tempo di servizio medio richiesto, distribuzione dei tempi di risposta e per l'appunto, come deve essere gestito il routing all'interno del sistema.

In conformità con la logica aziendale dell'applicazione e-commerce, l'esecuzione completa di una richiesta richiede:

- tre visite al server A
- una al server B
- una al server P

Ad ogni visita al server A, vengono eseguiti diversi servizi web rispetto ai precedenti, perciò richiedono tempi di servizio medi diversi. La sequenza delle visite durante un'esecuzione corretta e completa è

A – B – A – P – A. Attraverso l'utilizzo dei Class IDs, è possibile modellare questo routing deterministico delle richieste tra i server.

Problematiche del sistema

Per ogni ordine completato con successo, il flusso di esecuzione richiede tre accessi obbligatori al server A. Questa caratteristica comporta un incremento significativo del tempo medio di permanenza presso tale nodo al crescere del traffico. L'elevata variabilità dei tempi di servizio, dovuta al fatto che ciascuna visita al server A comporta operazioni differenti, amplifica ulteriormente l'impatto dei picchi di carico sui tempi di risposta complessivi. Se non gestito in modo adeguato, il server A può quindi diventare un collo di bottiglia, riducendo il throughput del sistema.

Il server B rappresenta invece un collo di bottiglia certo: nei periodi di picco la sua utilizzazione raggiunge il 96% e, poiché le richieste che vi transitano presentano il tempo di servizio medio più elevato tra tutte, costituisce il limite massimo teorico per il throughput complessivo.

Poiché ogni server adotta la disciplina **Processor Sharing**, le richieste non vengono rifiutate e non si accumulano in una coda separata: tutte le richieste attive condividono simultaneamente la risorsa. Tuttavia, quando il numero di richieste aumenta, ciascuna riceve una frazione minore di capacità di servizio, con conseguente rallentamento generalizzato. Questo meccanismo può avere effetti a catena sui nodi successivi, peggiorando le prestazioni end-to-end.

In condizioni di carico elevato, la condivisione della capacità dovuta alla presenza di numerose richieste comporta un marcato incremento dei tempi di permanenza. Se il tasso di arrivo complessivo supera la capacità effettiva di servizio del nodo, il sistema può entrare in uno stato di instabilità, caratterizzato da tempi di risposta che crescono indefinitamente.

Un'ulteriore criticità riguarda il server P, che essendo un server di un provider esterno, non è considerabile come una risorsa migliorabile all'interno del sistema. Inoltre, i QoS imposti dai servizi di pagamento sono molto meno tolleranti a quelli relativi ad attività normali.

Anche il routing deterministico adottato contribuisce a generare correlazioni tra le visite multiple al nodo A: se il server risulta congestionato alla prima visita, è altamente probabile che lo sia anche nelle successive.

Obiettivi

Nelle applicazioni di e-commerce come quella analizzata, i parametri di qualità del servizio (QoS) devono garantire tempi di risposta rapidi e un'elevata disponibilità del sistema. In generale, il tempo di risposta end-to-end per le operazioni comuni dovrebbe mantenersi al di sotto di una determinata soglia di tolleranza, mentre per azioni critiche, come la conferma del pagamento, è auspicabile che il tempo sia notevolmente inferiore rispetto a quello richiesto per attività ordinarie.

Per quanto riguarda l'esperienza utente, studi di settore mostrano che un acquisto online di prodotti alimentari viene completato mediamente in un intervallo compreso tra 5 e 10 minuti da parte di utenti abituali, mentre può richiedere fino a 10-20 minuti per utenti meno esperti. Questi tempi comprendono le fasi di ricerca e selezione dei prodotti e dipendono in larga misura dal comportamento dell'utente stesso. Tuttavia, i tempi di risposta del sistema incidono in modo significativo sul tasso di completamento degli ordini. Per questa ragione, l'analisi condotta considera trascurabile il *think time* dell'utente, concentrandosi invece sugli aspetti intrinseci del sistema.

Il tempo di processamento percepito dall'utente, costituisce un fattore critico. In prossimità di una determinata soglia durante il processo di checkout può infatti generare frustrazione e aumentare sensibilmente il rischio di abbandono, che cresce in modo drastico oltre tale soglia.

L'Analisi complessiva si concentra nel raggiungimento dei seguenti obiettivi principali:

- **Modello base:**

- **Capacity planning del data center:** Valutare la capacità attuale del data center rispetto al carico di lavoro corrente, focalizzandosi anche sull'impatto potenziale dell'implementazione di un sistema di autenticazione a due fattori sulle prestazioni e sulla scalabilità dell'infrastruttura. È necessario analizzare le differenze in termini di tempo di risposta del sistema e la quantità di job gestiti.
- **Performance evaluation in seguito ad un nuovo workload** L'acquisizione di una nuova compagnia di grocery online comporterà un aumento stimato del carico di lavoro pari a circa il 15%. In termini di tassi di arrivo, questo corrisponde a un valore massimo di 5000 richieste all'ora (req/h). L'analisi del sistema sarà quindi condotta tenendo conto di questi nuovi parametri.

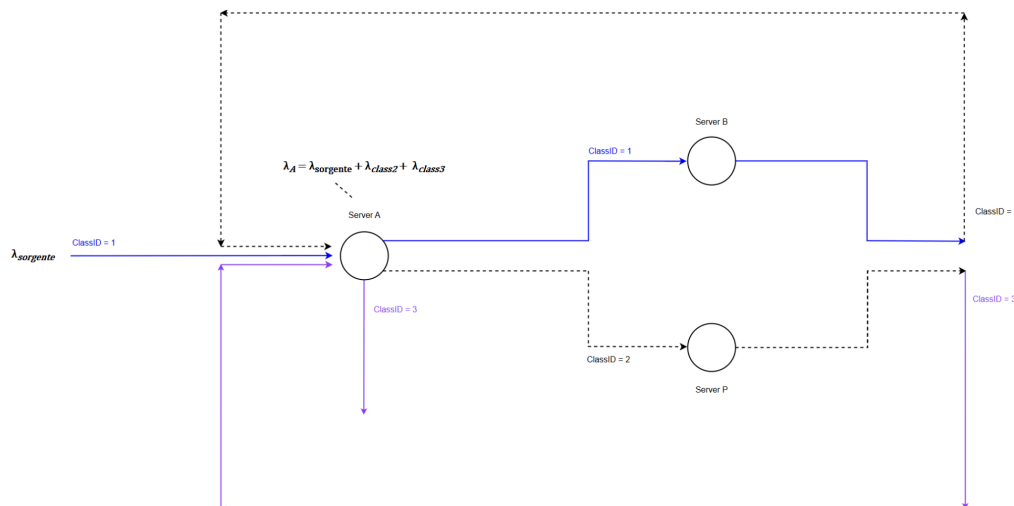
- **Modello migliorativo:**

- **Rispettare i requisiti di QoS** introdotti in termini di tempo medio di risposta del sistema e di ciascuna tappa.
- **Minimizzare le perdite del sistema** in relazione agli eventi di perdita aggiunti.

Modello concettuale

Ai fini dell'analisi, non è rilevante identificare chi siano gli utenti del sistema. Ogni richiesta proveniente da un singolo utente è stata modellata come un *job*; d'ora in poi, utilizzeremo il termine *job e/o richiesta*.

Il sistema oggetto di analisi è rappresentato dal seguente *System Diagram*:



Politiche di scheduling delle request (job) all'interno dei server:

Nel modello adottato, ogni server implementa una politica di servizio di Processor Sharing (PS) puro. Tutti i job presenti nel sistema ricevono immediatamente una quota identica della capacità computazionale, suddivisa in modo equo e simultaneo. Non esiste quindi distinzione tra job in servizio e job in attesa: ogni richiesta comincia ad avanzare non appena entra nel sistema e progredisce parallelamente alle altre, con velocità proporzionale alla capacità complessiva del server.

Il funzionamento della politica di servizio adottata è il seguente: ogni volta che un job arriva presso una stazione, quest'ultima può trovarsi in uno dei due stati possibili: idle o busy.

Se la stazione è idle, significa che non vi sono job attualmente in servizio. In tal caso, il job in arrivo può utilizzare in modo esclusivo l'intera capacità del server.

Quando successivamente un nuovo job entra in servizio, la capacità del server viene suddivisa equamente tra tutti i job in esecuzione. Se, ad esempio, sono presenti n job in servizio, ciascuno riceve una frazione pari a c/n della capacità complessiva del server.

In quel momento, ogni job mantiene un tempo residuo di servizio (remaining time) che viene aggiornato proporzionalmente alla capacità disponibile, ossia viene ridotto con tasso (c/n) .

Analogamente, quando un job termina il proprio servizio ed esce dalla stazione, la capacità totale del server viene distribuita tra i job rimanenti. Di conseguenza, se prima ciascun job riceveva (c/n) capacità computazionale, dopo l'uscita di un job ciascun job residuo riceverà una quota maggiore pari a $(c/(n-1))$.

Questa assunzione elimina i fenomeni legati a buffer limitati o a politiche di scheduling sequenziali, come l'head-of-line blocking e attesa in coda, e consente una modellazione analitica più semplice e astratta. La congestione del sistema si manifesta esclusivamente come incremento dei tempi di permanenza (sojourn time), che cresce con il numero di job concorrenti e può divergere se il tasso di arrivo supera la capacità di servizio del nodo.

Infine, l'approccio basato su Processor Sharing puro permette di ricondurre il sistema alle ipotesi formulate nelle specifiche iniziali, assicurando la consistenza del modello teorico e la validità delle analisi condotte.

Eventi modellati

Poiché la logica di servizio di tutte le stazioni del sistema in analisi adotta la disciplina Processor Sharing (PS), l'inizio del servizio di un job presso una stazione coincide con il suo momento di arrivo alla stazione stessa.

In questa politica di servizio, non esiste una fase di attesa separata: ogni job che giunge al centro di servizio entra immediatamente in esecuzione, condividendo la capacità del server con gli altri job già presenti.

Analogamente, quando un job termina il proprio servizio presso una stazione, esso inizia istantaneamente il servizio presso la stazione successiva del percorso previsto dal modello.

In tal modo, la dinamica del sistema è governata da un flusso continuo di job che avanzano attraverso i diversi centri senza tempi morti tra una fase di servizio e la successiva.

Ai fini della modellazione della rete di code relativa all'applicazione di e-commerce, gli eventi principali considerati per tutti i centri del sistema sono dunque:

- **Arrivo di un job presso una stazione** (che coincide con l'inizio del servizio).
- **Completamento di un job presso una stazione** (che coincide con il suo arrivo alla stazione successiva o con l'uscita dal sistema).

Descrizione degli eventi

Arrivo di un job presso una stazione

La capacità del server viene suddivisa equamente tra tutti i job attivi: se al momento dell'arrivo ci sono n job nel sistema (compreso quello nuovo), ciascun job riceve un tasso di servizio pari a

$$\mu_{job} = \frac{c}{n}$$

dove c è la capacità complessiva del server (normalizzata a 1 per semplicità). Il tempo di inizio servizio di ogni job viene registrato in base a due possibili scenari:

1. Se il job è in una fase intermedia del suo ciclo di vita all'interno del sistema, l'inizio del servizio coincide con l'istante in cui completa l'elaborazione presso la stazione precedente.
2. Altrimenti, se il job è appena arrivato nel sistema, il tempo di inizio servizio corrisponde all'istante di arrivo.

Completamento di un job presso una stazione

Viene calcolato il prossimo evento di completamento, in base ad una stima. Tutti i job con tempo rimanente minore di una tolleranza epsilon vengono considerati completati e rimossi dalla lista dei job attivi. Di questi, viene salvato un record dettagliato del servizio per quella stazione. Una volta fatto questo, il job viene indirizzato alla stazione successiva o diretto all'uscita del sistema in caso avesse raggiunto la stazione finale.

Stato del Sistema

Lo stato complessivo del sistema è dato dall' unione degli stati dei singoli centri di servizio. Ciascun centro adotta una disciplina di servizio Processor Sharing (PS), per cui il suo stato è descritto dal numero di richieste attualmente in servizio presso di esso.

Lo stato del sistema è rappresentato dalla seguente tupla $[(n_{a1}, n_{a2}, n_{a3}), n_b, n_p]$ in cui:

Server A

Il server A può servire contemporaneamente richieste appartenenti a più classi.

Pertanto, il suo stato è rappresentato da una tupla (n_{a1}, n_{a2}, n_{a3}) , dove:

- n_{a1} indica il numero di job di **classe 1 nel server A**
- n_{a2} indica il numero di job di **classe 2 nel server A**
- n_{a3} indica il numero di job di **classe 3 nel server A**

Ciascun job è inoltre associato a un identificatore di classe (*classID*), che ne rappresenta lo stato di avanzamento all'interno del sistema.

Server B

Il server B gestisce esclusivamente job di classe 1.

Di conseguenza, il suo stato è descritto da un singolo valore:

- n_b : numero di richieste di classe 1 attualmente in servizio presso B.

Server P

Analogamente, il server P gestisce unicamente job di classe 2.

Il suo stato è quindi rappresentato dal numero:

- n_p : numero di richieste di classe 2 attualmente in servizio presso P.

Inoltre, i server del sistema possono trovarsi in due stati distinti:

- **idle**: nessun job presente;
- **busy**: uno o più job attivi, che condividono equamente la capacità di servizio secondo la politica di Processor Sharing.

Modello delle specifiche

Distribuzioni

Le distribuzioni utilizzate nel nostro sistema sono esponenziali, con parametro di arrivo variabile nell'intervallo $\lambda \in [0.5, 1.4]$.

Tutti i server adottano un tempo di servizio estratto da una distribuzione esponenziale, ciascuno di parametro μ_i con $i \in [A_1, B, A_2, P, A_3]$.

Modellazione dei centri

- **Server A - Front end (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è **caratterizzato da**:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale
- 1 server
- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo complessivo** in entrata al server A è

$$\lambda_A = \lambda_{sorgente} + \lambda_{class2} + \lambda_{class3}$$

dove:

- $\lambda_{sorgente} \in [0.5, 1.2]$
- λ_{class2} *descrive l'intensità degli arrivi generati dal server B*
- λ_{class3} *descrive l'intensità degli arrivi generati dal server P*
- $\lambda_{sorgente}$ *corrisponde agli arrivi di classID = 1*

Tassi di servizio: seguono una distribuzione esponenziale e dipendono dal classID associato al job che entra in servizio:

- $E[S]_{A_1} = 0.2 \text{ secondi per i job di classID} = 1$
- $E[S]_{A_2} = 0.4 \text{ secondi per i job di classID} = 2$
- $E[S]_{A_3} = 0.1 \text{ secondi per i job di classID} = 3$
- $E[S]_{A_3} = 0.15 \text{ secondi per i job di classID} = 3 \text{ (caso two - factor Authentication)}$

- **Server B - Web App, DB (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è **caratterizzato da**:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale
- 1 server
- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo complessivo** in entrata al server B è

$$\lambda_B = \lambda_{sorgente}$$

dove:

$$\lambda_{sorgente} = \lambda_{class1}$$

Tasso di servizio: questo segue una distribuzione esponenziale.

- $E[S]_B = 0.8 \text{ secondi per i job di classID} = 2$

- **Server P - Payment Processing (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è **caratterizzato da**:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale

- 1 server
- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo complessivo** in entrata al server P è

$$\lambda_p = \lambda_{class2}$$

Tasso di servizio: questo segue una distribuzione esponenziale.

- $E[S]_p = 0.4 \text{ secondi per i job di classID} = 3$
- $E[S]_p = 0.7 \text{ secondi per i job di classID} = 3 \text{ (caso two - factor Authentication)}$

Modello computazionale

Algoritmo Next event simulation

La simulazione adotta l'approccio della Next-Event-Simulation, che consiste nel far avanzare il tempo processando gli eventi in ordine di arrivo. In altre parole, l'evoluzione del sistema avviene solo quando si verifica un evento che modifica lo stato. Gli eventi considerati sono comuni a tutti i server (A, B, P), anche se cambiano le condizioni che li generano.

I due tipi di eventi sono:

Arrivo (Arrival) presso una stazione: rappresenta l'ingresso di un job in un server.

Quando un evento di arrivo viene selezionato come il prossimo da simulare, il tempo globale della simulazione viene prima aggiornato fino all'istante dell'arrivo, così che tutti i server riflettano correttamente il lavoro eseguito fino a quel momento.

Al verificarsi dell'arrivo presso una stazione, la simulazione gestisce l'evento in base al tipo di server coinvolto, distinguendo tre casi principali:

- **Arrivo al server A:**

Se l'arrivo riguarda il server A come prima stazione del percorso, si tratta di un job **proveniente dalla sorgente**. In questo caso viene creato un nuovo job, a cui viene assegnato un identificativo

univoco. Si registra il tempo d'arrivo e si inizializzano le informazioni di base associate al job. Il job viene poi immediatamente ammesso al server A.

- **Arrivo al server B o P:**

Gli arrivi a B e P sono il **risultato del completamento del servizio in una stazione precedente**. In particolare, un job viene trasferito a B al termine della prima visita presso A, e a P al termine della seconda visita presso A. In entrambi i casi, il job mantiene il proprio identificativo ed entra immediatamente in servizio senza tempi di attesa.

In generale, al momento dell'ammissione in qualunque stazione:

- viene estratta una **domanda di servizio (service demand ovvero il tempo che dovrebbe impiegare il job i esimo ad essere processato se il server fosse vuoto, perciò avrebbe piena capacità)** da una distribuzione esponenziale specifica per quella stazione (di parametro $1/\mu$);
 - si registrano il tempo d'ingresso, il valore corrente del service **tag** del server (che rappresenta la quantità totale di servizio erogato fino a quel momento) e il numero di job attivi al momento dell'ammissione;
 - viene calcolato il **finish tag** del job, cioè il valore del service tag al quale il job terminerà il proprio servizio;
 - il job viene inserito nella **coda di priorità (heap)** del server, che tiene traccia dei prossimi completamenti.
- Infine, nel caso di arrivo di un nuovo job che visita per la prima volta il server A, viene pianificato il **prossimo arrivo dalla sorgente** nel sistema, generando un nuovo intervallo esponenziale e aggiornando l'istante relativo al prossimo arrivo, a meno che non si sia superato il tempo massimo di simulazione.

Evento di Completamento (Completion) presso una stazione

L'**evento di completamento** rappresenta il completamento del servizio di un job su un determinato server. Quando questo evento viene selezionato come il prossimo da simulare, il tempo globale della simulazione viene aggiornato fino all'istante del completamento, garantendo che i *service tag* e le statistiche temporali di tutti i server riflettano correttamente l'evoluzione del sistema fino a quel momento.

Il completamento viene determinato estraendo il job con il *finish tag* minimo, ossia quello il cui tempo di servizio virtuale è stato interamente erogato. Una volta individuato, il job viene rimosso dal server (riducendo di 1 il numero di job attivi presso tale stazione) e vengono calcolate le grandezze associate alla visita appena conclusa, come il tempo di risposta.

A seconda della stazione in cui si verifica il completamento, la simulazione gestisce il proseguimento del job in modo differente:

- **Completamento presso il server A:**

Il server A compare più volte nel percorso (A_1, A_2, A_3), pertanto la gestione dell'evento dipende dalla posizione della visita:

- Dopo la **prima visita (A_1)**, il job viene immediatamente trasferito al server **B**.
- Dopo la **seconda visita (A_2)**, il job prosegue verso il server **P**.
- Dopo la **terza visita (A_3)**, il job ha completato tutte le fasi del percorso e **lascia il sistema**.

In tutti i casi, il trasferimento è istantaneo e il job entra immediatamente in servizio presso la stazione successiva.

- **Completamento presso il server B:**

Quando un job completa il servizio su B, viene **inoltrato nuovamente al server A** per la seconda visita (A_2). Il completamento su B comporta inoltre l'aggiornamento del numero di job attivi sul server, la registrazione dei tempi di ingresso e uscita della visita, e il salvataggio dei dati statistici relativi alla domanda e al tempo di risposta effettivo.

- **Completamento presso il server P:**

Dopo il completamento del servizio sul server P, il job viene **trasferito al server A per la terza e ultima visita (A_3)**. Anche qui l'ingresso in servizio avviene immediatamente, e il sistema aggiorna coerentemente lo stato del server P e le informazioni sul job.

Se invece la visita appena conclusa corrisponde all'ultima (A_3), il job viene rimosso dal sistema e ne viene registrato il **tempo di risposta totale**, ottenuto come differenza tra l'istante di completamento e quello di arrivo iniziale.

In generale, ad ogni completamento:

- il server libera una posizione di servizio aggiornando il numero di job attivi;
- vengono registrati i tempi di ingresso (*IN*), uscita (*OUT*), il tempo di servizio richiesto ed il tempo di risposta effettivo;

- viene deciso se il job deve proseguire verso una stazione successiva o se deve essere rimosso definitivamente dal sistema.

Come linguaggio di programmazione è stato scelto Python. Le ragioni sono molteplici: una gestione della memoria implicita, l'elevata flessibilità e le librerie ad hoc per questo tipo di simulazione messe a disposizione dal linguaggio stesso. Inoltre, le librerie originariamente scritte in C sono state riscritte in Python, prendendo come riferimento gli esempi presenti nel libro, al fine di gestire il generatore di Lehmer e le relative distribuzioni statistiche in modo appropriato.

Poiché il nostro sistema prevede tre server tutti in modalità processor sharing, l'implementazione di questo avviene secondo la modalità **Tag-based Next-Event-Simulation**.

La **Tag-based Next-Event-Simulation**, una variante della simulazione event-driven, in cui gli eventi futuri non vengono generati come eventi temporali espliciti, quindi con tempo reale di completamento, ma tramite service tags che rappresentano il progresso cumulativo di una stazione nel tempo virtuale di servizio. La scelta di questa variante è motivata dalla capacità della Tag-based di simulare molti più job. Infatti, nella simulazione classica in cui uno o più server adottano una politica di servizio Processor Sharing, il sistema deve continuamente aggiornare i tempi residui di servizio di tutti i job attivi presso ciascuna stazione. Nella Tag-based, un singolo completamento richiede $O(\log n)$ per l'estrazione dall'heap. Il tempo avanza saltando direttamente al prossimo evento, senza micro-passaggi e non si calcolano più i tempi residui per ogni job, alleggerendo la computazione.

In particolare, per una stazione che adotta una politica Processor Sharing:

- Ogni job ha un tempo di servizio D_i
- Ogni server mantiene un service tag $S(t)$ che avanza nel tempo in modo proporzionale alla potenza di calcolo.
- Ogni job ha un finish tag $F_i = S(t_i^{admit}) + D_i$.
- Il completamento avviene quando $S(t) \geq F_i$.

In questo modo, non è necessario simulare il servizio "continuo" di ogni job ma si possono calcolare in modo analitico e discreto i tempi di completamento successivi.

Implementazione arrivo di un nuovo job:

Prima dell'arrivo, esiste un contesto iniziale, fornito da una variabile *next_arrival* calcolata attraverso *Exponential(mean_inter)*. L' algoritmo trova il prossimo evento t_{next} come il valore più piccolo tra

$next_arrival$ e $next_comp_time$) e chiama $advance(t_next - t)$ per aggiornare in modo coerente lo stato temporale del sistema prima di processare l'evento. Questa funzione esegue tre operazioni fondamentali:

- **Aggiornamento del tempo accumulato di occupazione dei server:** per ciascun server srv , la variabile $srv.N_time_accum$ viene incrementata di $srv.n * delta$. Questo termine rappresenta l'integrale di $N(t)$ nel tempo, cioè la quantità di tempo totale in cui il server ha avuto n job attivi durante l'intervallo considerato. Esso è essenziale per il calcolo successivo dei valori medi, come $AvgN$.
- **Avanzamento del service tag S :** nei server PS, ogni job riceve servizio simultaneamente, ma la velocità effettiva di accumulo di servizio per ciascun job è ridotta di un fattore proporzionale al numero di job attivi. Il *service tag* S del server misura la quantità di servizio "erogata" in unità di capacità normalizzata. Durante un intervallo di durata $delta$, se il server ha n job attivi, il tag aumenta di: $\Delta S = \frac{c}{n} \times \Delta t$, dove c rappresenta la capacità del server. Questo aggiornamento permette di determinare, in ogni istante, quanto servizio è stato cumulativamente distribuito ai job. Poiché ogni job possiede un *finish tag* $F_i = S(t_i^{admit}) + D_i$, il confronto tra S e F consente di stabilire esattamente quando un job termina, senza simulare l'evoluzione continua del servizio.
- **Avanzamento del tempo globale:** infine, il tempo simulato viene incrementato di $delta$, portando il sistema all'istante del prossimo evento.

Al fine di valutare la correttezza dell'approccio proposto, segue un breve esempio.

Scenario iniziale:

- Server PS con capacità $c = 1.0$
- Tempo iniziale $t = 0$
- Nessun job in servizio, quindi $n = 0$, $S = 0$

Arrivo del primo job J_1 :

- J_1 arriva a $t = 0$
- Viene immediatamente ammesso nel server
- Si estrae un tempo di servizio $E[S]_1 = 6$ unità di servizio.
- Lo snapshot del server al momento dell'ammissione è $S_{admit} = 0$
- Calcolo del finish tag: $F_1 = S_{admit} + E[S]_1 = 0 + 6 = 6$

J_1 entra quindi nell'head con finish tag = 6 e $n = 1$.

Avanzamento fino al prossimo evento:

Al momento non ci sono altri job, quindi il server lavora esclusivamente su J_1 , il service tag cresce con velocità $c/n = 1$. L'evento successivo avverrà quando J_1 terminerà, cioè quando $S = F_1 = 6$. L'intervallo richiesto è quindi uguale ancora a $\Delta t = 6$.

Arrivo di un secondo job durante il servizio di J1:

Supponiamo che J_2 arrivi a $t = 2$, mentre J_1 è ancora in servizio.

1. **Prima dell'arrivo**, si avanza fino a $t = 2$: $\Delta t = 2$, $n = 1$, quindi:

a. $S = 0 + 1 \times 2 = 2$

b. $t = 2$

2. **Ammissione di J2:**

a. $S_{\text{admit}} = 2$

b. $E[S]_2 = 4$

c. $F_2 = 2 + 4 = 6$

d. Inseriamo J_2 nell'heap

e. $n = 2$

Nell'heap ora abbiamo: $J_1 : F_1 = 6$, $J_2 : F_2 = 6$

Prossimo evento:

Dall'istante $t = 2$, entrambi i job condividono equamente la capacità.

Il server avanza con velocità per job pari alla capacità diviso $n = 2$. Ogni incremento di 1 unità di tempo reale corrisponde ad un incremento di 0.5 nel tag S . Poiché il tag attuale è $S = 2$, e il primo finish tag tra i job è 6, abbiamo bisogno di un incremento $\Delta S = 4$. Il tempo necessario per avere tale incremento è

$$\Delta t = \frac{\Delta S}{c/n} = \frac{4}{0.5} = 8. \text{ Quindi, il prossimo evento avverrà a } t = 2 + 8 = 10.$$

Validazione

Andiamo a vedere se i job hanno ricevuto la quantità di servizio corretta:

- $t \in [0, 2]$: solo $J_1 \rightarrow$ riceve 2 unità di servizio.
- $t \in [2, 10]$: entrambi attivi $\rightarrow$ ciascuno riceve $(10 - 2) * 0.5 = 4$ unità.

Totale per $J_1 = 2 + 4 = 6 = E[S]_1$

Totale per $J_2 = 4 = E[S]_2$

Entrambi completano correttamente a $t = 10$, esattamente quando $S = 6$.

Questo esempio mostra la **correttezza del meccanismo tag-based**:

- Ogni job compete per la risorsa proporzionalmente al numero di job attivi.
- Il *service tag* S cresce esattamente come previsto dal modello PS.
- Non è necessario aggiornare continuamente ogni job, poiché i *finish tag* determinano con precisione l'ordine degli eventi.
- La funzione *advance(delta)* garantisce che i valori di S , N_{time_accum} e t restino coerenti con la storia del sistema.

La funzione *advance(delta)* è definita nel seguente modo:

```
def advance(delta):
    nonlocal t
    if delta <= 0:
        return
    # accumulo N(t)*dt
    for srv in servers.values():
        srv.N_time_accum += srv.n * delta
    # avanzamento dei service tag
    for srv in servers.values():
        if srv.n>0:
            srv.S += (srv.c / srv.n) * delta
    t += delta
```

L'evento di arrivo è implementato nel seguente modo:

```
if abs(t_next - ta) <= 1e-12:
    # ARRIVO
    selectStream(STREAM_ARR)
    job_id += 1
    jid = job_id
    jobs_info[jid] = {"arrival0": t, "stage_recs": [], "demand_total": 0.0}
    admit_segment(jid, 1, stage_server_name(1), t)

    ia = Exponential(mean_inter)
    cand = t + ia
    next_arrival = cand if cand <= threshold_time else None
```

Questo blocco viene eseguito ogni volta che l'evento successivo della simulazione è un arrivo (cioè quando t_next coincide con $next_arrival$). La simulazione confronta il tempo del prossimo evento (t_next) con il tempo del prossimo arrivo (ta). Se i due valori coincidono (entro una piccola tolleranza numerica), significa che l'evento da processare è un nuovo arrivo di job. Ogni job riceve un identificativo progressivo (job_id). Viene poi inizializzata una struttura ($jobs_info[jid]$) che memorizza tutte le informazioni relative a quel job:

- arrival0: tempo di arrivo globale nel sistema;
- stage_recs: lista che conterrà le registrazioni delle visite alle diverse stazioni (A, B, P...);
- demand_total: accumulatore per la somma dei tempi di servizio nelle varie fasi.

La funzione admit_segment:

- genera la **domanda di servizio** (campionata da un'esponenziale con parametro μ_{A1});
- prende lo **snapshot del service tag** S_{admit} del server;
- calcola il **finish tag** $F = S_{admit} + D$;
- inserisce il job nel **min-heap** del server, dove i job sono ordinati in base al loro finish tag (cioè al "tempo virtuale" di completamento);
- incrementa il numero di job attivi nel server ($srv.n += 1$).

Nel caso di arrivo per la prima volta nel server A, il job in arrivo proverrà dalla sorgente, mentre negli altri casi, un arrivo presso una qualsiasi altra stazione corrisponde al completamento del servizio presso la stazione immediatamente precedente.

Infine, si campiona un nuovo **inter-arrival time** (tempo tra due arrivi consecutivi) da una distribuzione esponenziale con media $1/\lambda$. Il nuovo arrivo avverrà quindi al tempo:

$$t_{next_arrival} = t + ia.$$

Implementazione completamento presso una stazione:

L'evento di **completamento** rappresenta il momento in cui un job termina la propria visita presso un server *Processor Sharing (PS)*.

```
# COMPLETION su comp_srv
F_min, jid, seg = heapq.heappop(comp_srv.heap)
comp_srv.n -= 1

IN = seg["IN"]
OUT = t
Demand = seg["Demand"]
Wait = 0.0
Service = OUT - IN # tempo effettivo trascorso nella visita (PS)

# Valori "istantanei" all'ammissione
n_at_admit = seg["N_at_Admit"]
avgN_during = n_at_admit # tua scelta: riportiamo lo stesso valore
```

```

jobs_info[jid]["stage_recs"].append({
    "pos": seg["pos"],
    "server": seg["server"],
    "id_label": f"{jid}{{seg['a_visit']}}",
    "IN": IN, "OUT": OUT, "Demand": Demand, "Wait": Wait,
    "Service": Service,
    "N_at_Admit": n_at_admit,
    "AvgN_during": avgN_during
})
jobs_info[jid]["demand_total"] += Demand

# Prosegue o chiude
if seg["pos"] < 5:
    admit_segment(jid, seg["pos"]+1, stage_server_name(seg["pos"]+1), t)
else:
    # chiude job: aggrega le visite
    recs = sorted(jobs_info[jid]["stage_recs"], key=lambda r: r["pos"])
    row = {
        "OrigID": jid,
        "Arrival0": jobs_info[jid]["arrival0"],
        "Completion": t,
        "TempoRispostaTotal": t - jobs_info[jid]["arrival0"],
        "TempoServizioTotaleSellJobFosseSOlo": jobs_info[jid]["demand_total"]
    }
    for i, rec in enumerate(recs, 1):
        row.update({
            f"Server_{i}": rec["server"],
            f"ID_{i}": rec["id_label"],
            f"IN_{i}": rec["IN"],
            f"OUT_{i}": rec["OUT"],
            f"Demand_{i}": rec["Demand"],

            f"Response_Tlme_{i}": rec["Service"],
            f"N_at_Admit_{i}": rec["N_at_Admit"],
            f"AvgN_during_{i}": rec["AvgN_during"],
        })
    jobs_rows.append(row)

```

Quando il simulatore individua che il prossimo evento è un completamento, estrae dal *min-heap* del server (*comp_srv*) l'elemento con *finish tag* più basso, tramite *heapq.heappop(comp_srv.heap)*. Questo valore rappresenta il job che ha ricevuto abbastanza servizio affinché il tag del server *S* abbia raggiunto il suo *finish tag F*, quindi il job è completato. Successivamente, il numero di job attivi *comp_srv.n* viene decrementato di uno, poiché il job lascia il server.

A questo punto vengono registrati i dati temporali del segmento completato: l'istante di ingresso (*IN*), l'istante di completamento (*OUT* = tempo corrente *t*), la domanda di servizio assegnata e il tempo effettivo di permanenza nel server, calcolato come *OUT - IN*.

Le informazioni raccolte vengono memorizzate nella struttura *jobs_info[jid]["stage_recs"]*, che conserva le visite di ciascun job attraverso le diverse stazioni. In questo modo la simulazione tiene traccia della cronologia completa del job.

Terminato l'aggiornamento, il simulatore verifica se il job deve proseguire verso la prossima stazione: se la posizione *pos* del segmento è inferiore a 5, viene immediatamente chiamata la funzione *admit_segment(...)* per inserirlo nel server successivo al tempo corrente *t*. In caso contrario, il job viene considerato completato: le sue visite vengono aggregate e si costruisce una riga riepilogativa (*row*) con i tempi di arrivo, completamento e risposta totale.

Per semplicità di implementazione, il completamento di un job è determinato considerando una precisione numerica fissata: un job viene considerato completato quando il tempo rimanente (**remaining time**) scende al di sotto di una soglia $\varepsilon = 1 \times 10^{-6} s$

Verifica

I calcoli e le verifiche successive sono stati eseguiti sulla configurazione one-factor authentication con $\lambda = 1.2 \text{ req/s}$. Mantenendo questa configurazione, i requisiti saranno implicitamente soddisfatti anche per il two-factor, in quanto quest'ultima implica solo una variazione dei parametri relativi ai tempi di servizio non alterando il funzionamento del sistema.

Verifica Analitica

Per ottenere un'ulteriore verifica della simulazione e i dati riportati, abbiamo calcolato analiticamente le metriche e queste risultano essere in linea con la simulazione eseguita.

Analisi SERVER A

Assumiamo, come anticipato, che il sistema sia senza perdita. Il traffico in ingresso al server A è espresso come:

$$\lambda_A = \lambda_{sorgente} + \lambda_{class2} + \lambda_{class3} = 3 * \lambda_{sorgente} = 3.6 \text{ req/s}$$

Il tasso di arrivo λ_A è espresso come la **somma di tre contributi**: uno proveniente dalla **sorgente esterna** e due generati dagli **altri due server** del sistema.

Poiché nel modello base non sono previsti **eventi di perdita**, il **flusso in uscita** da ciascun server risulta **uguale al flusso in ingresso**. Di conseguenza, per ogni stazione, il tasso di arrivo coincide con il tasso di uscita, garantendo la **conservazione del traffico** all'interno del sistema.

Come specificato dalla tabella, il tempo di servizio richiesto da un job che attraversa il server A dipende dal suo classID:

$$E[S_1]_A = 0.2 \text{ sec} \quad E[S_2]_A = 0.4 \text{ sec} \quad E[S_3]_A = 0.1 \text{ sec}$$

Ogni job attraversa il server A per tre volte consecutive, appartenendo di volta in volta a una classe diversa e richiedendo un tempo di servizio distinto.

Si assume inoltre che le tre visite abbiano uguale probabilità di verificarsi a regime, ossia che ogni job completi sempre tutte le tre fasi di elaborazione.

Di conseguenza, il traffico generato da ciascuna classe risulta bilanciato, e il tasso complessivo di arrivo al server A può essere considerato equamente distribuito tra le tre classi.

Per tale motivo, il **tempo medio di servizio globale del server A** è stato rappresentato come **la media aritmetica dei tempi medi di servizio delle singole classi**, ponderata sul numero di visite equivalenti, uguale per ciascuna classe.

$$E[S]_A = \frac{0.2 + 0.4 + 0.1}{3} = 0.2\bar{3} \text{ sec}$$

$$E[S]_{A - \text{two factor authentication}} = \frac{0.2 + 0.4 + 0.15}{3} = 0.25 \text{ sec}$$

Poiché la politica di servizio è processor sharing, i tempi medi di risposta, globali e per singolo job e il numero di job medi in servizio dipendono direttamente dall'utilizzazione del server A.

$$E[T_S] = \frac{E[S]_A}{1 - \rho_A} = \frac{0.2\bar{3}}{1 - 0.84} = 1.45 \text{ s} \quad E[T_{Si}] = \frac{E[S_i]_A}{1 - \rho_A}$$

$$E[T_S]_{A1} = \frac{E[S_1]_A}{1 - \rho_A} = \frac{0.2}{1 - 0.84} = 1.25 \text{ s} \quad E[T_S]_{A3} = \frac{E[S_3]_A}{1 - \rho_A} = \frac{0.1}{1 - 0.84} = 0.625 \text{ s}$$

$$E[T_S]_{A5} = \frac{E[S_5]_A}{1 - \rho_A} = \frac{0.4}{1 - 0.84} = 2.5 \text{ s}$$

$$\rho_A = \frac{\lambda_A}{\mu_A} = E[S]_A * \lambda_A = 0.83\bar{9} = 84\%$$

$$E[N_S]_A = \frac{\rho_A}{1 - \rho_A} = 5.25$$

L'analisi e i calcoli effettuati per il server B e il server P sono del tutto analoghi e conformi a quanto specificato precedentemente.

Analisi SERVER B

$$\lambda_B = \lambda_{sorgente} = 1.2 \text{ req/s} \quad E[S]_B = 0.8 \text{ sec}$$

$$\rho_B = \frac{\lambda_B}{\mu_B} = E[S]_B * \lambda_B = 96\% \quad E[N_S]_B = \frac{\rho_B}{1 - \rho_B} = 24$$

$$E[T_S]_B = \frac{E[S]_B}{1 - \rho_B} = \frac{0.8}{1 - 0.96} = 20 \text{ s}$$

Analisi SERVER P

$$\lambda_P = \lambda_{class2} = \lambda_{sorgente} = 1.2 \text{ req/s} \quad E[S]_P = 0.4 \text{ sec nel caso one-factor auth.}$$

$$\rho_P = \frac{\lambda_P}{\mu_P} = E[S]_P * \lambda_P = 48\% \quad E[N_S]_P = \frac{\rho_P}{1 - \rho_P} = 0.923076$$

Di seguito, vengono riportate le tabelle con tutte le metriche calcolate per i casi simulati con il modello base presentato dalla specifica.

Risultati ottenuti

Di seguito è riportata una **tabella riassuntiva** contenente i tempi di servizio medi richiesti da ciascun job e i corrispondenti tassi di servizio utilizzati nella simulazione.

Stazioni	Classe / ClassID - Caso One Factor Authentication					
	1		2		3	
Server A	E[S] = 0.2 s	$\mu_{A_1} = 5 \text{ req/s}$	E[S] = 0.4 s	$\mu_{A_2} = 2.5 \text{ req/s}$	E[S] = 0.1 s	$\mu_{A_3} = 10 \text{ req/s}$
Server B	E[S] = 0.8 s	$\mu_B = 1.25 \text{ req/s}$	N.D.		N.D.	
Server P	N.D.		E[S] = 0.4 s	$\mu_P = 2.5 \text{ req/s}$	N.D.	

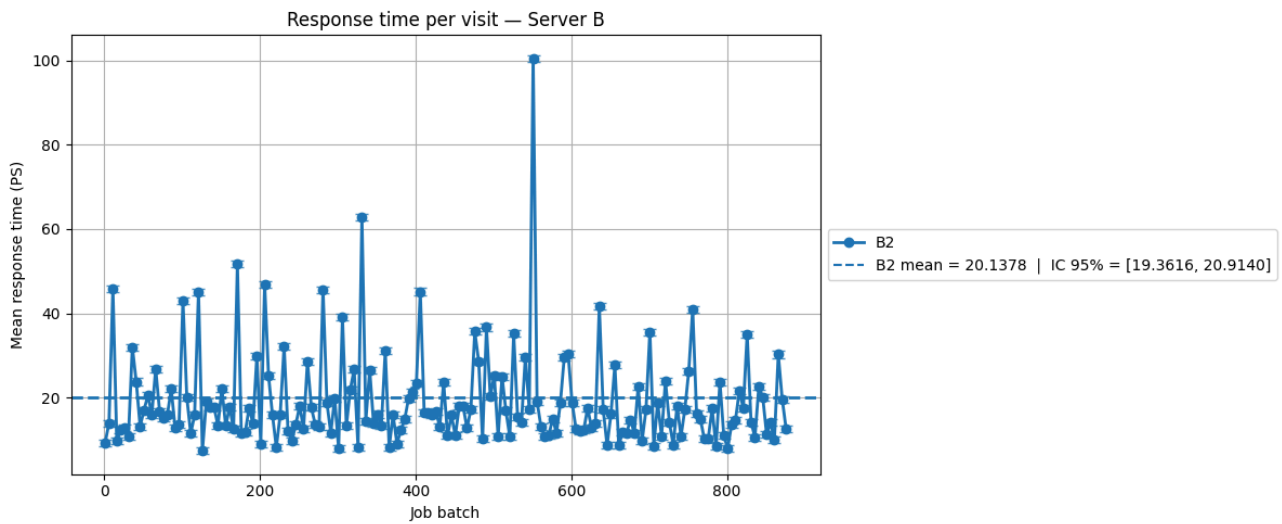
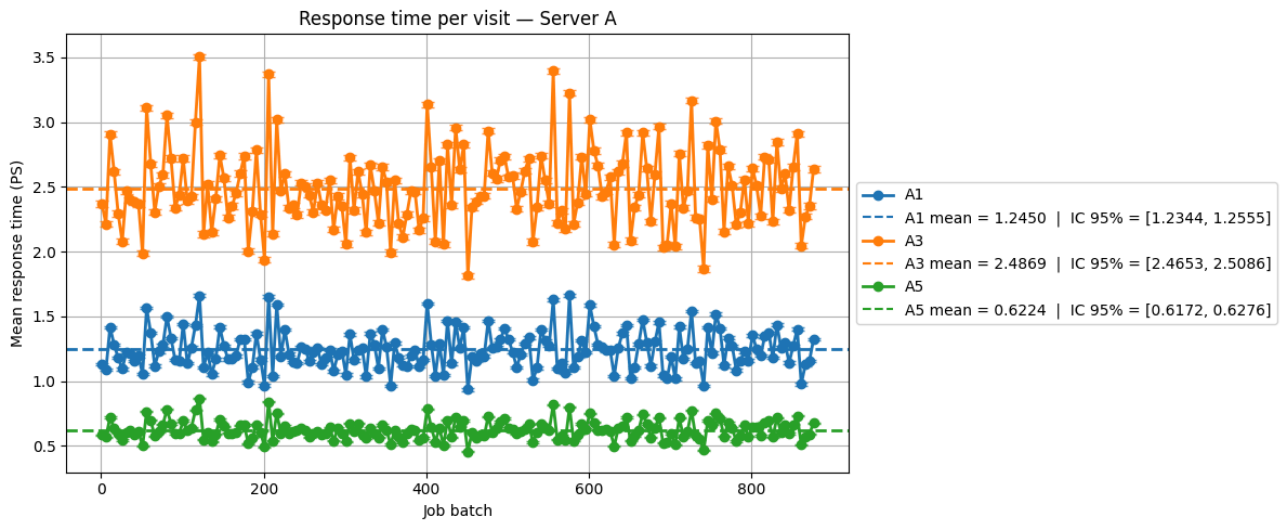
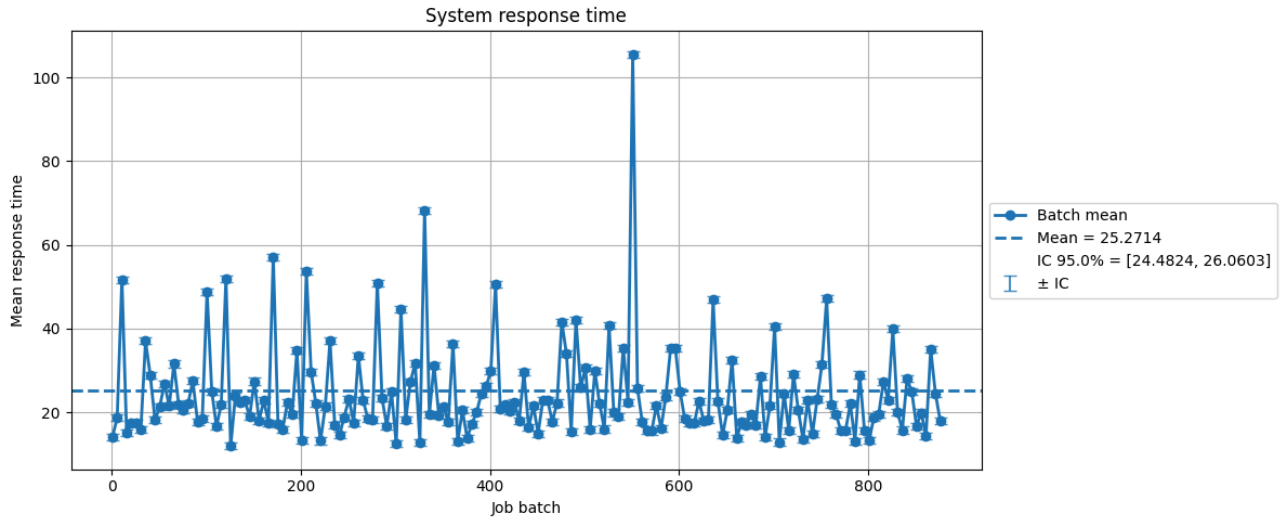
La simulazione è stata eseguita utilizzando il **seed** 123456789. È stata condotta **una singola run**, mantenendo invariato il seed per garantire la riproducibilità dei risultati.

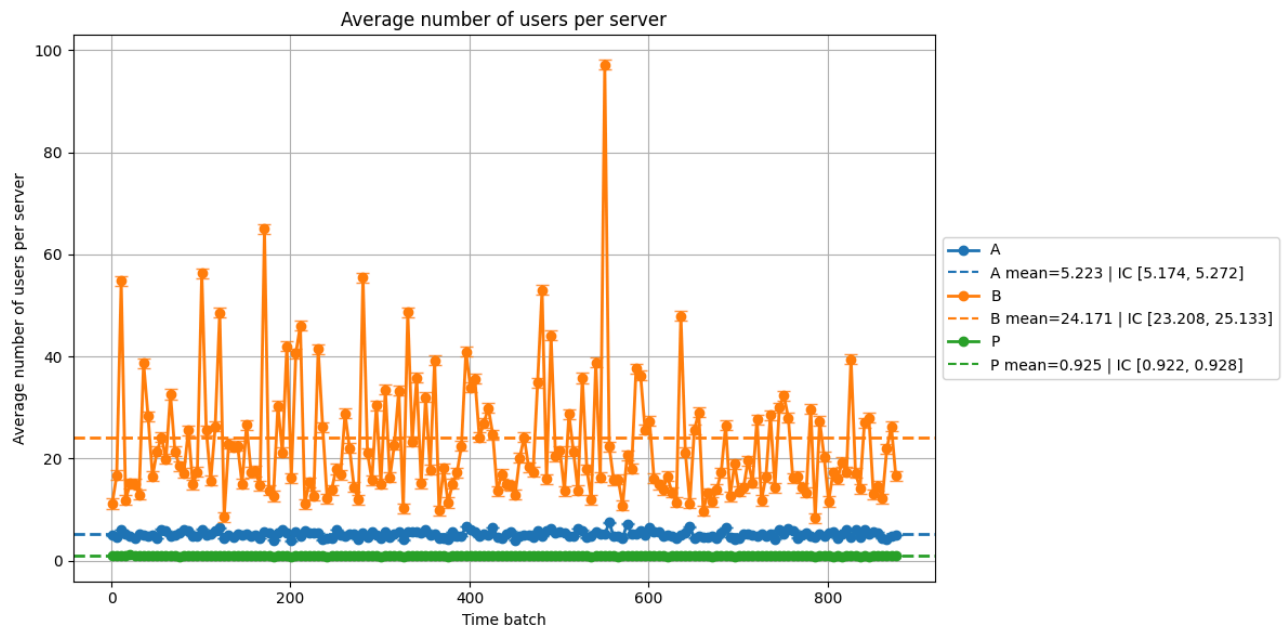
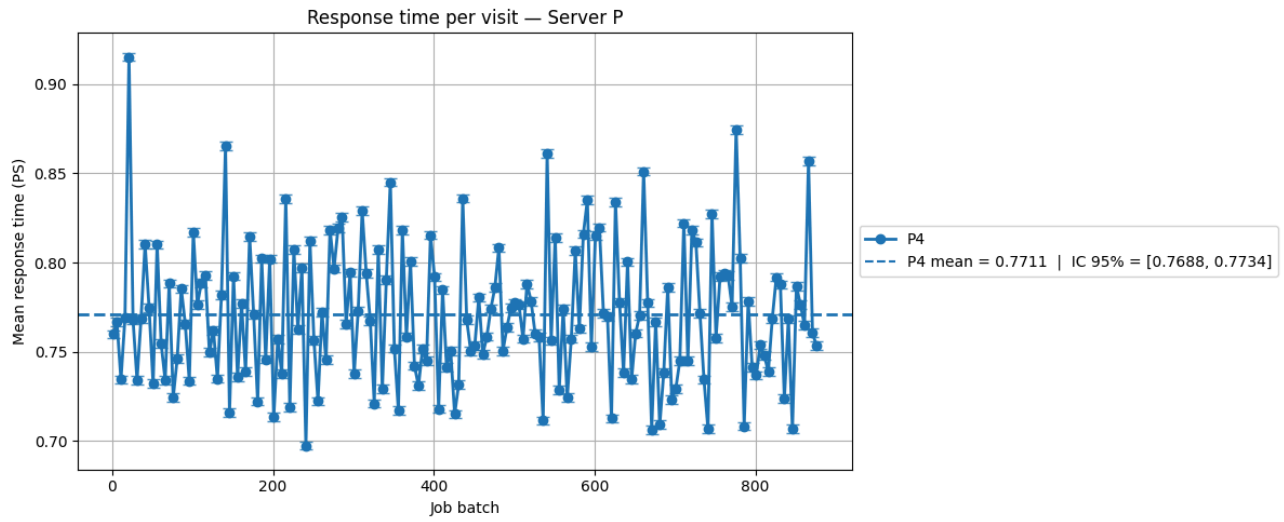
Gli intervalli di confidenza e l'esecuzione (run) condotta con il metodo Batch Means sono stati calcolati e illustrati nel capitolo "Progettazione degli esperimenti", a pagina 31.

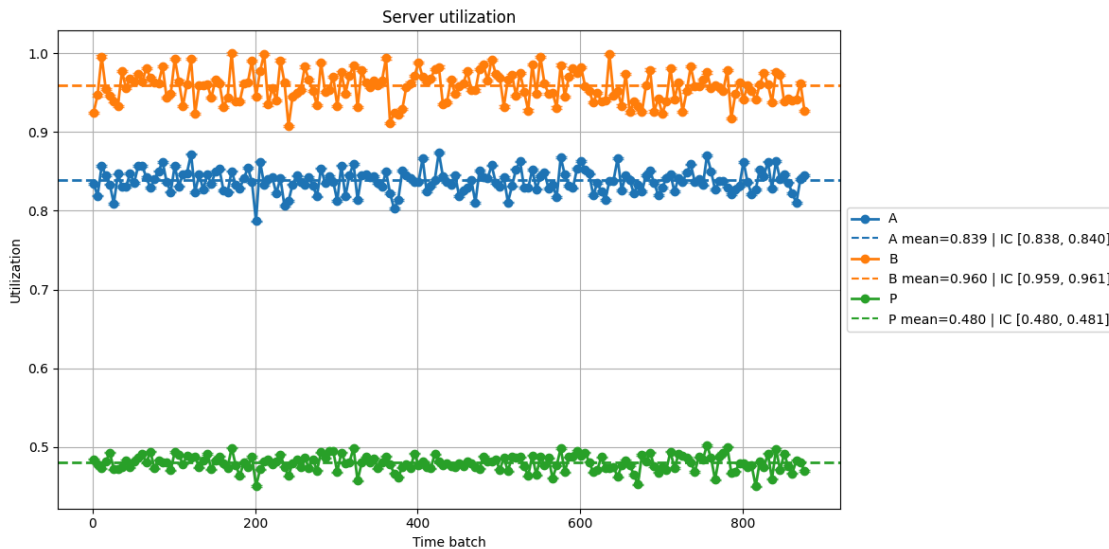
Caso $\lambda_{\text{sorgente}} = \lambda_{\text{max}} = 1.2 \text{ req/s}$ - One factor Authentication

Metrica	Valori medi ottenuti dalla simulazione IC 95% simulazione con t Student	Valori medi ottenuti dall'analisi analitica
Tempo di risposta del sistema	[24.4824, 26.0603]	25.14423 s
Tempo di risposta A1	[1.2450, 1.2555]	1.25 s
Tempo di risposta B	[19.3616, 20.9140]	20 s
Tempo di risposta A3	[2.4653, 2.5086]	2.5 s
Tempo di risposta P	[0.7688, 0.7734]	0.769230 s
Tempo di risposta A5	[0.6224, 0.6276]	0.625 s
Numero di utenti in A	[5.174, 5.272]	5.25 s
Numero di utenti in B	[23.208, 25.133]	24 s
Numero di utenti in P	[0.922, 0.928]	0.923076 s
Utilizzazione server A	[83.8%, 84.0%]	84%
Utilizzazione server B	[95.9%, 96.1%]	96%
Utilizzazione server P	[48.0%, 48.1%]	48%

Di seguito, è possibile osservare i grafici ottenuti dalle metriche ottenute attraverso la simulazione, svolta con le modalità espresse precedentemente.







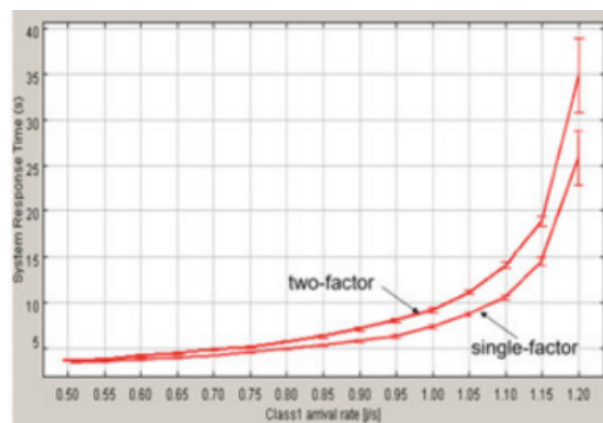
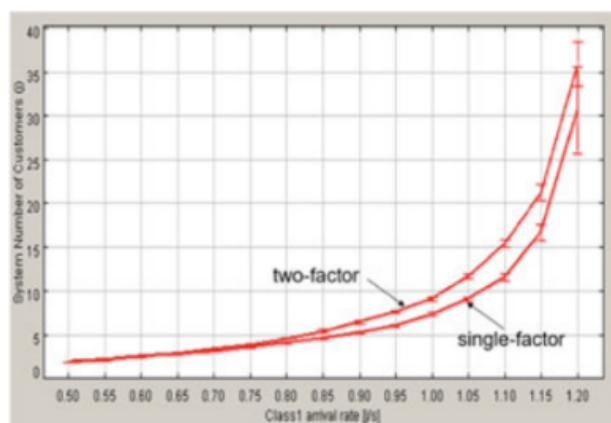
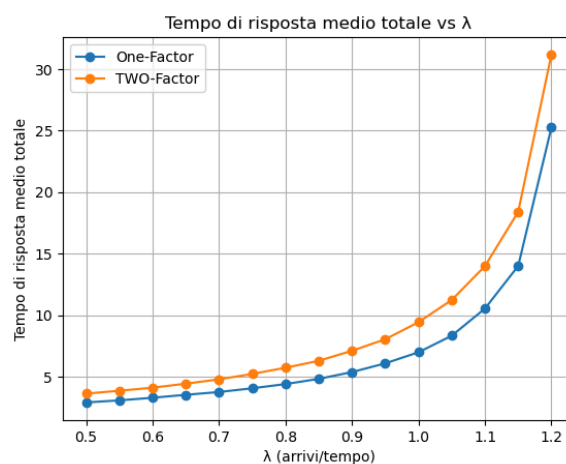
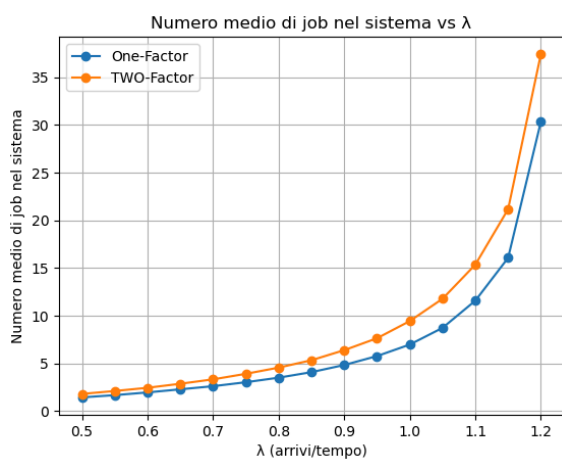
In definitiva i calcoli analitici risultano essere conformi ai dati riportati dalle simulazioni, confermando la robustezza della simulazione progettata e implementata.

Validazione

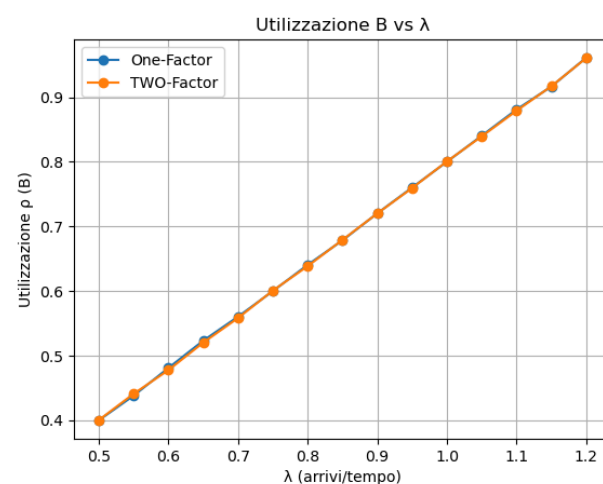
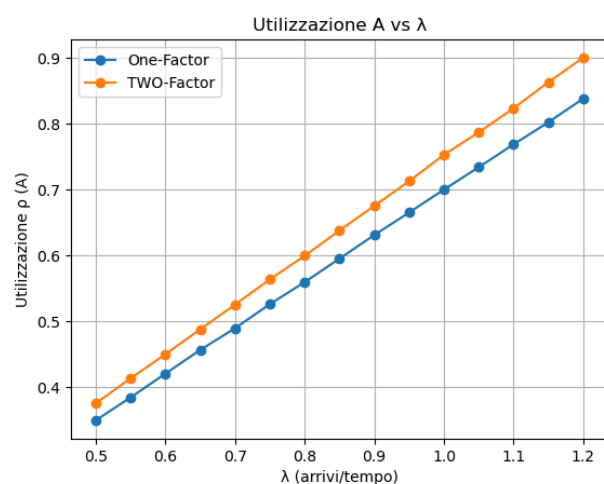
Per validare i risultati ottenuti dalla simulazione del sistema in esame, si è deciso di confrontarli con i pochi dati disponibili nella specifica del progetto. In particolare, sono state simulate due configurazioni: **One-Factor Authentication** e **Two-Factor Authentication**, analizzando i **tempi di risposta globali del sistema**, il **numero medio di job presenti nel sistema** e il **throughput**.

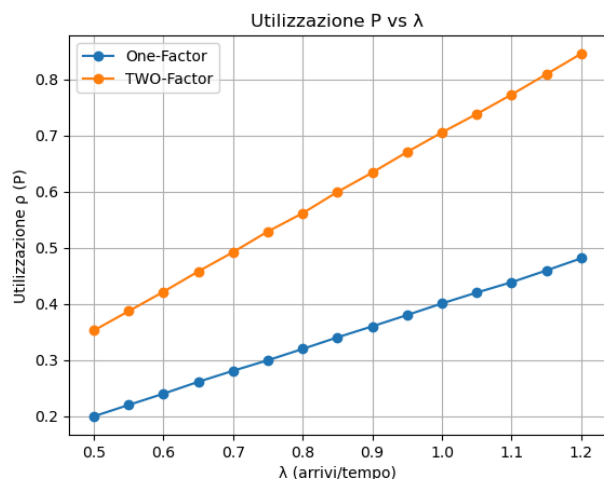
La simulazione è stata condotta variando il **tasso di arrivo λ** da **0.5** a **1.2** req/s.

Di seguito vengono riportati i grafici relativi alla simulazione e forniti dalla specifica. Questi mostrano il **numero medio di job nel sistema** al variare del tasso di arrivo e del **tempo medio di risposta complessivo di un job** nel sistema.

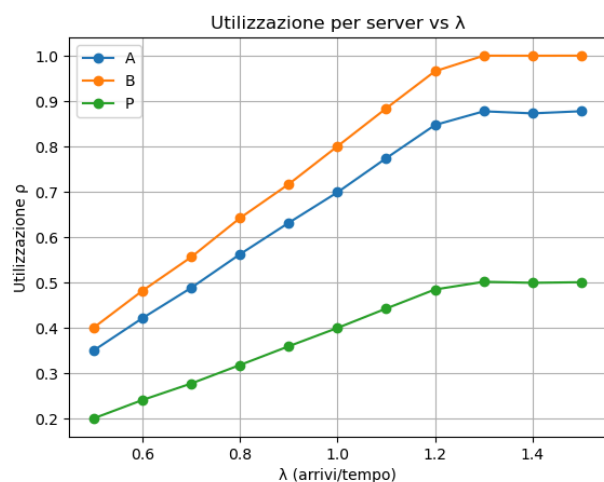
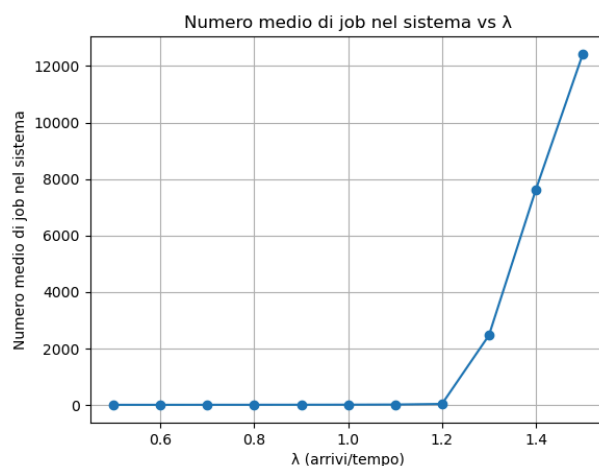
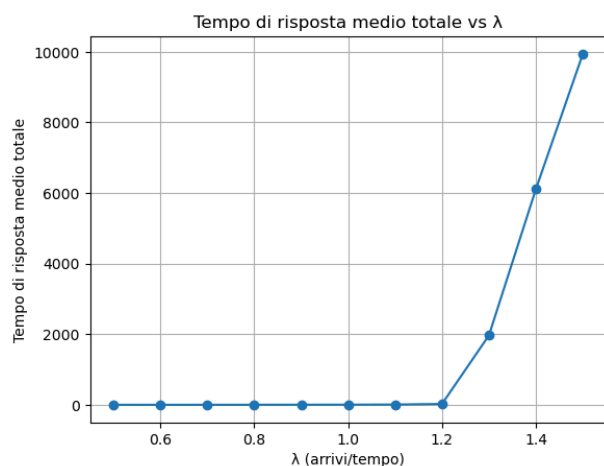


Vengono inoltre riportati i grafici relativi all'utilizzazione per server ottenuti dalla simulazione.





Per effettuare un'ulteriore validazione, è stato deciso di analizzare come reagisce il sistema rispetto ad un aumento progressivo del tasso di arrivo. Ci si aspetta infatti che i tempi di risposta e il numero medio di job presso ciascuna stazione divergano.



Osservando i grafici è facile notare come il sistema si comporti come previsto al variare del tasso di arrivo. I risultati ottenuti risultano pienamente conformi a quelli previsti dalla specifica iniziale.

Progettazione degli esperimenti

La fase di progettazione degli esperimenti si articola nei seguenti tre punti:

1. **Analisi preliminare (Batch Means, Intervalli di Confidenza e Autocorrelazione):** Questo studio preliminare è essenziale per la verifica.
2. **Analisi del collo di bottiglia:** fondamentale per identificare le componenti del sistema che limitano le prestazioni complessive, soprattutto in condizioni di elevato carico.
3. **Analisi del transitorio:** Si esamina la convergenza del sistema verso lo stato stazionario.
4. **Simulazione a orizzonte finito:** Il sistema viene simulato per N ore utilizzando il metodo delle Replications.

Considerazioni e accorgimenti preliminari

Per la verifica, ci siamo avvalsi del metodo dei *batch means* e del calcolo degli intervalli di confidenza. Questi elementi sono stati cruciali per quantificare l'errore statistico e garantire l'affidabilità delle stime prodotte.

Autocorrelazione

Nelle Discrete Event Simulations, le osservazioni $X_1, X_2, \dots, X_n$ ottenute durante la simulazione non sono indipendenti, ma autocorrelate, cioè il valore di una osservazione può dipendere dai valori precedenti. Per stimare correttamente la media $\mu = E[X]$ e la sua varianza è necessario che i campioni utilizzati siano quasi indipendenti. La tecnica adottata per ottenere stime affidabili è quella del batch means. Partendo dalla sequenza di osservazioni originale, questa viene divisa in m blocchi di dimensione b .

I batch means sono:

$$\bar{X}_j = \frac{1}{b} \sum_{i=1}^b (X_{(j-1)b+1}) \quad j = 1, 2, \dots, m.$$

Se i batch sono sufficientemente lunghi, le $\bar{X}_j$ possono essere considerate approssimativamente indipendenti e distribuite secondo una normale. Il nostro obiettivo è quindi scegliere la batch size in modo da rendere trascurabile la correlazione tra i vari batch.

A tal fine, è stata adottata la tecnica empirica di Chatfield, che consente di scegliere la batch size in base all'autocorrelazione della sequenza X_t . Per una sequenza di n osservazioni, la funzione di autocorrelazione ACF al lag j è definita come:

$$r_j = \frac{\sum_{t=1}^{n-j} (X_t - \bar{X})(X_{t+j} - \bar{X})}{\sum_{t=1}^n (X_t - \bar{X})^2} \quad \text{dove} \quad \bar{X} = \frac{1}{n} \sum_{t=1}^n X_t \quad \text{è la media campionaria.}$$

Il lag j misura la distanza in termini di passi temporali.

Chatfield propone un criterio empirico per determinare fino a quale lag le autocorrelazioni siano statisticamente significative. Sotto l'ipotesi che la sequenza sia rumore bianco, cioè indipendente, l'errore standard di r_j è $SE(r_j) \approx \frac{1}{\sqrt{n}}$, $n = \#campioni$. Per un intervallo di confidenza approssimato al 95% per r_j è $[-\frac{2}{\sqrt{n}}, \frac{2}{\sqrt{n}}]$.

La regola di Chatfield afferma che un valore di autocorrelazione è da considerarsi non significativamente diverso da zero se $|r_j| < \frac{2}{\sqrt{n}}$. Al fine di individuare il lag j in cui la correlazione diventa trascurabile, si fanno delle prove con valori di lag che sembrano opportuni. Una volta identificato il lag j_{min} tale che la precedente condizione viene rispettata, questo rappresenta un'indicazione della lunghezza di dipendenza della serie. In seguito, la batch size ottimale b deve essere scelta in modo da essere maggiore del lag j_{min} , tipicamente si sceglie $b \geq c \cdot j_{min}$, $c \in [1.2, 2]$. In questo modo, è garantito che le osservazioni all'interno di un batch siano ancora correlate, quindi la media sul batch è ancora rappresentativa del processo, ma in modo tale che le medie dei batch siano considerabili come indipendenti.

Infine, il numero di batch è quindi la parte inferiore di $m = \frac{n}{b}$.

Calcolo degli intervalli di confidenza

Scelto un batch size ottimale, e quindi il numero di batch, si vuole ora stimare la media stazionaria $\mu = E[X_t]$ e costruire un intervallo di confidenza al 95% per essa. Dividiamo quindi la sequenza complessiva di job simulati in batch contigui di uguale lunghezza. Per ciascuno di essi, calcoliamo la media, ottenendo m valori medi $\bar{X}_1, \bar{X}_2, \dots, \bar{X}_m$. Queste medie rappresentano campioni quasi indipendenti. A partire da questi, viene calcolata la media complessiva stimata è

$$\bar{X} = \frac{1}{m-1} \sum_{i=1}^m (\bar{X}_i).$$

e la varianza campionaria associata è

$$S^2 = \frac{1}{m-1} \sum_{i=1}^m (\bar{X}_i - \hat{\mu})^2$$

Sotto l'ipotesi che le medie dei batch siano approssimativamente indipendenti, un intervallo di confidenza al livello del 95% per la media stazionaria è dato da:

$$\bar{X} \pm t_{0.975, m-1} \cdot \frac{S}{\sqrt{m}}$$

dove $t_{0.975, m-1}$ è il quantile della distribuzione t di Student con $m-1$ gradi di libertà, in modo da stimare l'intervallo entro cui la media vera del processo stazionario con probabilità del 95%. La bontà dell'intervallo è garantita dall'indipendenza tra i batch.

Una volta definiti tutti i componenti del sistema, si procede alla verifica del modello di simulazione, con l'obiettivo di accertarne la corretta implementazione rispetto al modello analitico di riferimento. Tale verifica si basa sul confronto tra i risultati ottenuti mediante simulazione e quelli derivanti dall'analisi teorica, al fine di valutare la coerenza tra il comportamento atteso del sistema e quello effettivamente osservato. Questo approccio consente di individuare eventuali discrepanze riconducibili a errori di modellazione o di implementazione, garantendo che il modello computazionale riproduca fedelmente le assunzioni e le logiche previste dalla specifica di progetto.

Per rendere il confronto significativo, il sistema è stato implementato mantenendo inalterate le condizioni operative del modello analitico. In particolare, in questa fase non sono considerate perdite di job: tutti i job che accedono al sistema vengono quindi completati correttamente. La simulazione è stata eseguita in condizioni di carico massimo, utilizzando il tasso di arrivo più elevato previsto dal modello base, pari a **$\lambda = 1.2 \text{ req/s}$** e assumendo l'impiego di un meccanismo di autenticazione a singolo fattore.

Si è scelto di eseguire la simulazione con il **tasso di arrivo massimo** previsto per il modello base, **$\lambda_{high}=1.2 \text{ req/s}$** , corrispondente alla condizione di **carico massimo**. La simulazione è stata condotta assumendo l'utilizzo di un meccanismo di **autenticazione a singolo fattore**.

La simulazione è stata condotta a **orizzonte temporale infinito** ed è stato adottato il metodo del *Batch Means* per la stima delle metriche di interesse e dei relativi intervalli di confidenza. In una fase preliminare, sono state valutate diverse dimensioni di batch, testando valori di lag compresi tra 1024 e 16352, con passo 32, al fine di individuare una configurazione che garantisca un livello di autocorrelazione tra batch inferiore alla soglia proposta da Chatfield. Il primo valore di batch size b che consente di rispettare la soglia empirica di Chatfield a lag 1 risulta pari a **batch_size = 16352**. Tuttavia, tale dimensione è stata ritenuta eccessivamente elevata, in quanto comporta una riduzione significativa del numero di batch disponibili e, di conseguenza, una minore affidabilità nella stima della varianza inter-batch.

Per questo motivo, si è valutata una configurazione alternativa con **$b = 4096$** . Sebbene questa dimensione di batch non garantisca, per tutte le metriche considerate, il rispetto della soglia più stringente di Chatfield, essa soddisfa il criterio meno restrittivo e comunemente accettato di $|\rho| < 0.2$ permettendo al contempo di ottenere una maggiore varianza tra batch e un numero adeguato di osservazioni statisticamente significative.

Nel corso della simulazione sono stati generati complessivamente **$n = 3_599_303$ job**, che, con una batch size pari a **4096**, corrispondono ad un totale di **$k = 878$ batch**. Per tale valore di k , la soglia empirica di Chatfield per l'autocorrelazione tra batch risulta pari a $|\rho| < 0.067496$. I valori di autocorrelazione calcolati sulle statistiche di simulazione sono riportati di seguito.

```

=== JOB-BASED BATCHES ===
RT_system    | rho1=0.1073 | Chatfield OK=False | Fixed OK=True
RT_A          | rho1=0.0028 | Chatfield OK=True  | Fixed OK=True
RT_B          | rho1=0.1067 | Chatfield OK=False | Fixed OK=True
RT_P          | rho1=0.0449 | Chatfield OK=True  | Fixed OK=True

=== TIME-BASED BATCHES ===
A_avg_users   | rho1=0.0029 | Chatfield OK=True  | Fixed OK=True
A_utilization | rho1=-0.0339 | Chatfield OK=True  | Fixed OK=True
B_avg_users   | rho1=0.0767 | Chatfield OK=False | Fixed OK=True
B_utilization | rho1=-0.0037 | Chatfield OK=True  | Fixed OK=True
P_avg_users   | rho1=0.0303 | Chatfield OK=True  | Fixed OK=True
P_utilization | rho1=0.0272 | Chatfield OK=True  | Fixed OK=True
Throughput    | rho1=-0.0284 | Chatfield OK=True  | Fixed OK=True

```

Per la stima delle metriche di utilizzazione e del numero medio di job (utenti) presso le stazioni, il dataframe prodotto dalla simulazione è stato partizionato in batch temporali contigui di durata **$\Delta T = 3413$ secondi**. Tale intervallo temporale corrisponde al tempo medio necessario alla generazione di 4096 job, dato il tasso di arrivo considerato (**$\lambda = 1.2$ req/s**). Perciò, a ciascun batch viene associata una stima delle metriche di interesse calcolata sui dati raccolti all'interno di tale intervallo temporale. Queste stime sono trattate come

medie calcolate batch per batch, analogamente a quanto previsto dal metodo delle Batch Means, e utilizzate per il calcolo degli intervalli di confidenza sulle stesse.

In conclusione, la scelta finale dei parametri $(b,k) = (4096, 878)$ rappresenta un compromesso tra il rispetto delle condizioni di indipendenza dei batch e la necessità di ottenere una varianza inter-batch sufficientemente elevata.

Analisi del collo di bottiglia

In un sistema con politica di servizio Processor Sharing, il throughput massimo sarà determinato dalla capacità del server che ha il tempo di servizio più lungo. Questo è fondamentale in quanto il throughput totale del sistema non potrà mai superare la capacità del server più lento, che rappresenta un collo di bottiglia.

Quindi, per determinare il throughput bound del sistema è necessario osservare il server che ha il tempo di servizio massimo. In questa analisi consideriamo i valori già ottenuti dalla fase di verifica analitica, che nello specifico utilizza un'autenticazione a singolo fattore. Per ogni server, **il tempo di servizio medio** è dato dai seguenti valori:

- **Server A:** $E[S]_A = 0.23 \text{ s}$ $E[S]_{A - \text{two factor authentication}} = 0.25 \text{ sec}$
- **Server B:** $E[S]_B = 0.8 \text{ s}$
- **Server P:** $E[S]_P = 0.4 \text{ s}$ $E[S]_{P - \text{two factor authentication}} = 0.7 \text{ sec}$

Il massimo tempo di servizio (cioè D_{max}) è quindi 0.8 s, corrispondenti al server B, anche nel caso in cui si stesle considerando un'autenticazione a due fattori. Il throughput bound è dato quindi da

$$\frac{1}{D_{max}} = \frac{1}{0.8} = 1.25 \text{ req/s.}$$

Il collo di bottiglia è il server che limita il throughput complessivo nel sistema. In questo caso quindi tale bottleneck è rappresentato dal server B.

Analisi del transitorio

Protocollo di simulazione

Sono state eseguite **5 repliche** della simulazione con $\lambda=0.5$, tutte con tasso di arrivo uguale a 0.5 req/s. Ogni replica ha una **soglia temporale** pari a **170.000 secondi**, così da valutare se e quando il sistema mostri **convergenza verso un regime stazionario**.

Raccolta delle metriche

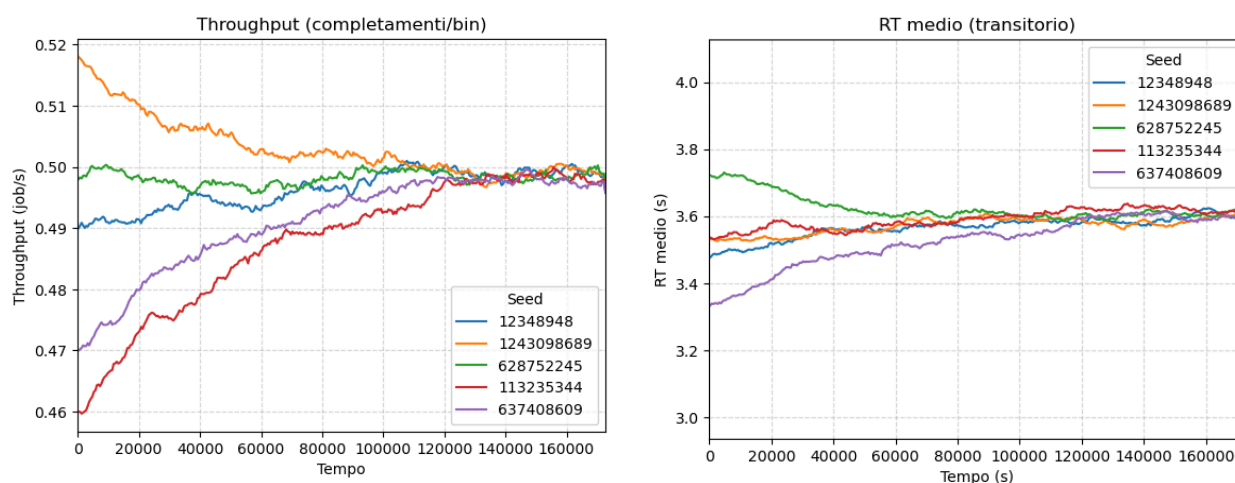
Le metriche operative del sistema sono state **campionate ogni 100 secondi** per l'intera durata di ciascuna replica.

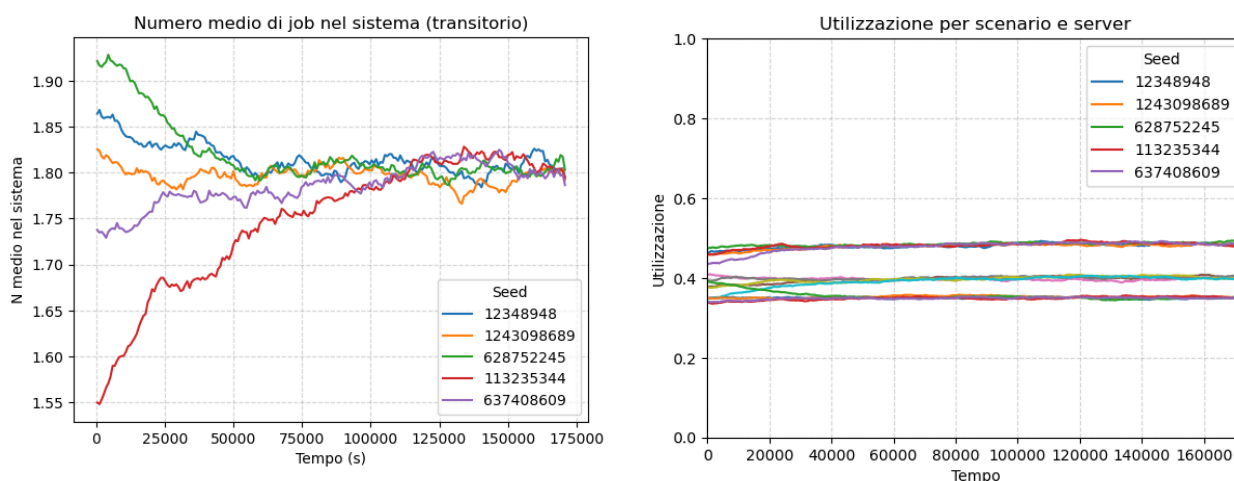
RNG multi-stream: inizializzazione e uso

Il generatore di numeri casuali è stato **inizializzato una sola volta** prima del ciclo delle 40 repliche tramite la funzione **PlantSeeds()**, che prepara i **256 stream** indipendenti previsti dalla libreria. All'interno del ciclo viene chiamata la simulazione che **seleziona gli stream necessari per ogni server** e lasciamo che la **libreria prosegua automaticamente** dagli stati correnti degli stream. In questo modo le repliche utilizzano **sequenze non sovrapposte** (proseguizioni disgiunte degli stessi stream) e risultano, per gli scopi dell'analisi, **i.i.d.** (indipendenti e identicamente distribuite).

Replicabilità

Al termine di ogni replica **salviamo lo stato/seed di ciascuno stream** in **seed.csv**. Questo salvataggio **non è necessario** per far "ripartire" la replica successiva (che continua comunque dagli stati interni mantenuti dalla libreria), ma consente la **replicazione esatta** degli esperimenti, il **ripristino** dopo eventuali interruzioni e la **tracciabilità** delle varie run.





Dai risultati ottenuti vediamo che il tempo di risposta medio di tutto il sistema e utilizzazione per ogni server vengono raggiunti intorno ai 100.000 s.

Simulazione orizzonte finito

È stata scelta una simulazione a orizzonte temporale limitato per esaminare l'evoluzione del sistema nell'arco di **172.000 secondi**, un intervallo che coincide con la durata del periodo di osservazione relativo al fine settimana. L'applicazione è stata sviluppata mirando a un pubblico locale, in questo caso italiano. Tale configurazione consente di valutare il comportamento del sistema in condizioni di **carico elevato (stress)**. In base alle stime fornite dalla fonte di riferimento, opportunamente **normalizzate rispetto alle caratteristiche del nostro sistema**, il **weekend** risulta essere il periodo con il **maggiore afflusso di utenti**, in quanto rappresenta il momento di massima attività per l'effettuazione di acquisti.

Di conseguenza, in questo intervallo temporale si osserva il **più alto tasso di arrivi** al sistema. Analizzare la simulazione in tale scenario permette quindi di ottenere informazioni particolarmente significative sulla **qualità del servizio offerto**, sulla **robustezza del sistema** e sulla sua capacità di sostenere condizioni operative critiche, evidenziando eventuali colli di bottiglia e degradazioni delle prestazioni che potrebbero non emergere in scenari di carico medio o ridotto.

Gli intervalli di confidenza per la simulazione ad orizzonte finito sono definiti come:

$$\bar{x} \pm \frac{t^* s}{\sqrt{n-1}}$$

Dove:

- $\bar{x}$ è la media campionaria

- s è la deviazione standard campionaria
- n è il numero di repliche
- t^* è il quantile della distribuzione t di Student con $n-1$ gradi di libertà

Questo intervallo fornisce una stima dell'intervallo entro il quale si trova la media μ con probabilità pari a $(1 - \alpha)$.

Nel nostro caso il numero di repliche è superiore a 40. Poiché per n grandi la distribuzione t converge alla distribuzione normale standard, l'intervallo viene approssimato utilizzando il valore asintotico del quantile normale. Per questa ragione, utilizziamo il valore asintotico:

$$t_{\infty}^* = \lim_{n \rightarrow \infty} \text{idfStudent}(n - 1, 1 - \alpha/2) = \text{idfNormal}(0.0, 1.0, 1 - \alpha/2)$$

Nel caso di un intervallo di confidenza al 95%, tale valore è 1.960 (per $n > 40$).

Poiché inizialmente non si dispone di una stima attendibile della deviazione standard, e desideriamo ricavare il numero di repliche necessario a partire dalla formula precedente, è possibile riscrivere l'equazione in forma esplicita rispetto a n . L'operazione consente di determinare il numero minimo di repliche necessario per ottenere un intervallo di confidenza con ampiezza prefissata, ottenendo la formula:

$$n = \left(\frac{t_{\infty}^* s}{w} \right)^2 + 1$$

dove si considera la parte inferiore del quadrato. Tuttavia, questa non è applicabile direttamente, perché richiede la conoscenza di s e w .

Un approccio alternativo consiste nello stimare il rapporto tra w/s in modo da trasformare l'equazione precedenti (originariamente in due variabili) ad una sola incognita. Specificato

$$\frac{w}{s} = 0.1$$

in quanto desideriamo che l'ampiezza w rappresenti il **10%** della deviazione standard s , allora il numero di repliche necessario affinché la media μ ricada negli estremi $\pm w$ è pari a :

$$n = \left(\frac{1.96}{0.1} \right)^2 + 1 \approx 384 + 1 = 385$$

Sono state quindi effettuate **385 repliche** indipendenti della simulazione a orizzonte finito. Questo numero di repliche consente di ottenere, con un **livello di confidenza del 95%**, un'accuratezza relativa pari al 10% rispetto all'ampiezza dell'intervallo di confidenza della media.

Per gestire le repliche, il **seed** è stato fissato all'esterno del ciclo di simulazione e, al termine di ogni esecuzione, è stato salvato lo stato di tutti gli stream casuali, in modo da poter riprendere la simulazione dalla successiva configurazione di stato nella run seguente.

One-factor authentication orizzonte finito

Il **tasso di arrivo λ** è stato variato da **0.5** fino a **picchi di 1.4** richieste al secondo, con l'obiettivo di testare il sistema in diversi scenari di carico e valutare la sua capacità di gestire correttamente i picchi di traffico. Tale simulazione è stata condotta per il one factor authenticator.

Segue una tabella che riporta i valori del tasso di arrivo λ , **differenziati in base al giorno** della settimana (sabato o domenica) e **alle rispettive fasce orarie**.

Queste metriche sono state ricavate dalla fonte

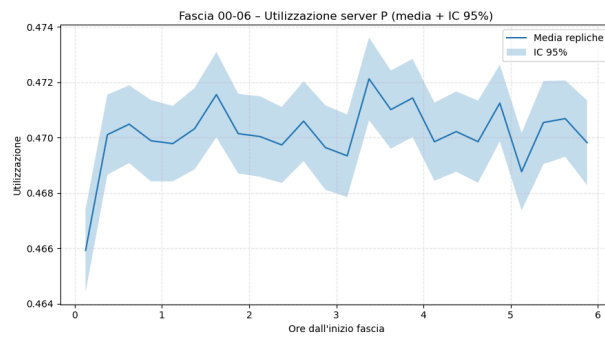
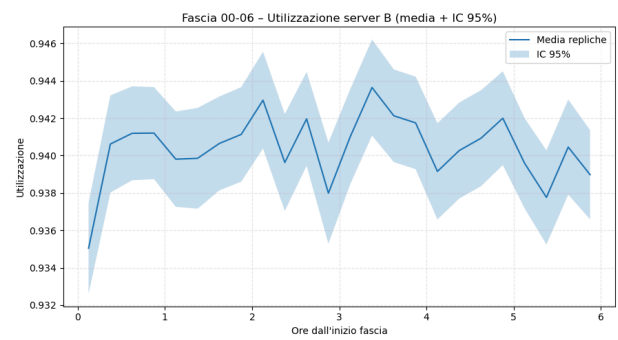
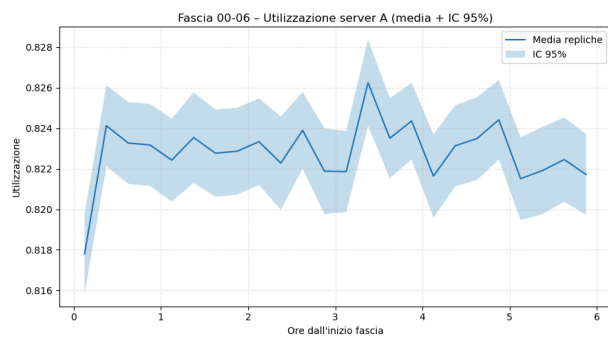
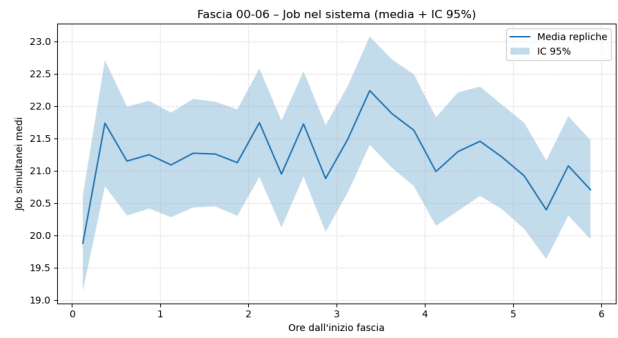
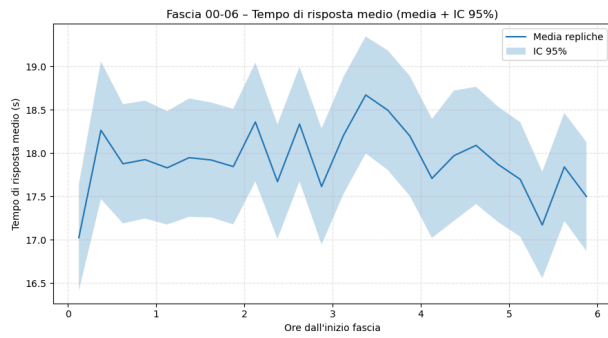
<https://scispace.com/pdf/characterization-of-e-commerce-traffic-2vit280tp0.pdf> e successivamente normalizzate per adattarsi al nostro caso d'uso, portando ai seguenti parametri:

	00-06	06-09	09-12	12-17	17-21	21-24
Sabato	1.176	1.224	1.23	1.351	1.251	1.126
Domenica	1.13	1.24	1.287	1.379	1.244	1.166

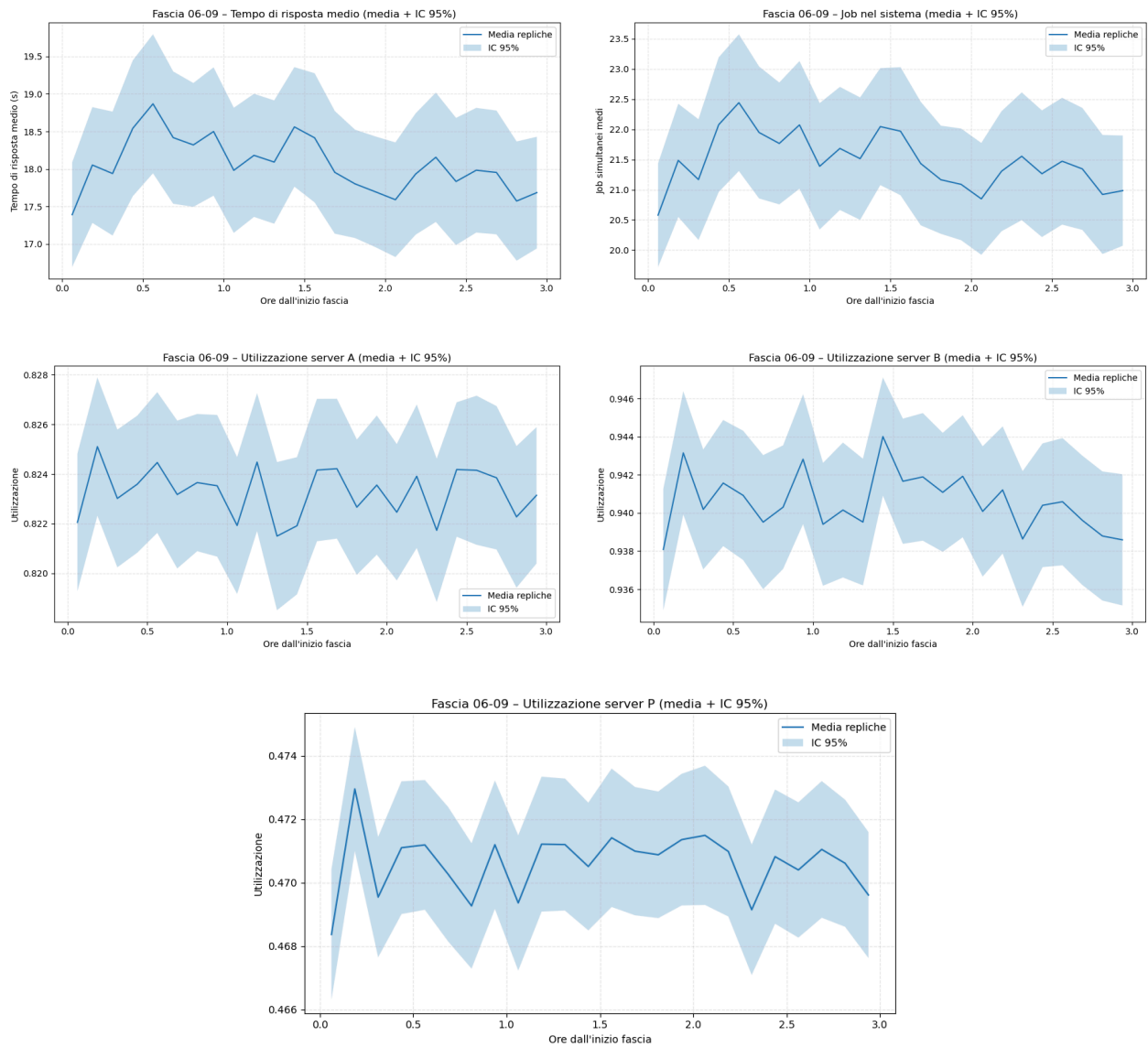
L'obiettivo è quello di riprodurre un **comportamento realistico**, con arrivi distribuiti in base alle **fasce orarie** e ai **giorni della settimana**, riflettendo situazioni tipiche del mondo reale, in cui si osservano **sovraccarichi nel fine settimana**. I rispettivi seed vengono salvati nel csv: *seed Orizzonte Finito.csv*.

Vengono riportati i grafici in cui, per ciascuna fascia oraria, è tracciato l'andamento delle metriche calcolate come media del sabato e della domenica. In particolare, per ogni fascia mostriamo **tempo di risposta medio**, **numero medio di job nel sistema** e **utilizzazione** (per ciascun server A, B, P), dove ogni punto dell'asse temporale della fascia rappresenta la **media** della metrica nel corrispondente sotto-intervallo con le shadow che rappresentano gli intervalli di confidenza in precedenza calcolati.

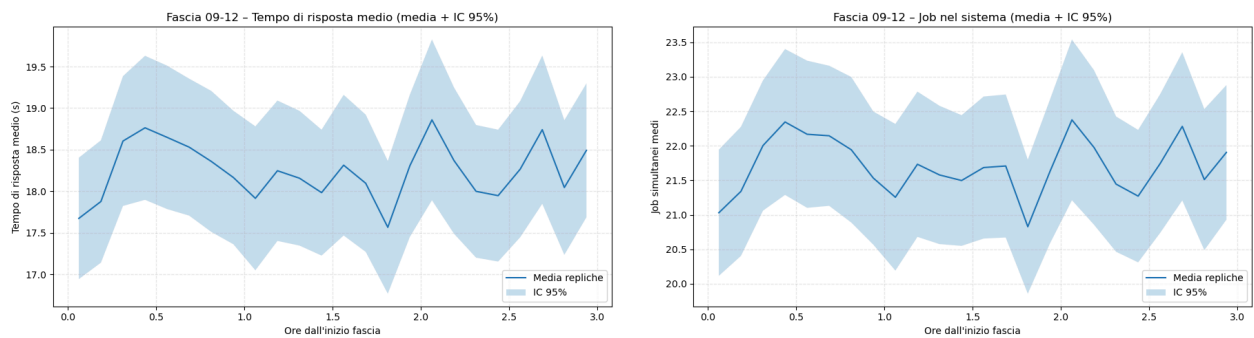
Fascia Oraria 00:00-06:00

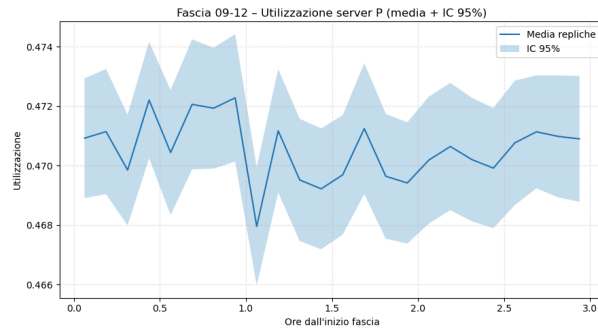
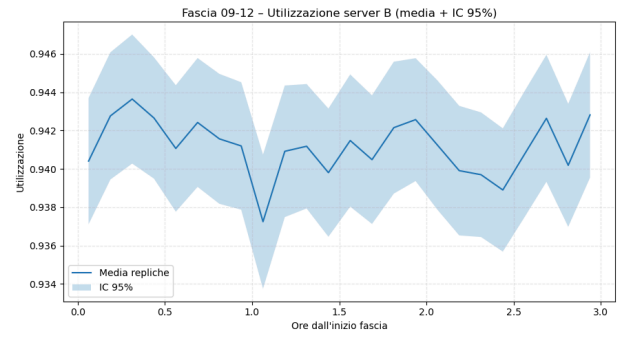
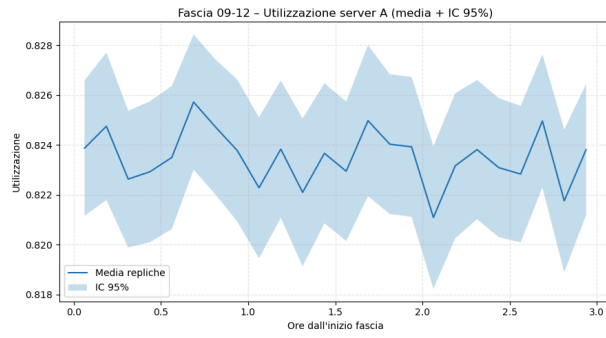


Fascia Oraria 06:00-09:00

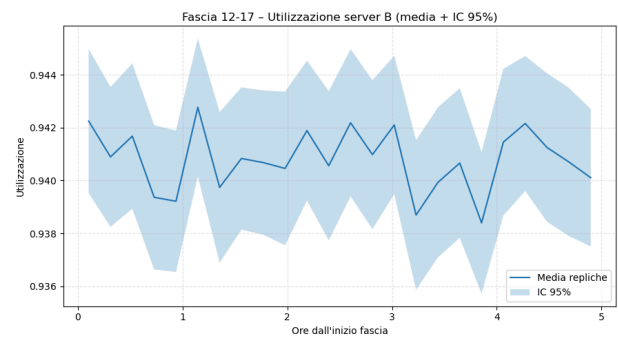
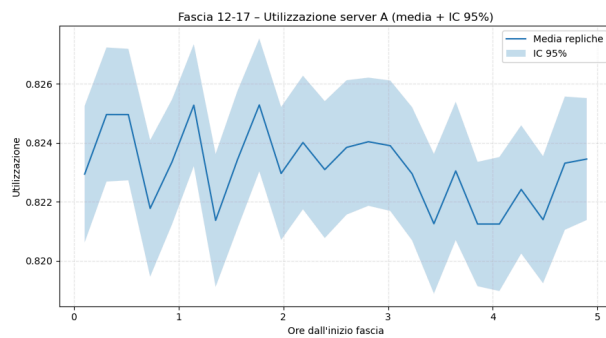
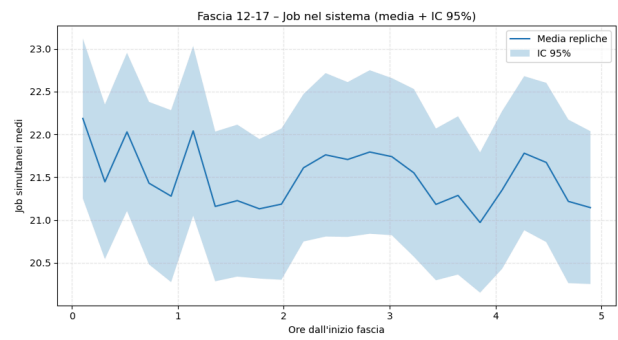
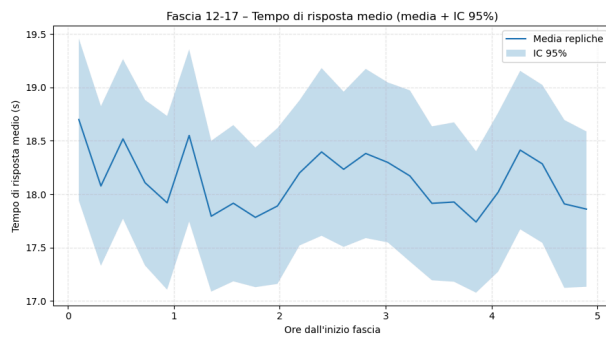


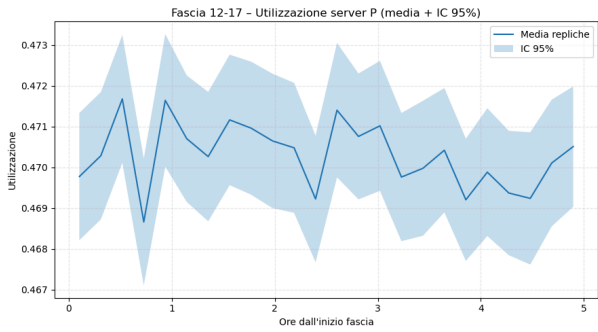
Fascia Oraria 09:00-12:00



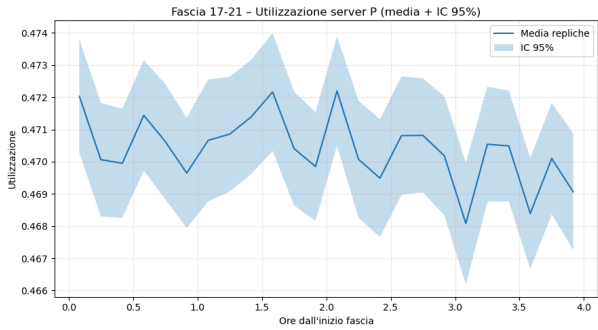
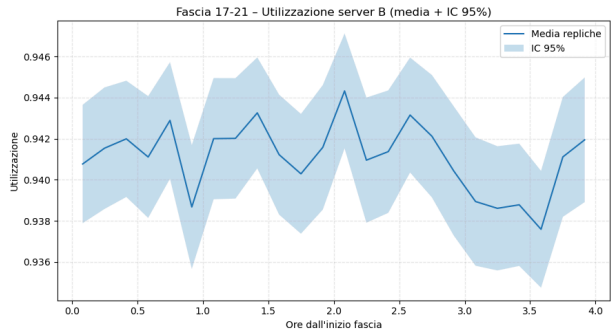
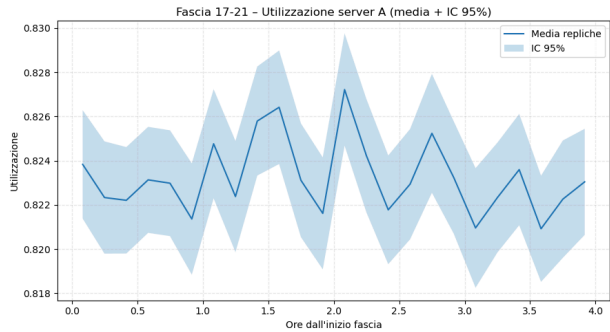
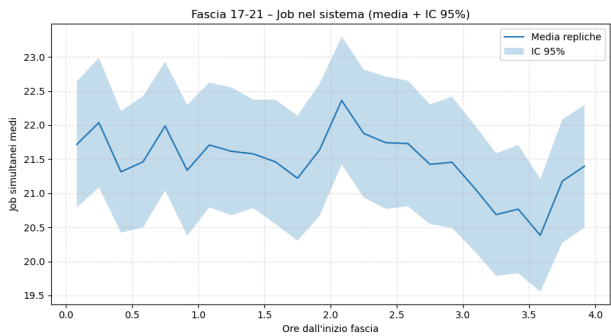
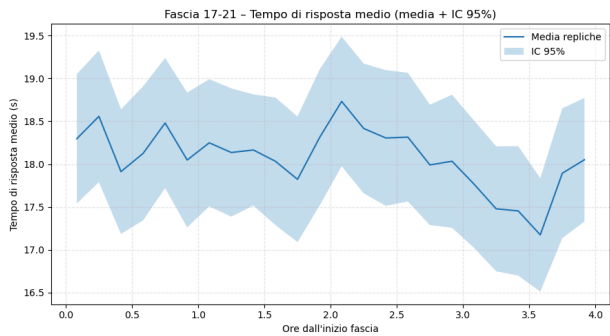


Fascia Oraria 12:00-17:00

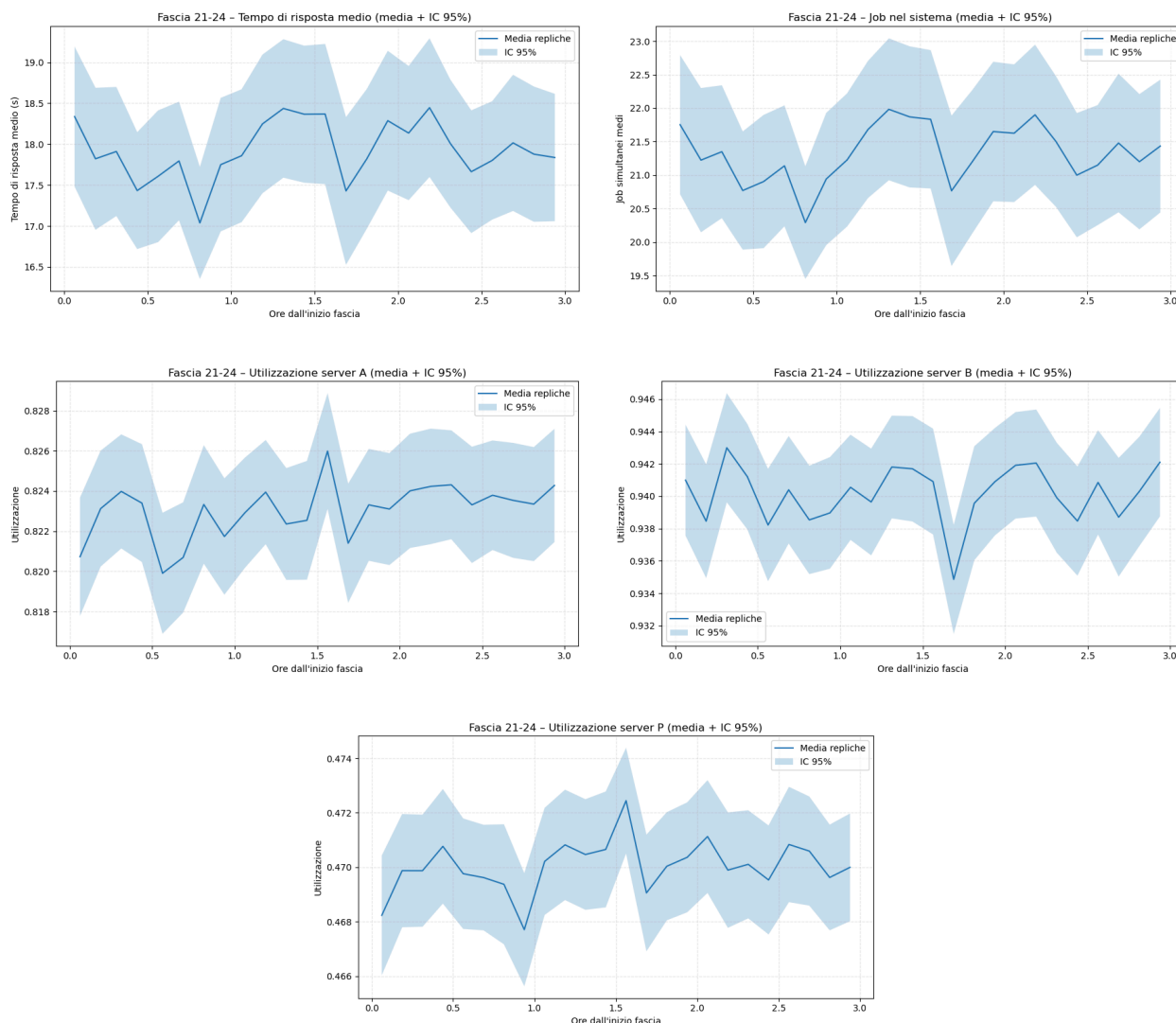




Fascia Oraria 17:00-21:00



Fascia Oraria 21:00-24:00



Dall'analisi dei grafici relativi alle metriche considerate, emerge un andamento pienamente coerente con quanto previsto dal modello teorico. Nelle fasce orarie mattutine, in particolare nella fascia 00:00-06:00, le metriche si mantengono stabili, indicando un sistema in equilibrio e non soggetto a condizioni di sovraccarico.

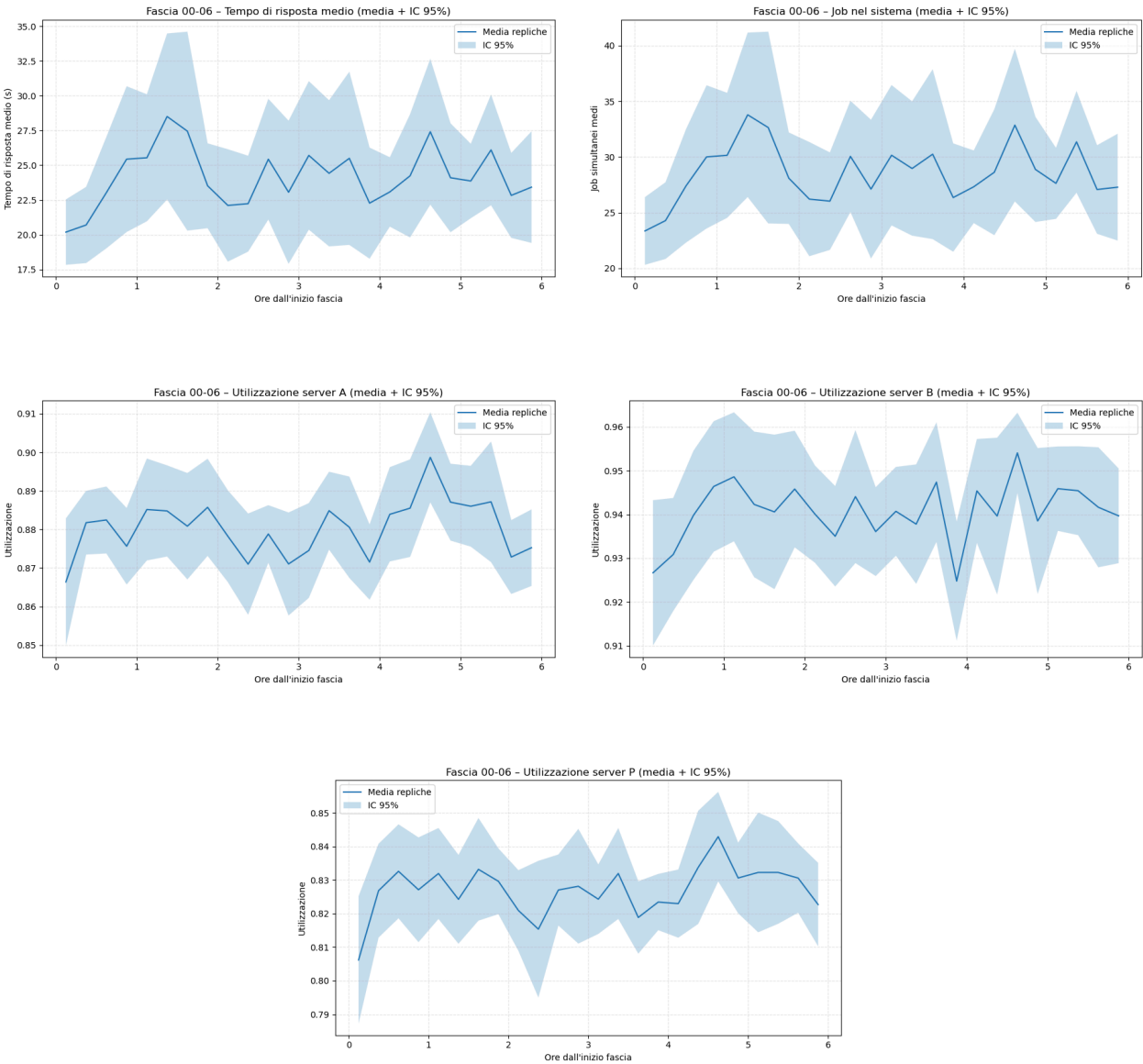
Come osservato, con l'aumentare della fascia oraria e del conseguente incremento del carico di lavoro, il sistema mostra maggiori difficoltà nel mantenere metriche convergenti, principalmente a causa dell'elevato numero di job attivi.

In particolare, si conferma che **il server B costituisce il principale collo di bottiglia** dell'intero sistema, limitandone le prestazioni complessive nelle ore di maggiore affluenza. Al contrario, nelle fasce serali, a partire indicativamente dall'intervallo **17:00-21:00** si osserva una progressiva riduzione del carico, che consente al sistema di ritornare gradualmente verso condizioni di equilibrio e di smaltire le richieste residue.

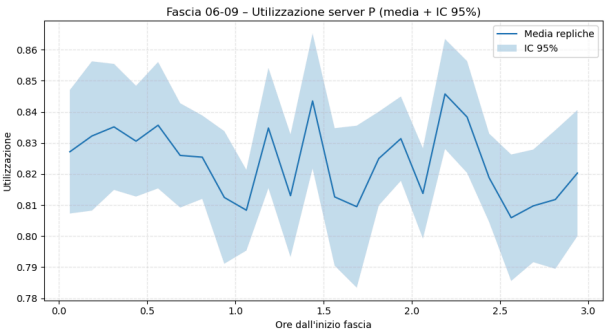
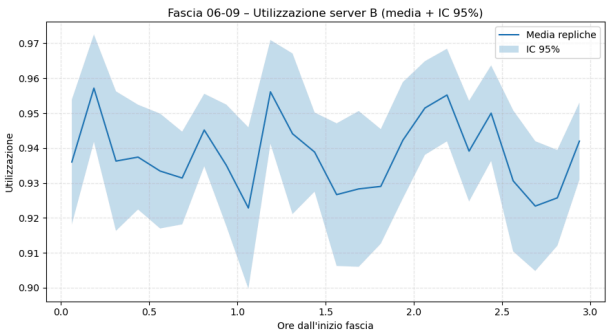
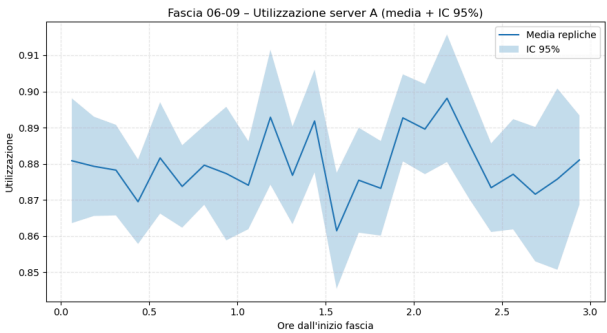
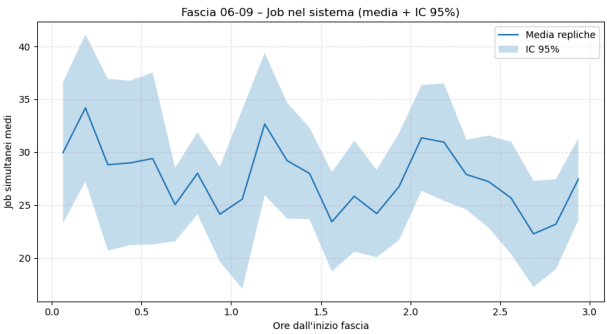
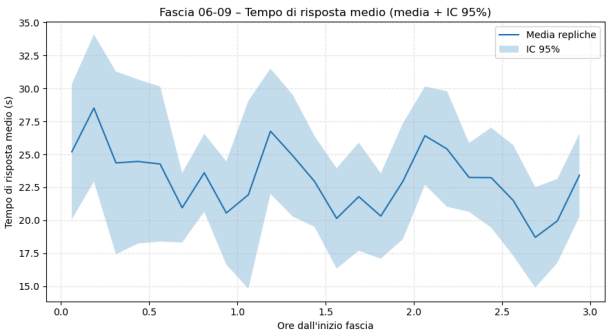
Two-factor authentication orizzonte finito

La tabella dei lambda resta invariata; procedendo con la simulazione otteniamo:

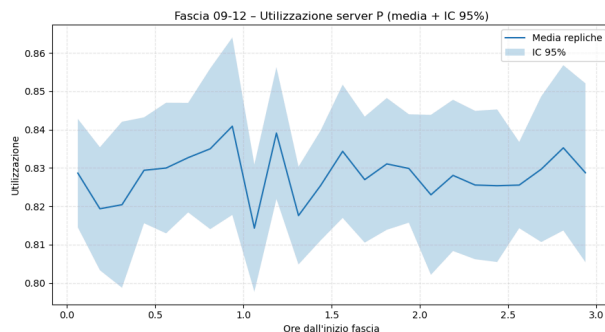
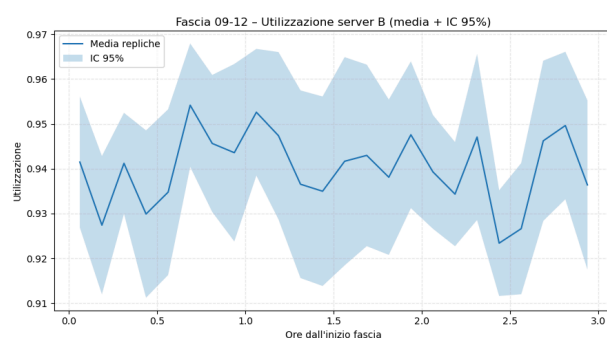
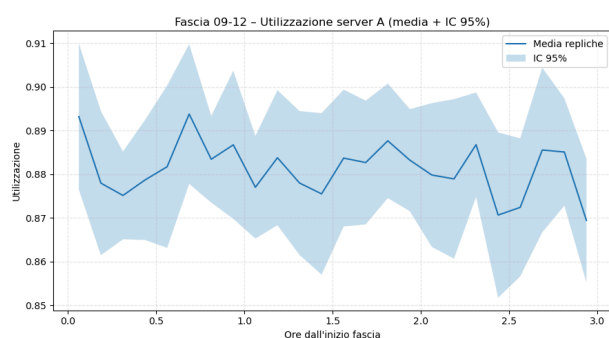
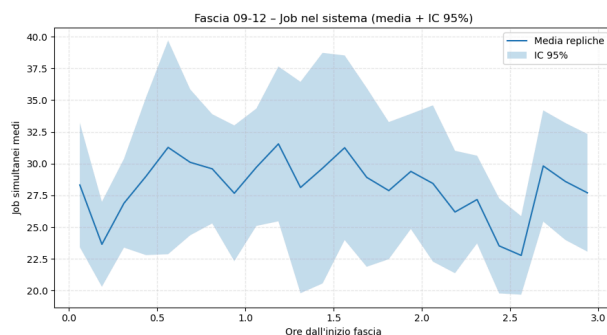
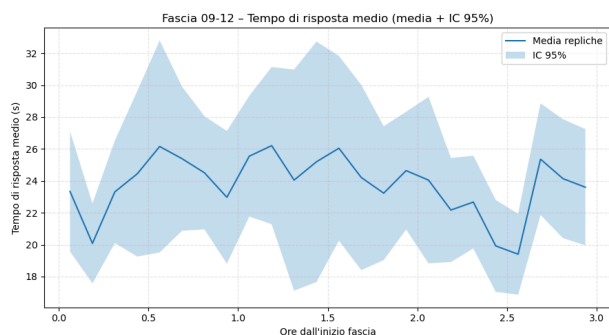
Fascia Oraria 00:00-06:00



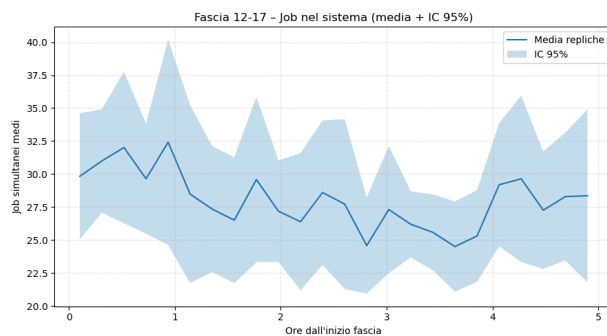
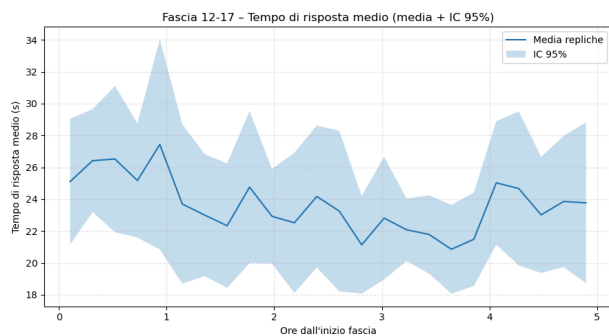
Fascia Oraria 06:00-9:00

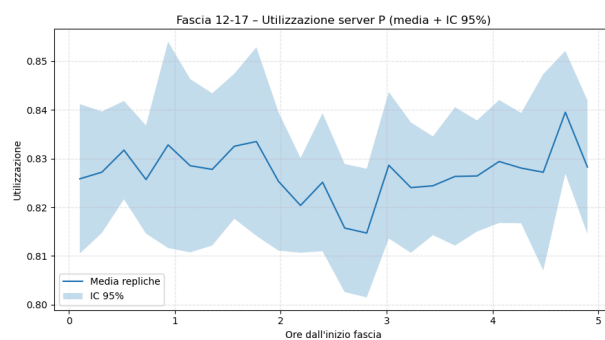
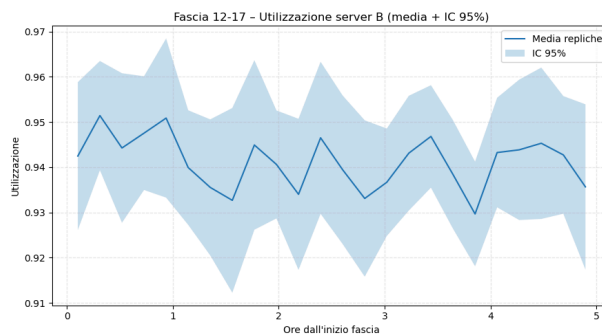
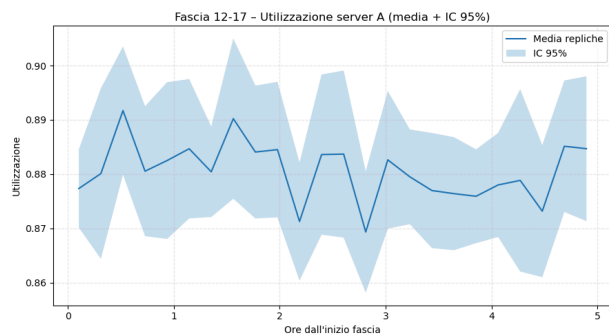


Fascia Oraria 9:00-12:00

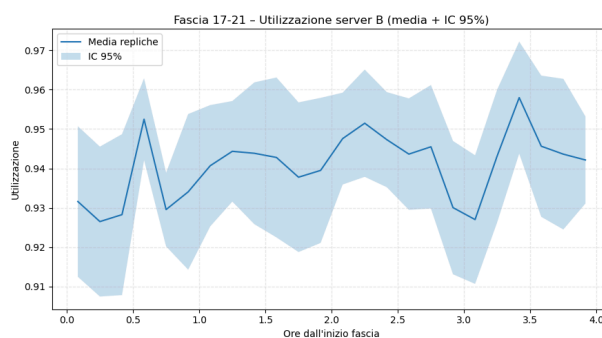
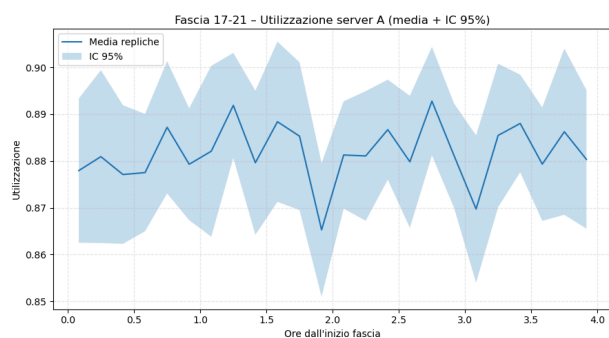
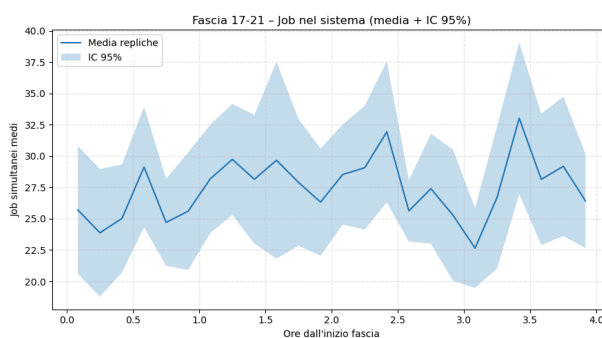
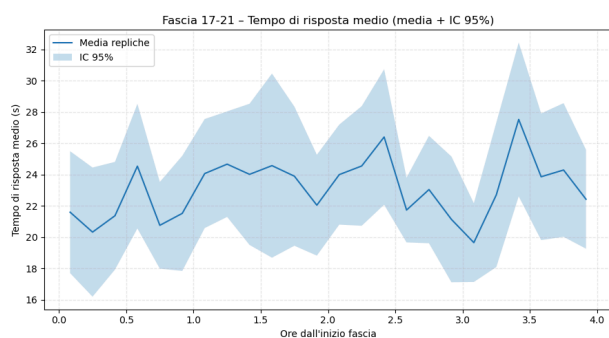


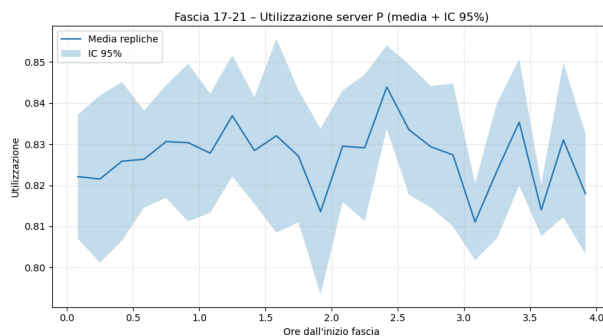
Fascia Oraria 12:00-17:00



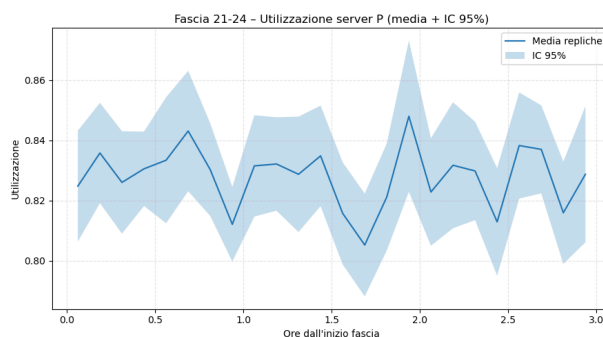
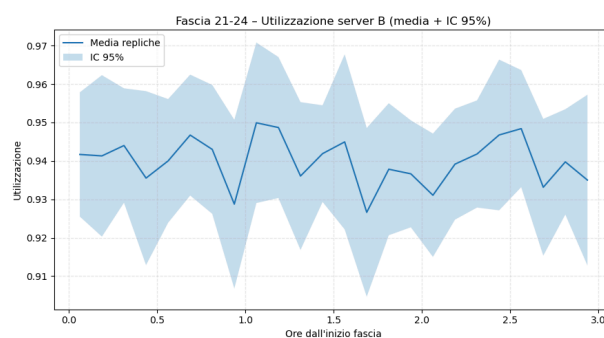
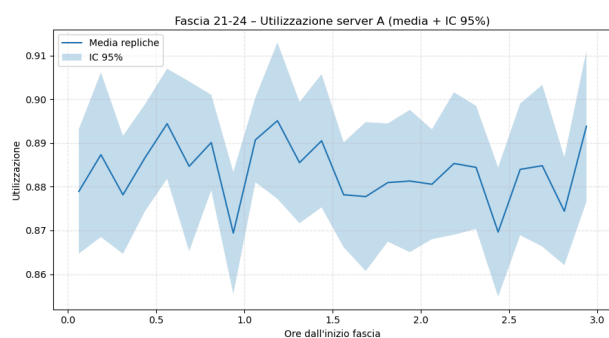
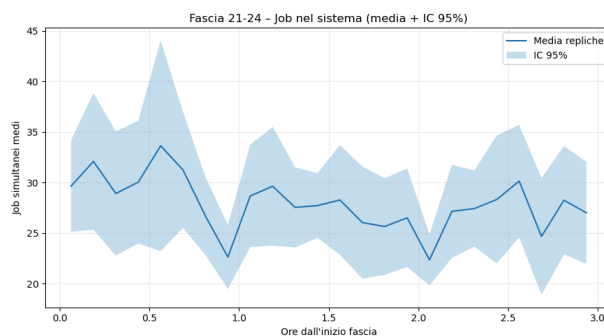
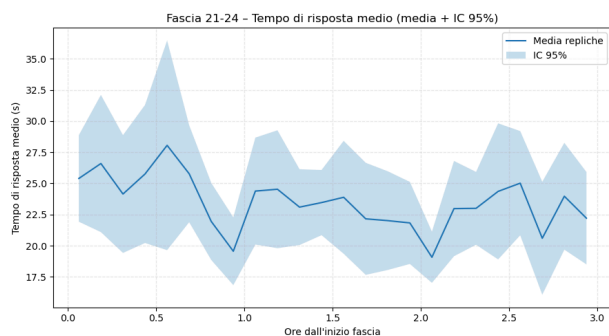


Fascia Oraria 17:00-21:00





Fascia Oraria 21:00-24:00



Come previsto nella simulazione Two-factor authentication orizzonte finito, le metriche risultano ovviamente peggiori, con il massimo incremento delle prestazioni riscontrato nella fascia 12-17.

Nello specifico:

1. Sono stati analizzati in dettaglio sia l'autenticatore a fattore singolo che quello a due fattori, valutando l'impatto potenziale di quest'ultimo sulle **prestazioni e scalabilità** dell'infrastruttura. Sono state esaminate le metriche chiave, inclusa l'analisi delle differenze in termini di **tempo di risposta del sistema** e della **quantità di job gestiti**.
2. Le analisi sulle utilizzazioni dei data center sono state effettuate, ponendo le basi per affrontare l'obiettivo di **minimizzare i tempi di risposta** e **rispettare i requisiti di QoS** nel modello migliorativo.
3. L'analisi ha incluso una **Performance Evaluation in seguito ad un nuovo workload**, tenendo conto dell'aumento stimato del carico di lavoro e dei **nuovi parametri** (tasso di arrivo massimo di 5000 richieste all'ora) derivanti dall'acquisizione della nuova compagnia di grocery online.

Verifica Repliche

Come ultima verifica prima di passare al modello migliorativo si è verificato che le repliche della simulazione fossero effettivamente indipendenti e identicamente distribuite (i.i.d.) e, soprattutto, riproducibili, abbiamo condotto il seguente esperimento.

Durante l'esecuzione delle repliche, sono stati salvati, per ogni run, tutti i seed finali associati a ciascuno stream di generatore di numeri casuali (RNG), ad esempio ARR, A1, B, ecc. Questi seed sono stati registrati nel file **SeedOrizzonteFinito.csv**.

Per testare la riproducibilità, abbiamo scelto la run numero 250, per la quale avevamo già a disposizione il file dei risultati originali (**jobs_run_250.csv**). La procedura è stata la seguente:

1. Abbiamo letto dal file **SeedOrizzonteFinito.csv** i seed finali della run 249, cioè lo stato esatto degli RNG subito prima dell'inizio della run 250.
2. Abbiamo reimpostato tutti gli stream RNG utilizzando quei seed .
3. Abbiamo eseguito la run 250, ottenendo un nuovo file di output (**jobs_run_250_replicated.csv**).

Infine, è stato effettuato il confronto, elemento per elemento, dei due DataFrame al fine di verificare la corrispondenza esatta:

$$jobs_run_250[i][j] - jobs_run_250_replicata[i][j] < \varepsilon \quad \forall i, j$$

In pratica, per ogni riga e per ogni colonna dei due CSV originali e replicati, la differenza è risultata **essere minore di ε , con $\varepsilon=1e-10$** . Poiché molti campi del dataset contengono numeri in virgola mobile, il confronto non può essere effettuato con uguaglianza esatta (che richiederebbe differenza = 0).

I numeri floating-point, infatti, introducono inevitabili micro-variazioni dovute alle normali imprecisioni aritmetiche della rappresentazione IEEE-754.

Per questo motivo, il confronto è stato effettuato applicando una tolleranza numerica.

Questo conferma che:

- il sistema di gestione dei seed multi-stream funziona correttamente;
- ogni run è effettivamente riproducibile, poiché gli RNG vengono ripristinati allo stato corretto;
- la sequenza di eventi generati nella simulazione è deterministica rispetto ai seed;
- le repliche sono i.i.d., in quanto ciascuna parte da uno stato dei generatori completamente indipendente e noto.

Il seguente meccanismo è implementato nella funzione **ReplicaRunEConfronta(target_run, csv_jobs_originale, csv_seed, settimana)**, che consente di replicare e confrontare run di simulazione per verificare la riproducibilità e la correttezza del sistema di gestione multi-stream dei generatori di numeri casuali.

In particolare, a partire da una run di riferimento, la funzione carica i dati originali relativi ai job generati e il file contenente gli stati finali (seed) di ogni stream RNG salvati a conclusione di ogni run. Viene quindi estratto lo stato esatto dei generatori immediatamente precedente alla run da replicare, cioè i seed finali della run immediatamente precedente.

Questi seed vengono utilizzati per reimpostare ogni stream RNG allo stato corrispondente, garantendo così che la replica della run parta da una condizione identica a quella originale. Successivamente, la simulazione viene eseguita con gli stessi parametri e profilo temporale utilizzati in origine, producendo un nuovo set di dati.

Il confronto tra i risultati originali e quelli replicati avviene nel seguente modo: si confrontano i valori cella per cella applicando una tolleranza numerica quando necessario, per tenere conto di eventuali arrotondamenti o lievi differenze numeriche.

Se tutti i valori corrispondono entro la tolleranza stabilita, si conclude che la run è perfettamente riproducibile, confermando il determinismo della sequenza RNG rispetto agli stati iniziali e la correttezza del sistema di gestione multi-stream. Questo meccanismo garantisce inoltre che ogni run parta da uno stato RNG noto e indipendente, assicurando l'indipendenza e la distribuzione identica (i.i.d.) delle repliche.

In output, la funzione restituisce i dati originali, quelli replicati e un indicatore booleano che segnala l'esito del confronto.

```

--- REPLICHA RUN 250: seeds PRIMA ---
ARR: 114815798
A1: 1272253186
A2: 1016438085
A3: 1645528675
B: 1047131173
P: 872618303
1.176
--- REPLICHA RUN 250: seeds DOPO ---
ARR: 1765575998
A1: 1356776204
A2: 1610953857
A3: 840894266
B: 1687832578
P: 874256064
CSV replicato salvato in: PMCSN/jobs_run_250_replicata.csv
Massima differenza numerica (|orig-rep|): 2.910e-11
✅ I due DataFrame sono uguali entro la tolleranza numerica.
Risultato verifica riproducibilità: True

```

Modello Migliorativo

Al fine di avvicinare il sistema in analisi a scenari realistici tipici del dominio dell'e-commerce, è stata adottata la modellazione di eventi di perdita. Questa scelta consente di evidenziare difetti caratteristici del modello di base, andando oltre la mera stabilità del sistema e includendo nuovi indicatori di qualità del servizio (QoS) significativi nel contesto di un sistema di e-commerce reale.

E' stato scelto di modellare i seguenti eventi di perdita correlati a dei QoS del sistema:

Le probabilità di perdita sono state elaborate adattando quanto citato dalla fonte [Modeling and Performability Evaluation of e-Commerce Systems \(extended abstract\)](#) al nostro caso d'uso. Questo processo ha comportato una ridistribuzione, attraverso vari passaggi, delle perdite in base allo specifico scenario, al fine di assegnarle ai vari server attivi in conformità con i tempi di elaborazione.

- **Timeout al processamento del carrello:** poiché il server B rappresenta il livello logico del sistema in analisi, esso gestisce tutte le operazioni relative al processamento del carrello, inclusa la gestione dell'ordine dei prodotti in esso contenuti. Tra le sue principali funzioni vi è l'interazione con la base dati sottostante, necessaria per associare i prodotti selezionati agli articoli fisici. Il tempo richiesto per questa operazione varia generalmente tra 4 e 6 secondi. Per modellare la probabilità di perdita

dovuta a un'attesa eccessiva in questa fase, si è scelto di adottare la seguente funzione matematica: La probabilità di perdita varia in funzione del tempo di risposta del server B/B2:

- È pari a 0 se il tempo di risposta è inferiore a 4 secondi.
- Segue la formula specificata se il tempo di risposta è compreso tra 4 e 6 secondi.
- È pari a 1 se il tempo di risposta è superiore a 6 secondi.

Esempio con numeri reali:

$$P(t) = 0 \quad \text{per } t \leq 4$$

$$P(t) = \left(\frac{t-4}{2}\right)^6 \quad \text{per } 4 < t \leq 6$$

$$P(t) = 1 \quad \text{per } t > 6$$

Esempio:

$t = 4.25$	$P(t) = 0.000003$
$t = 4.5$	$P(t) = 0.000244$
$t = 5$	$P(t) = 0.015625$
$t = 5.5$	$P(t) = 0.177978$
$t = 5.75$	$P(t) = 0.448795$

- **Timeout al processamento del pagamento:** il tempo di risposta del server P, gestito da un provider esterno di servizi di pagamento online, deve essere inferiore a 12 secondi. Superato tale limite la richiesta viene persa. Le ragioni che giustificano questa scelta numerica sono le seguenti:
 - **Vincoli esterni legati al provider di pagamento:** il server P, appartenendo ad un provider esterno fa sì che il tempo di risposta di questo non sia in pieno controllo del sistema in analisi. La soglia dei 12 secondi riflette un limite ragionevole, per garantire che la transazione sia gestita anche in condizioni di picco di carico.
 - **Bilanciamento tra esperienza utente e robustezza del sistema:** studi di usabilità e pratiche di mercato mostrano che gli utenti tendono ad abbandonare un processo di acquisto se il tempo di attesa supera i 15 secondi. Fissando il limite a 12, si resta al di sotto di questa soglia critica.
 - **Coerenza con il Service Level Agreement:** i principali provider di pagamento online (PayPal, Stripe, Adyen) garantiscono tempi medi di elaborazione molto più bassi, ma prevedono margini più ampi per gestire casi occasionali. Imporre un limite di 12 secondi consente di allinearsi a SLA realistici senza compromettere la continuità del servizio.

Per modellare la probabilità di perdita dovuta a un'attesa eccessiva in questa fase, si è scelto di adottare la seguente funzione matematica: Per modellare la probabilità di perdita dovuta a

un'attesa eccessiva in questa fase, si è scelto di adottare la seguente funzione matematica: La probabilità di perdita varia in funzione del tempo di risposta del server di P :

- È pari a 0 se il tempo di risposta è inferiore a 10 secondi.
- Segue la formula specificata se il tempo di risposta è compreso tra 10 e 12 secondi.
- È pari a 1 se il tempo di risposta è superiore a 12 secondi.

Esempio con numeri reali:

$$P(t) = 0 \quad \text{per } t \leq 10$$

$$P(t) = \left(\frac{t-10}{2}\right)^6 \quad \text{per } 10 < t \leq 12 \quad \text{Esempio:}$$

$$P(t) = 1 \quad \text{per } t > 12$$

$t = 10.5$	0.000244
$t = 10.75$	0.002780
$t = 11$	0.015625
$t = 11.5$	0.177978
$t = 11.75$	0.448795

Tutti gli eventi di perdita dovuti a timeout riducono il throughput utile del sistema, poiché la richiesta non completa il flusso previsto.

Limitazioni del modello base

L'obiettivo principale di questa analisi è duplice: da un lato confrontare il modello di base con i requisiti di qualità del servizio (QoS) precedentemente definiti, dall'altro valutare il comportamento del sistema in scenari realistici, sulla base dei workload proposti.

Come indicato nella specifica e successivamente dimostrato sia per via analitica sia tramite simulazione, il sistema in analisi nella sua configurazione base presenta un evidente *bottleneck*. In particolare, al verificarsi di un picco di carico pari a $\lambda = 1.2 \text{ req/s}$, l'utilizzazione del sistema si avvicina a valori prossimi al collasso. Ne consegue che il primo limite è individuabile in corrispondenza del secondo obiettivo della specifica, ossia il raggiungimento del 115% del carico originario. Come dimostrato dagli esperimenti eseguiti, superata la soglia di utilizzazione > 1 , il tempo medio di servizio e il numero medio di utenti presenti nel sistema divergono, evidenziando l'instabilità intrinseca del modello di base, che dunque non è in grado di sostenere tali carichi senza modifiche strutturali.

Un ulteriore limite riguarda i QoS stabiliti: attraverso calcoli analitici e simulazioni è possibile dimostrare che, per valori di utilizzazione superiori all'86%, il server B manifesta tempi di servizio ben oltre il vincolo di 6 secondi previsto. In queste condizioni, il tasso di perdite tende sostanzialmente al 100%.

Di seguito, viene fatta una verifica a livello analitico per confermare quanto esposto.

Supponiamo che per ipotesi, il tempo medio di servizio di un job sia $E[S]_B = 0.8$ s. Impostiamo come tempo di risposta limite da parte dal server B quello imposto dal QoS, cioè impostiamo come soglia massima 6 secondi.

$$HP : E[T]_{S_B \max} = \frac{E[S]_B}{1 - \rho_B} \text{ da cui è semplice ricavare } \rho_B = 1 - \frac{E[S]_B}{E[T]_{S_B \max}}$$

$$\text{Sostituendo i valori, si ottiene } \rho_B = 1 - \frac{0.8 \text{ s}}{6 \text{ s}} = 0.866667 = 86.6667\%$$

Da qui, andiamo a ricavare il tasso di arrivo corrispondente a tale utilizzazione:

$$\rho_B = \frac{\lambda_{\text{sorgente}}}{\mu_B} = E[S]_B * \lambda_{\text{sorgente}} \Rightarrow \lambda_{\text{sorgente}} = \frac{\rho_B}{E[S]_B} = \frac{0.866667}{0.8} = 1.083333 \text{ req/s}$$

Quindi, con un carico di gran lunga inferiore al massimo, si presentano evidenti limiti del sistema in analisi.

Sempre considerando il modello base, è opportuno osservare un ulteriore aspetto. Il secondo obiettivo della specifica prevede l'analisi di un carico incrementato al 115%, combinato con l'applicazione del primo obiettivo, ossia l'implementazione della two-factor authentication, che introduce un tempo aggiuntivo nel processamento di ciascun job relativo al tracciamento degli ordini.

Va inoltre sottolineato che, nel modello base, non sono considerati eventi di perdita. Alla luce di queste condizioni, risulta evidente che, con il carico incrementato, il server A diventa il nuovo bottleneck, poiché la sua utilizzazione supera 1:

$$\lambda_A = \lambda_{\text{sorgente}} + \lambda_B + \lambda_P = 3 * \lambda_{\text{sorgente}} = 1.38 * 3 = 4.14 \text{ req/s}$$

$$E[S]_A = \frac{E[S]_{A \text{ visita 1}} + E[S]_{A \text{ visita 2}} + E[S]_{A \text{ visita 3}}}{3} = \frac{0.2 + 0.4 + 0.15}{3} = 0.25 \text{ s}$$

$$\rho_A = \frac{\lambda_A}{\mu_A} = E[S]_A * \lambda_A = 1.035 > 1$$

Obiettivi

- Il **primo obiettivo** del modello migliorativo consiste nella **risoluzione dei *bottleneck*** individuati inizialmente nel server B e successivamente nel server A. Tale intervento risulta indispensabile per garantire lo sviluppo di un sistema capace di mantenere prestazioni adeguate anche sotto i carichi di lavoro previsti.
- Il **secondo obiettivo** riguarda la **valutazione del comportamento** del sistema migliorativo in presenza degli **stessi carichi applicati al modello base**, con l'intento di ottenere metriche stabili e non soggette a divergenza nel tempo.
- Il **terzo obiettivo** è assicurare il **rispetto dei vincoli di *Quality of Service* (QoS)** relativi ai tempi di servizio presso ciascuna stazione, così da ridurre al minimo le perdite di *job*, corrispondenti, nel contesto applicativo, a ordini reali di *grocery* e, di conseguenza, minimizzare le perdite economiche per il gestore del sistema. In particolare, il sistema deve dimostrare un comportamento robusto anche in presenza di picchi di carico, considerando anche l'autenticazione a due fattori e un incremento del carico base fino al 115% rispetto al valore nominale.

Modello Concettuale

Soluzione proposta

La soluzione proposta prevede l'introduzione di un secondo server, denominato B2, identico all'originale, da attivare e disattivare dinamicamente secondo una logica di scale up e scale down. L'attivazione di B2 è subordinata al superamento di una soglia di congestione nel server B, così da assicurare il rispetto dei vincoli di QoS. In questo scenario, il server B2 prende in carico il workload eccedente, permettendo al server principale di ridurre progressivamente l'accumulo di richieste.

Successivamente, è prevista l'applicazione di una politica di load balancing che consenta una ripartizione equa del carico tra i due server, superata la fase di "traboccamento" iniziale verso B2. Parallelamente, è contemplata la possibilità di disattivare il server B2 qualora l'utilizzazione di B si riduca al di sotto di una soglia prestabilita, evitando così un uso inefficiente delle risorse.

Non è stata invece perseguita la soluzione alternativa di sostituire il server B con un'unica macchina più potente, poiché tale opzione avrebbe reso eccessivamente semplice e poco rappresentativa la fase di modellazione, non consentendo un'analisi approfondita dei meccanismi di scalabilità e bilanciamento del carico.

Per le ragioni già presentate, è stata adottata la stessa soluzione pensata per il server B, aggiungendo quindi con lo stesso approccio un server A2, a seconda delle esigenze del sistema.

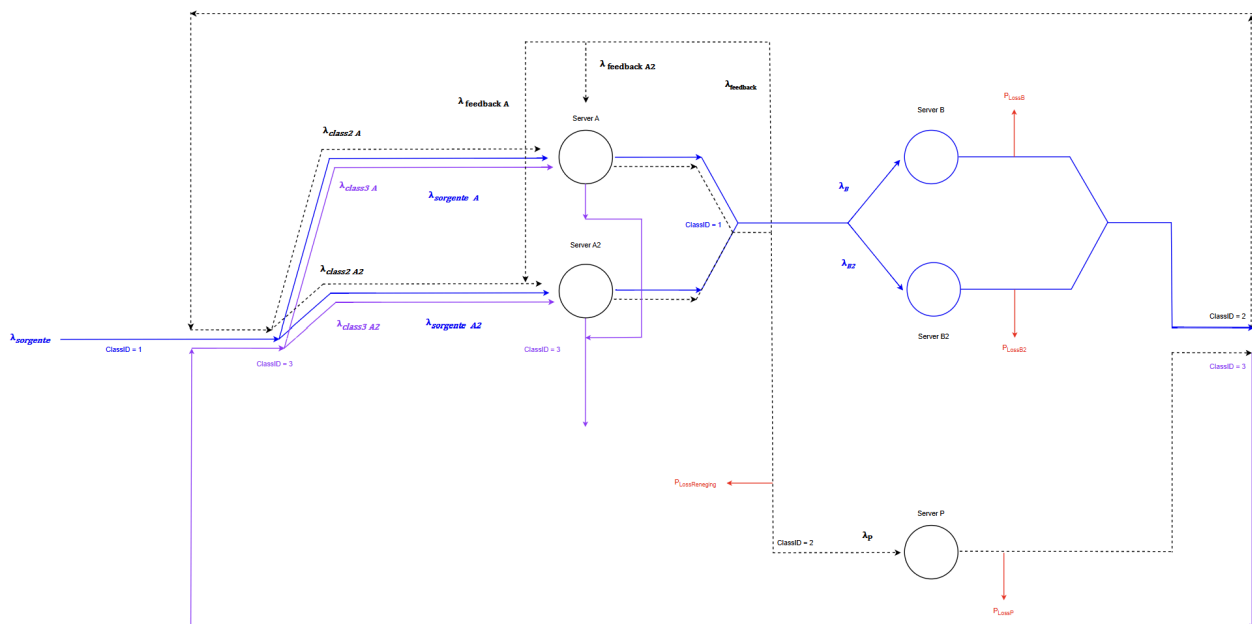
Infine un meccanismo di feedback A/A2

Il principio alla base del meccanismo è il seguente: una frazione dei job, definiti **probe jobs**, viene utilizzata come sentinella per misurare la latenza osservata su P. Ogni volta che un probe attraversa P, la sua latenza (tempo di risposta) viene registrata in una finestra mobile di osservazioni. Su questa finestra si calcola la latenza media osservata, indicata come $\hat{L}$. Il valore di $\hat{L}$ viene poi confrontato con una soglia prefissata (L_{target}):

- se $\hat{L}$ risulta superiore a L_{target} , significa che P è congestionato e la probabilità di instradare nuovi job a P viene ridotta;
- se invece $\hat{L}$ è inferiore o uguale a L_{target} , la probabilità di instradamento verso P viene aumentata.

L'aggiornamento della probabilità non avviene in modo istantaneo, ma graduale, tramite un fattore di adattamento (γ), che regola la velocità con cui il sistema reagisce alle variazioni della latenza.

Il sistema oggetto di analisi è rappresentato dal seguente *System Diagram*:



Politiche di scheduling dei job all'interno dei server

Le politiche di scheduling adottate rimangono conformi a quanto definito nel modello di riferimento. Sono stati effettuati diversi esperimenti al fine di valutare le prestazioni ottenibili mediante l'impiego di strategie alternative, quali lo scheduling size-based con priorità e la coda FIFO. Tali approcci, tuttavia, non hanno prodotto risultati soddisfacenti: in particolare, i job caratterizzati da tempi di servizio più elevati tendevano a subire un peggioramento delle performance rispetto alla soluzione già implementata basata sul processor sharing puro.

Eventi modellati

Gli eventi modellati all'interno del modello migliorativo coincidono con quelli definiti nel modello base.

L'unica modifica sostanziale riguarda l'introduzione di server gemelli per le stazioni A e B, al fine di supportare le operazioni di *scale up* e *scale down*. In ogni caso, i processi di arrivo e completamento presso tali stazioni risultano del tutto analoghi a quelli descritti nel modello base; per completezza, essi vengono brevemente richiamati di seguito, evitando inutili ripetizioni. È importante sottolineare che un *job* servito da un server gemello viene gestito esattamente secondo le stesse modalità con cui verrebbe servito dal server originale.

- **Arrivo di un job presso una stazione** (che coincide con l'inizio del servizio).
- **Completamento di un job presso una stazione** (che coincide con il suo arrivo alla stazione successiva o con l'uscita dal sistema).

Stato del sistema

Lo stato complessivo del sistema è dato dalla combinazione (o unione) degli stati dei singoli centri di servizio. Ciascun centro adotta una disciplina di servizio Processor Sharing (PS), per cui il suo stato è descritto dal numero di richieste attualmente in servizio presso di esso.

Lo stato del sistema è rappresentato dalla seguente tupla,

$$[(n_{a1}, n_{a'1}), (n_{a2}, n_{a'2}), (n_{a3}, n_{a'3}), (n_{b1}, n_{b'2}), n_p]$$

in cui:

Server A

Il server A può servire contemporaneamente richieste appartenenti a più classi.

Pertanto, il suo stato è rappresentato da una tupla $((n_{a1}, n_{a'1}), (n_{a2}, n_{a'2}), (n_{a3}, n_{a'3}))$, dove:

- n_{a1} indica il numero di job di **classe 1 nel server A1**
- $n_{a'1}$ indica il numero di job di **classe 1 nel server A2**
- n_{a2} indica il numero di job di **classe 2 nel server A**
- $n_{a'2}$ indica il numero di job di **classe 2 nel server A2**
- n_{a3} indica il numero di job di **classe 3 nel server A**
- $n_{a'3}$ indica il numero di job di **classe 3 nel server A3**

Ciascun job è inoltre associato a un identificatore di classe (*classID*), che ne rappresenta lo stato di avanzamento all'interno del sistema.

Server B - Server B2

Il server B gestisce esclusivamente job di classe 1.

Di conseguenza, il suo stato è descritto $(n_{b1}, n_{b'2})$:

- n_{b1} : numero di richieste di classe 1 attualmente in servizio presso B.
- $n_{b'1}$: numero di richieste di classe 1 attualmente in servizio presso B2.

Server P

Analogamente, il server P gestisce unicamente job di classe 2.

Il suo stato è quindi rappresentato dal numero:

- n_p : numero di richieste attualmente in servizio presso P.

Inoltre, i server del sistema possono trovarsi in due stati distinti:

- **idle**: nessun job presente;
- **busy**: uno o più job attivi, che condividono equamente la capacità di servizio secondo la politica di Processor Sharing.

Modello delle specifiche

Distribuzioni

Le distribuzioni utilizzate nel modello migliorativo sono esponenziali come nel modello base, con parametro di arrivo variabile nell'intervallo $\lambda \in [0.5, 1.4]$.

Tutti i server adottano un tempo di servizio estratto da una distribuzione esponenziale, ciascuno di parametro μ_i con $i \in [A_1, B, A_2, P, A_3]$.

Modellazione dei centri

- **Server A - Front end (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è caratterizzato da:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale
- 1 server
- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo** complessivo in entrata al server A è

$$\lambda_A = \lambda_{class1} + \lambda_{class2} * P_{Load\ balancing A} + \lambda_{class3} * P_{Load\ balancing A} + \lambda_{feedback} * P_{Load\ balancing A}$$

dove i , λ scritti sopra, nello specifico sono calcolati come:

- $\lambda_{class1} = \lambda_{sorgente} * P_{Load\ balancing A}$
- $\lambda_{class2} = \lambda_B * (1 - P_{LossB}) + \lambda_{B2} (1 - P_{LossB2})$
- $\lambda_{class3} = \lambda_P * (1 - P_{LossP})$
- $\lambda_{sorgente} \in [0.5, 1.38] \text{ req/s}$

$$- \lambda_{feedback} = \lambda_p * P_{feedback}$$

La probabilità che il sistema di **load balancing** assegni un job al Server A, piuttosto che al suo server gemello, dipende dalle utilizzazioni calcolate tappa per tappa. Per tale motivo si assume che le varie probabilità di instradamento verso il Server A $P_{Load\ balancing A}$ siano differenti tra loro e non possano quindi essere raccolte a fattor comune.

Tassi di servizio: questi seguono una distribuzione esponenziale e dipendono dal classID associato al job che entra in servizio:

- $E[S]_{A,1} = 0.2 \text{ secondi per i job di classID} = 1$
- $E[S]_{A,2} = 0.4 \text{ secondi per i job di classID} = 2$
- $E[S]_{A,3} = 0.1 \text{ secondi per i job di classID} = 3$
- $E[S]_{A,3} = 0.15 \text{ secondi per i job di classID} = 3 \text{ (caso two - factor Authentication)}$

- **Server A2 - Front end (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro rappresenta un server gemello, attivato quando l'utilizzazione del Server A originale supera una soglia prefissata, stabilita in funzione delle condizioni di carico. Esso condivide tutte le caratteristiche del Server A, ad eccezione delle probabilità associate al meccanismo di load balancing. Naturalmente, la somma delle probabilità di assegnazione di un job ad A o ad A2 risulta pari ad 1.

- **Server B - Web App, DB (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è caratterizzato da:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale
- 1 server
- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo** complessivo in entrata al server B è

$$\lambda_B = \lambda_{class1} * P_{Loadbalancing\ B}$$

dove:

- $P_{Loadbalancing\ B}$ è la probabilità che un job sia assegnato a B in base alle utilizzazioni di B e B2

Possibilità di perdita: come descritto precedentemente, le probabilità che un job sia perso durante la sua permanenza nel server B è da imputare al non soddisfacimento dei QoS imposti. Le probabilità di perdita sono state modellate mediante apposite funzioni di perdita, definite in accordo con le specifiche del sistema.

$$P_{LossB}(t) = 0 \quad per\ t \leq 4$$

$$P_{LossB}(t) = \left(\frac{t-4}{2}\right)^6 \quad per\ 4 < t \leq 6$$

$$P_{LossB}(t) = 1 \quad per\ t > 6$$

Tasso di servizio: questo segue una distribuzione esponenziale.

$$- \quad E[S]_B = 0.8 \text{ secondi per i job di classID} = 2$$

- **Server B2 - Web App, DB (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro rappresenta un server gemello, attivato quando l'utilizzazione del Server B originale supera una soglia prefissata, stabilita in funzione delle condizioni di carico. Esso condivide tutte le caratteristiche del Server B, ad eccezione delle probabilità associate al meccanismo di load balancing. Naturalmente, la somma delle probabilità di assegnazione di un job a B o a B2 risulta pari ad 1.

- **Server P - Payment Processing (Sistema a code M/M/1 con disciplina Processor Sharing)**

Questo centro è caratterizzato da:

- arrivi di Poisson
- tempi di servizio con distribuzione esponenziale
- 1 server

- capacità illimitata, in quanto non esistono buffer né code di attesa; ogni job entra immediatamente nel sistema e riceve istantaneamente una frazione della capacità computazionale;
- disciplina di servizio Processor Sharing

Il **tasso di arrivo** complessivo in entrata al server P è

$$\lambda_P = \lambda_{class2} * (1 - P_{feedback})$$

Possibilità di perdita: come descritto precedentemente, le probabilità che un job sia perso durante la sua permanenza nel server B è da imputare al non soddisfacimento dei QoS imposti. Le probabilità di perdita sono state modellate mediante apposite funzioni di perdita, definite in accordo con le specifiche del sistema.

$$P(t) = 0 \quad \text{per } t \leq 10$$

$$P(t) = \left(\frac{t-10}{2}\right)^6 \quad \text{per } 10 < t \leq 12$$

$$P(t) = 1 \quad \text{per } t > 12$$

Tasso di servizio: questo segue una distribuzione esponenziale.

- $E[S]_P = 0.4 \text{ secondi per i job di classID} = 3$
- $E[S]_P = 0.7 \text{ secondi per i job di classID} = 3 \text{ (caso two - factor Authentication)}$

Modello Computazionale

Algoritmi per la next-event simulation

Mantenendo inalterata la costruzione del modello precedente abbiamo aggiunto un algoritmo che ci consente di monitorare l' utilizzo del server B e del Server A in modo tale da andare ad effettuare il meccanismo di Scale up / Scale down dei server. Perciò gli algoritmi utilizzati dalla next-event simulation del modello base sono stati modificati solo in modo tale da permettere suddetto load balancing tra le varie repliche dei server.

Scale up

La logica di base è quella di andare a misurare l'utilizzazione del server $\rho = \frac{\text{busy Time}}{\text{Tempo Totale}}$ e nel caso di superamento di una certa soglia viene configurato il secondo server.

```

Funzione Choose_B_Server(t_now):

  Se t_now > 0 allora
    util_B ← busy_time[B] / t_now
  Altrimenti
    util_B ← 0

  // Verifica se l'utilizzazione supera la soglia
  Se util_B > k_threshold_B allora
    // Se il server B2 non è ancora attivo, crealo
    Se B2 non appartiene a servers allora
      Crea server B2 con:
        name = "B2"
        capacità =
        active = lista vuota
      Imposta busy_time[B2] = 0
      Ritorna "B2" // invia il job al server B2
    Altrimenti
      Ritorna "B" // utilizza il server B di default

```

Lo stesso meccanismo è stato sviluppato anche per il Server A.

Scale Down

Lo **Scale Down** opera in maniera inversa rispetto allo **Scale Up**.

Quando l'utilizzazione del **secondo server** di un determinato centro scende al di sotto della soglia prestabilita, tale server viene **dealloca**to dal sistema, riducendo così le risorse attive e ottimizzando i consumi.

Procedura CHECK_SCALEDOWN(tempo_corrente):

// Calcolo utilizzazione server A

Se tempo_corrente > 0 allora

utilizzo_A $\leftarrow$ busy_time[A] / tempo_corrente

Altrimenti

utilizzo_A $\leftarrow$ 0

// Se esiste il server A2, è inattivo e l'utilizzazione è sotto soglia

Se (A2 $\in$ servers $\wedge$ A2 non attivo) $\wedge$ (utilizzo_A < k_threshold_A_down) allora

Rimuovi A2 da servers

Rimuovi A2 da busy_time

Lo stesso meccanismo è stato sviluppato anche per il Server B.

Probabilità di perdita

Il meccanismo per andare a conteggiare se un job viene perso oppure no è il seguente:

Ad ogni completamento:

1. Si calcola la probabilità $P_{loss}(t)$ tramite la funzione sopra.
2. Si estrae un numero casuale $u \in [0,1]$
3. Se $u < P_{loss}(t)$, il job viene marcato come **perso e rimosso dal flusso**, incrementando il contatore delle perdite.
4. Altrimenti il job prosegue verso lo stadio successivo.

I meccanismi adottati per simulare le funzioni di perdita in precedenza decise sono i seguenti:

Funzione LOSS_PROBABILITY_FROM_SOJOURN_B(t):

Se $t \leq 4$ allora

restituisce 0

Altrimenti se $t \geq 6$ allora

restituisce 1

Altrimenti

$val \leftarrow ((t - 4) / 2) ^ LOSS_B$

Se $val < 0$ allora $val \leftarrow 0$

Se $val > 1$ allora $val \leftarrow 1$

restituisce val

Meccanismi adottati

La configurazione è stata mantenuta invariata e, attraverso l'adeguamento dei parametri prestabiliti, è stata applicata anche al **server P** e al **server A** nella fase di seconda visita.

Rispetto al modello computazionale precedente, in questo vengono introdotti due meccanismi aggiuntivi:

1. **Scaling dinamico** sui server A e B, attivato in base al livello di utilizzazione.
2. **Meccanismo di feedback** per rendere più uniforme il tasso di arrivo sul server P.

Inoltre, vengono definite delle soglie di utilizzazione (**$k_threshold_A_down$** e **$k_threshold_B_down$**) che consentono lo spegnimento dei server A2 e B2 in caso di sottoutilizzo. In questo modo il sistema diventa auto-adattativo, riducendo gli sprechi di risorse.

Algoritmo Proposto per il meccanismo di feedback:

Una volta applicate le soluzioni precedenti, il punto critico del sistema diventa inevitabilmente il server P, che, nel caso di autenticazione a due fattori, raggiunge un'utilizzazione prossima al collasso, alzando di conseguenza i tempi di servizio a livelli incompatibili con i QoS stabiliti. Tuttavia, non è possibile intervenire direttamente sul server P, in quanto si tratta di un servizio esterno fornito da un provider.

Per questo motivo, è stata concepita una soluzione mirata a ridurre il tasso di arrivo delle richieste verso P, evitando il sovraccarico. Le richieste arrivano al server P dopo la seconda visita di un job al server A, e il cluster composto da A e A2 è in grado di elaborarle a un ritmo molto superiore rispetto al modello originale, incrementando il rischio di saturazione su P.

Il principio alla base del meccanismo è il seguente: una frazione dei job, definiti **probe jobs**, viene utilizzata come sentinella per misurare la latenza osservata su P. Ogni volta che un probe attraversa P, la sua latenza (tempo di risposta) viene registrata in una finestra mobile di osservazioni. Su questa finestra si calcola la latenza media osservata, indicata come $\hat{L}$. Il valore di $\hat{L}$ viene poi confrontato con una soglia prefissata (L_{target}):

- se $\hat{L}$ risulta superiore a L_{target} , significa che P è congestionato e la probabilità di instradare nuovi job a P viene ridotta;
- se invece $\hat{L}$ è inferiore o uguale a L_{target} , la probabilità di instradamento verso P viene aumentata.

L'aggiornamento della probabilità non avviene in modo istantaneo, ma graduale, tramite un fattore di adattamento (γ), che regola la velocità con cui il sistema reagisce alle variazioni della latenza. In questo modo, il meccanismo evita oscillazioni instabili e si adatta progressivamente al livello di congestione effettivamente osservato. Il risultato è un **controllo ad anello chiuso** che sfrutta P quando conviene, ma lo alleggerisce quando diventa un collo di bottiglia.

È importante sottolineare due aspetti chiave del meccanismo:

1. L'adattamento si basa esclusivamente sui probe jobs, evitando che le scelte siano influenzate dal traffico complessivo, che dipende dalle politiche generali del sistema.
2. La casualità utilizzata per selezionare i probe e per decidere se un job venga inviato a P è gestita da un generatore separato, indipendente dal resto della simulazione, in modo da non alterare gli arrivi o i tempi di servizio degli altri job.

Quando un job non viene inviato a P, non viene scartato, ma viene **riciclato** su un'alternativa locale, tipicamente uno dei server A. Questo apre il secondo livello della politica: il bilanciamento adattivo tra server gemelli. Infatti, sia A che B possono attivare dinamicamente un gemello (A2 o B2) quando l'utilizzazione supera determinate soglie. Lo spillover dei job esclusi da P segue la stessa logica: se A2 esiste, il carico viene distribuito tra A e A2 proporzionalmente alla loro utilizzazione; altrimenti, tutto viene instradato su A.

Algoritmo di Load Balancing

L'algoritmo implementato per il load balancing nel modello migliorativo è stato progettato per gestire in modo adattativo sia la distribuzione del carico tra i server A e A2, sia tra i server B e B2.

Il principio alla base di questo metodo è che il carico dovrebbe essere distribuito preferenzialmente verso il server meno utilizzato, riducendo così il rischio di congestione e migliorando l'efficienza complessiva del cluster. La procedura funziona come segue:

1. **Calcolo dell'utilizzazione corrente:** per ciascun server attivo (B e, se presente, B2) si calcola la frazione di tempo durante il quale il server è occupato rispetto al tempo totale di osservazione. Questa misura rappresenta l'utilizzazione corrente del server, indicativa del suo livello di congestione.
2. **Conversione in probabilità inversa:** per favorire i server meno caricati, si considera l'inverso dell'utilizzazione di ciascun server. Più un server è libero, maggiore sarà il valore dell'inverso dell'utilizzazione. In questo modo, un server meno utilizzato riceverà una quota maggiore di nuovi job.
3. **Determinazione della probabilità di instradamento:** la probabilità che un nuovo job venga assegnato al server principale B è calcolata normalizzando l'inverso dell'utilizzazione rispetto alla somma degli inversi dei server disponibili. Analogamente, la probabilità di assegnamento al gemello B2 è complementare. Matematicamente, la probabilità di assegnamento a B è data da:

$$P_B = \frac{inv_util_B}{inv_util_B + inv_util_B2}$$

Scelta probabilistica del server: il job viene quindi instradato verso B o B2 generando un numero casuale e confrontandolo con P_B . In questo modo, il sistema assegna i job in modo probabilistico, dando maggior peso ai server meno caricati, ma mantenendo un certo livello di casualità che aiuta a evitare pattern di congestione fissi.

Questo metodo presenta i seguenti vantaggi:

- **Adattamento dinamico:** il bilanciamento tiene conto in tempo reale dell'utilizzazione dei server, permettendo di distribuire i carichi in maniera proporzionale alla disponibilità effettiva.
- **Riduzione della congestione:** instradando preferenzialmente i job verso i server meno occupati, si evita il sovraccarico del server principale e si migliora l'efficienza complessiva del cluster.

Dal punto di vista delle metriche, i job gestiti dai server gemelli vengono monitorati allo stesso modo di quelli passati dai server principali. Ciò garantisce che le statistiche aggregate come tempi di servizio, occupazione media e tempi di attraversamento rimangano coerenti.

Verifica

La verifica analitica è stata condotta assumendo che il sistema:

- operi su un **orizzonte temporale infinito**
- abbia raggiunto un **regime stazionario**
- sia alimentato da un processo di **arrivo con tasso costante**

Nel processo di verifica è stata adottata una configurazione in cui i server A2 e B2 risultano attivi sin dall'inizio. Inoltre, è stata fissata di default a zero la probabilità che un job venga reindirizzato come feedback verso A2, anziché procedere verso il server P.

Questa scelta si è resa necessaria poiché il calcolo analitico delle probabilità di perdita, così come delle probabilità impiegate per il load balancing tra A e A2 e tra B e B2, risulterebbe estremamente complesso e richiederebbe l'introduzione di assunzioni aggiuntive.

Per la valutazione del modello, si è quindi assunto quanto segue:

- il tasso di arrivo esterno è pari a 1.2 richieste/secondo;
- l'autenticazione è a singolo fattore.
- il meccanismo di load balancing è fair, distribuendo equamente il 50% del carico tra A e A2, così come tra B e B2; Questa assunzione consente di modellare dei tassi di arrivo noti e ben definiti.
- la probabilità di perdita di un job in seguito alla visita in B o B2 è circa il 4.159%;
- la probabilità di perdita di un job in fase di checkout è approssimabile a 0.
- Il sistema abbia raggiunto un regime stazionario;
- la probabilità che il meccanismo di feedback rimandi indietro job dal server P ad A/A2 è impostata a zero;

Server A:

Il tasso totale in entrata al server A, come mostrato dal modello delle specifiche, è

$$\lambda_A = \lambda_{class1} * P_{Load\ balancing\ A} + \lambda_{class2} * P_{Load\ balancing\ A} + \lambda_{class3} * P_{Load\ balancing\ A} + \lambda_{feedback} * P_{Load\ balancing\ A}$$

Poiché, come già specificato, il load balancing è assunto come fair, è possibile scrivere il tasso in entrata al server A (per il server A2 i calcoli sono perfettamente identici) nel seguente modo raccogliendo

$$P_{Load\ balancing\ A} = 0.5 \text{ a fattore comune.}$$

Poiché i tassi di arrivo provenienti dalle altre stazioni verso il server A siano equamente distribuiti con la sua replica A2, è possibile assumere che i tassi in entrata siano dimezzati rispetto al totale.

$$\lambda_A = (\lambda_{class1} + \lambda_{class2} + \lambda_{class3} + \lambda_{feedback}) * 0.5$$

dove:

- $\lambda_{class1} = \lambda_{sorgente} = 1.2 \text{ req/s}$
- $\lambda_{class2} = \lambda_B * (1 - P_{LossB}) + \lambda_{B2} * (1 - P_{LossB2})$

Per ipotesi, $\lambda_B = \lambda_{B2}$ in quanto il load balancing è fair, e si assume che B/B2 abbiano probabilità di perdita coincidenti, pari al 4.16%. Inoltre, $\lambda_B = \lambda_{class1} * 0.5$

Quindi, è possibile scrivere:

$$\lambda_{class2} = 2 * \lambda_B * (1 - P_{LossB}) = 2 * 0.5 * \lambda_{class1} * 0.9584 = 1.15008 \text{ req/s}$$

- $\lambda_{class3} = \lambda_P$ in quanto, per ipotesi, il server P non subisce perdite
- $\lambda_{class3} = \lambda_{class2} = 1.15008 \text{ req/s}$ infatti, il tasso di entrata nel server P viene dalla somma dei tassi di uscita dalla seconda visita ai server A/A2, che sommati corrispondono alla somma dei tassi uscenti da B/B2.

Otteniamo così il tasso di entrata totale al server A:

$$\lambda_A = (\lambda_{class1} + \lambda_{class2} + \lambda_{class3}) * 0.5 = (1.2 + 1.15008 + 1.15008) * 0.5 = 1.75008 \text{ req/s}$$

In particolare, i tassi in entrata al server A o in maniera equivalente al server A2, presi singolarmente, risultano essere:

$$\lambda_{A1/A2} = \frac{\lambda_{class1}}{2} + \frac{\lambda_{class2}}{2} + \frac{\lambda_{class3}}{2} = 0.6 + 0.57504 + 0.57504 = 1.75008 \text{ req/s}$$

Per calcolare $E[S]_A$ definiamo una visita generica al server A come un arrivo scelto a “caso” tra tutte le visite. Sia $C \in \{1, 2, 3\}$ la classe del job associata alla fase che lo ha generato. Sia inoltre S il tempo di servizio associato a quel particolare job. Allora:

$$E[S] = E[E[S | C]] = \sum_{i=1}^3 P(C = i) * E[S_{Ai}]$$

La probabilità che una visita casuale sia di classe i non è $\frac{1}{3}$, bensì, a causa dell'introduzione delle probabilità di perdita, deve essere calcolata come:

$$P(C = i) = \frac{\lambda_{class i}}{\lambda_{class1} + \lambda_{class2} + \lambda_{class3}} = \frac{\lambda_{class i}}{\lambda_A}$$

Sostituendo nella equazione precedente, si ottiene:

$$E[S] = \sum_{i=1}^3 \frac{\lambda_{class i}}{\lambda_{class1} + \lambda_{class2} + \lambda_{class3}} * E[S_{Ai}] = \sum_{i=1}^3 \frac{\lambda_{class i}}{\lambda_A} * E[S_{Ai}] = \frac{\sum_i \lambda_{class i} * E[S_{Ai}]}{\lambda_A}$$

Infine, è possibile sostituire i valori:

$$E[S]_A = \frac{0.6 \cdot 0.2 + 0.57504 \cdot 0.4 + 0.57504 \cdot 0.1}{1.75008} = \frac{0.40752}{1.75008} = 0.232857 \text{ s}$$

Da cui ricaviamo l'utilizzazione, il tempo di risposta medio e i tempi di risposta medi per classe.

$$\rho_A = E[S]_A * \lambda_A = 0.232857 * 1.75008 = 0.40752 = 40.752\%$$

$$E[N_s]_A = \frac{\rho_A}{1 - \rho_A} = 0.687820 \quad E[T_s]_A = \frac{E[S]_A}{1 - \rho_A} = 0.393020 \text{ s}$$

$$E[T_s]_{A \text{ visita } 1} = \frac{0.2}{1 - \rho_A} = \frac{0.2}{1 - 0.40752} = 0.349357 \text{ s}$$

$$E[T_s]_{A \text{ visita } 2} = \frac{0.4}{1 - \rho_A} = \frac{0.4}{1 - 0.40752} * 2 = 0.349357 \text{ s} * 2 = 0.698714 \text{ s}$$

$$E[T_s]_{A \text{ visita } 3} = \frac{0.1}{1 - \rho_A} = \frac{0.1}{1 - 0.40752} = 0.168782 \text{ s}$$

Server B:

Poiché nel server A non ci sono perdite, è possibile affermare:

$$\lambda_B = \frac{\lambda_{class1}}{2} = 0.6 \text{ req/s}$$

Da cui è possibile ricavare utilizzazione, numero medio di utenti e tempo di risposta medio.

$$\rho_B = \frac{\lambda_B}{\mu_B} = \frac{0.6}{1.25} = 0.48 = 48\%$$

$$E[N_s]_B = \frac{\rho_B}{1 - \rho_B} = \frac{0.48}{0.52} = 0.923076$$

$$E[T_s]_B = \frac{E[S]_B}{1 - \rho_B} = \frac{0.8}{0.52} = 1.538461 \text{ s}$$

Server P:

Il server P prende in ingresso la somma dei tassi provenienti dal server B e B2:

$$\lambda_p = \lambda_{class2}$$

$$\lambda_p = [\lambda_B * (1 - P_{LossB}) + \lambda_{B2} * (1 - P_{LossB2})]$$

Grazie alle assunzioni già affermate, $P_{LossB} = P_{LossB2} = 4.16\%$ e $\lambda_B = \lambda_{B2} = 0.6 \text{ req/s}$, raccogliendo a fattor comune otteniamo:

$$\lambda_{class2} = 2 * \lambda_B * (1 - P_{LossB}) = 2 * 0.6 * (1 - 0.0416) = 1.15008 \text{ req/s}$$

$$\lambda_p = 1.15008 \text{ req/s}$$

Da cui calcoliamo l'utilizzazione, numero medio di utenti e tempo di risposta medio:

$$\text{Dato } E[S]_p = 0.4 \text{ nel caso two-factor auth, } \mu_p = \frac{1}{E[S]_p}$$

$$\rho_p = \frac{\lambda_p}{\mu_p} = \frac{1.15008}{2.5} = 0.460032 = 46.0032\% \quad E[N_S]_p = \frac{\rho_p}{1 - \rho_p} = 0.851961$$

$$E[T_s]_p = \frac{E[S]_p}{1 - \rho_p} = \frac{0.4}{1 - 0.460032} = 0.740784 \text{ s}$$

I risultati ottenuti dalla verifica analitica sono racchiusi nella seguente tabella e confrontati con i risultati ottenuti dalla simulazione. La simulazione è stata eseguita con seed 123456789 ed è stata eseguita a orizzonte infinito.

Per la simulazione sono stati adottati i seguenti parametri e le seguenti ipotesi:

- Tasso di arrivo esterno pari a 1.2 richieste/secondo, con autenticazione a due fattori.
- Soglie di scale up e scale down opportunamente impostate, in modo tale che i server gemelli A2 e B2 risultino attivi sin dall'avvio della simulazione. Questa scelta è motivata dal fatto che, al momento dell'attivazione di B2 o A2, la maggior parte del carico tende inizialmente a concentrarsi su di esso, generando un *burst* temporaneo. L'attivazione anticipata consente invece di ottenere un bilanciamento del carico più equo. In caso contrario, le metriche relative ad A2 e A risulterebbero non confrontabili, poiché influenzate dalle richieste elaborate da A prima dell'evento di scale up.

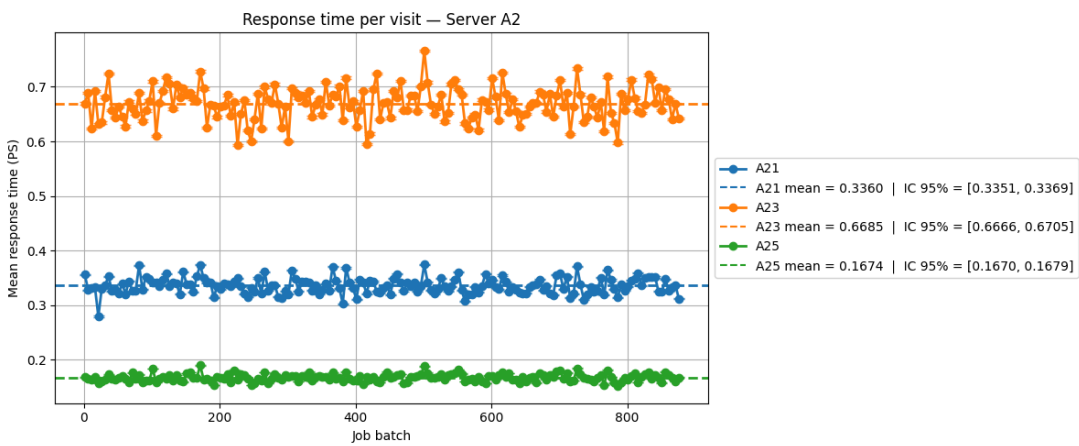
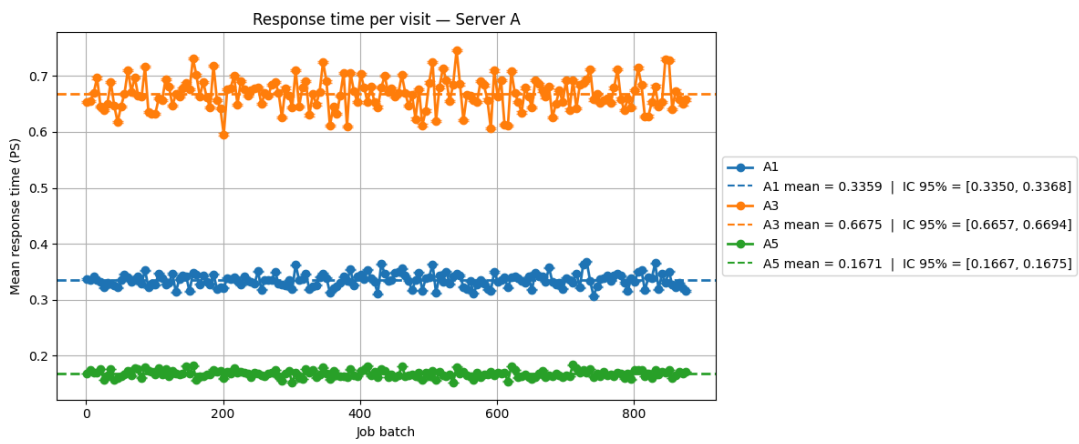
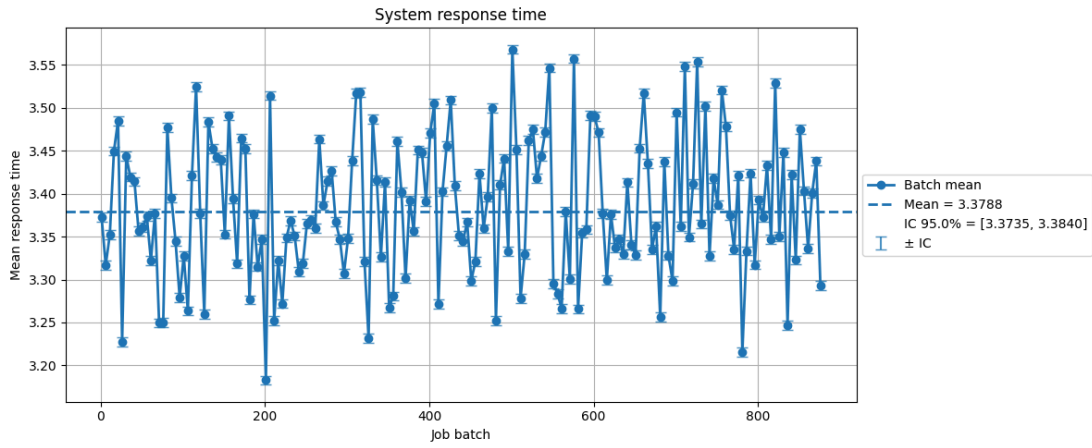
- Probabilità di feedback nella seconda visita ad A2 impostata a 0. In questo modo, tutti i job che escono da A2 vengono instradati direttamente verso il server P, evitando ulteriori rientri nel sistema.

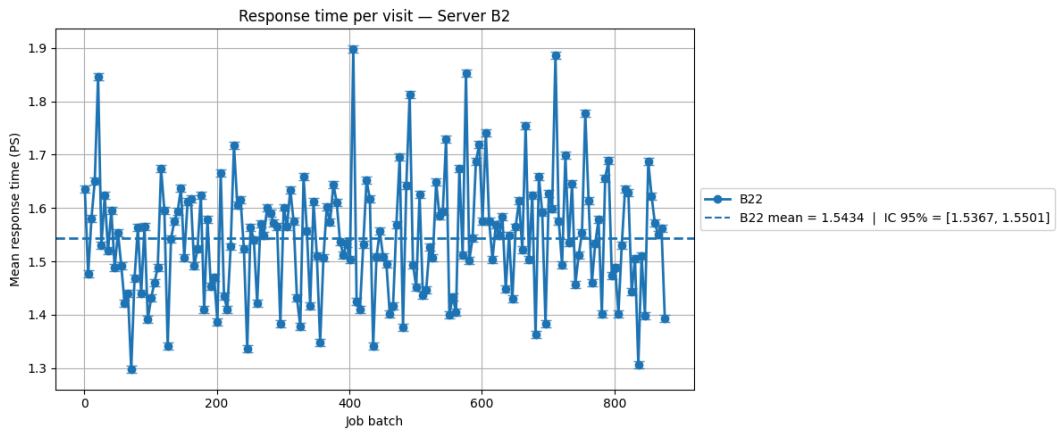
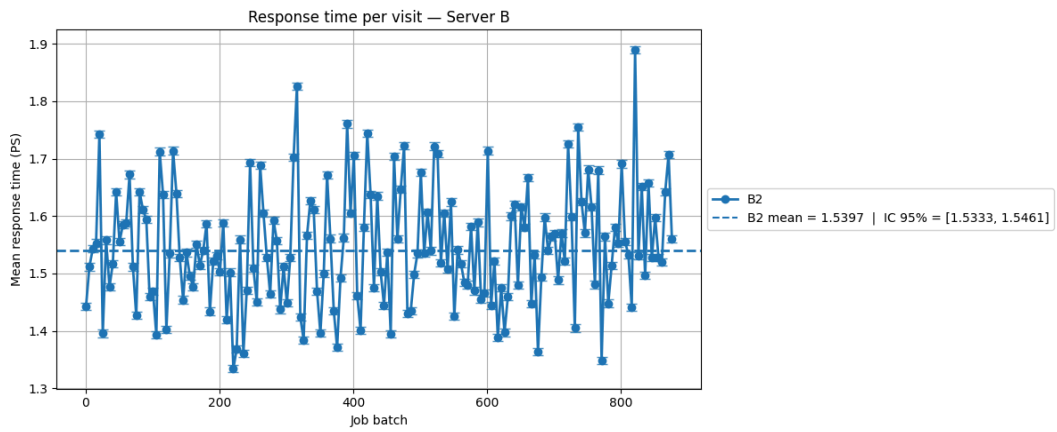
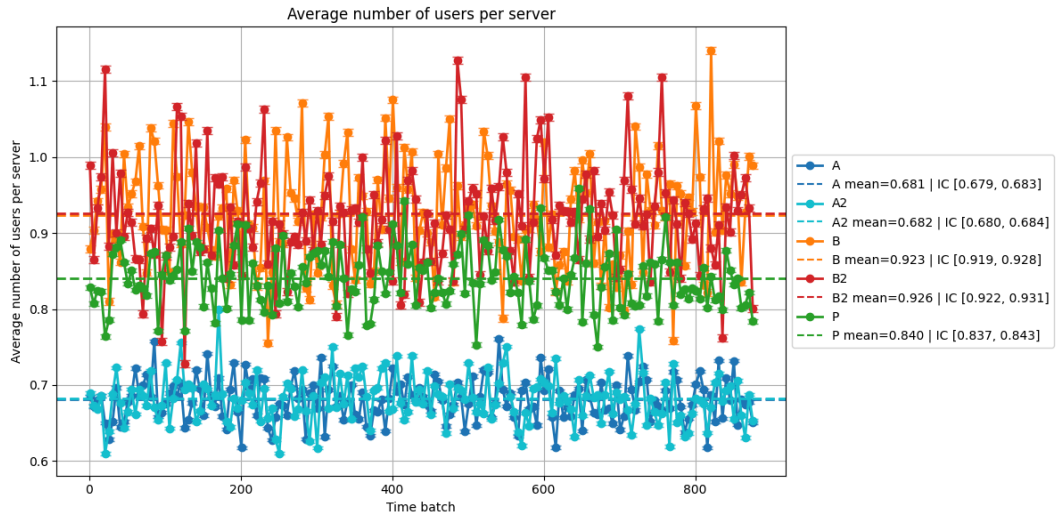
In questo modo si riesce a creare le stesse condizioni spiegate nelle premesse della verifica analitica. I risultati ottenuti dalla simulazione eseguita con i parametri sopra indicati sono i seguenti:

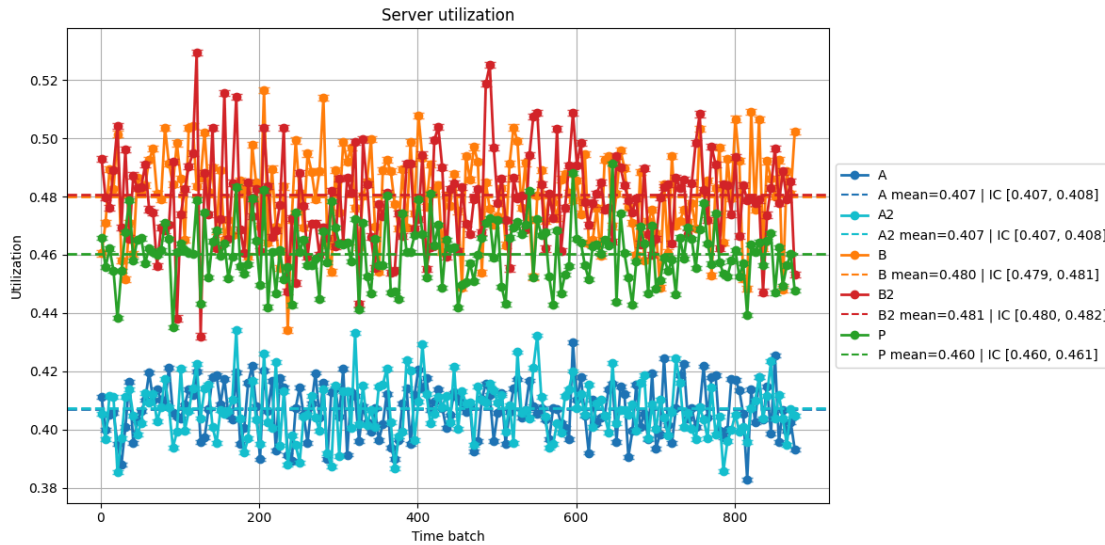
Caso $\lambda_{\text{sorgente}} = \lambda_{\text{max}} = 1.2 \text{ req/s}$ - One factor Authentication

Metrica	Valori medi ottenuti dalla simulazione IC 95% simulazione con t Student	Valori medi ottenuti dall'analisi analitica
Tempo di risposta del sistema	[3.3735, 3.3840]	3.496098
Tempo di risposta A1	A: [0.3350, 0.3368] - A2: [0.3351, 0.3369]	0.349357
Tempo di risposta B	B: [1.5397, 1.5461] - B2: [1.5367, 1.5501]	1.538461
Tempo di risposta A3	A: [0.6657, 0.6694] - A2: [0.6666, 0.6705]	0.698714
Tempo di risposta P	[0.7287, 0.7328]	0.740784
Tempo di risposta A5	A: [0.1667, 0.1675] - A2: [0.1670, 0.1679]	0.168782
Numero di utenti in A	A: [0.679, 0.683] - A2: [0.680, 0.684]	0.687820
Numero di utenti in B	B: [0.919, 0.928] - B2: [0.922, 0.931]	0.923076
Numero di utenti in P	[0.837, 0.843]	0.851961
Utilizzazione server A	A: [0.407, 0.408] - A2: [0.407, 0.408]	40.752%
Utilizzazione server B	B: [0.479, 0.481] - B2: [0.480, 0.482]	48%
Utilizzazione server P	[0.460, 0.461]	46.0032%

Di seguito, è possibile osservare i grafici ottenuti dalle metriche ottenute attraverso la simulazione, eseguita con le modalità espresse precedentemente.







Nel complesso, i risultati ottenuti attraverso la simulazione risultano coerenti con quelli derivati dall'analisi analitica e ne confermano il comportamento atteso. Le metriche locali risultano perfettamente allineate nei due approcci, a dimostrazione che i parametri del sistema e le ipotesi di regime stazionario sono stati modellati in modo consistente.

Le lievi discrepanze osservate nelle metriche globali, come il tempo di risposta complessivo del sistema, non sono imputabili a fluttuazioni statiche residue, ma riflettono il diverso livello di astrazione con cui il sistema viene rappresentato nei due casi. In particolare, l'analisi analitica fornisce una rappresentazione idealizzata del comportamento del sistema, in cui il bilanciamento del carico e gli eventi di perdita vengono incorporati in modo aggregato e indipendente dallo stato istantaneo. La simulazione, riproducendo la dinamicità temporale dei job in maniera più dettagliata, consente agli eventi di perdita e alla competizione tra job sotto la disciplina del Processor Sharing di agire direttamente sull'evoluzione del carico e sulla sequenza delle visite ai server. Questo si traduce in una lievissima sottostima delle metriche globali rispetto alle stime teoriche.

Nel corso della simulazione sono stati generati complessivamente $n = 3_599_303$ job, che, con una batch size pari a **4096**, corrispondono ad un totale di $k = 878$ batch. Per tale valore di k , la soglia empirica di Chatfield per l'autocorrelazione tra batch risulta pari a $|\rho| < 0.067496$. I valori di autocorrelazione calcolati sulle statistiche di simulazione sono riportati di seguito.

```

=== JOB-BASED BATCHES ===
RT_system | rho1=-0.0370 | Chatfield OK=True | Fixed OK=True
RT_A      | rho1=0.0247 | Chatfield OK=True | Fixed OK=True
RT_A2     | rho1=-0.0302 | Chatfield OK=True | Fixed OK=True
RT_B      | rho1=-0.0457 | Chatfield OK=True | Fixed OK=True
RT_B2     | rho1=-0.0678 | Chatfield OK=False | Fixed OK=True
RT_P      | rho1=-0.0106 | Chatfield OK=True | Fixed OK=True

=== TIME-BASED BATCHES ===
A_avg_users | rho1=0.0389 | Chatfield OK=True | Fixed OK=True
A_utilization | rho1=-0.0091 | Chatfield OK=True | Fixed OK=True
B_avg_users | rho1=-0.0281 | Chatfield OK=True | Fixed OK=True
B_utilization | rho1=-0.0273 | Chatfield OK=True | Fixed OK=True
P_avg_users | rho1=-0.0408 | Chatfield OK=True | Fixed OK=True
P_utilization | rho1=-0.0332 | Chatfield OK=True | Fixed OK=True
A2_avg_users | rho1=-0.0627 | Chatfield OK=True | Fixed OK=True
A2_utilization | rho1=-0.0526 | Chatfield OK=True | Fixed OK=True
B2_avg_users | rho1=-0.0764 | Chatfield OK=False | Fixed OK=True
B2_utilization | rho1=-0.0834 | Chatfield OK=False | Fixed OK=True
Throughput | rho1=-0.0217 | Chatfield OK=True | Fixed OK=True

```

Per la stima delle metriche di utilizzazione e del numero medio di job (utenti) presso le stazioni, il dataframe prodotto dalla simulazione è stato partizionato in batch temporali contigui di durata $\Delta T = 3413$ secondi. Tale intervallo temporale corrisponde al tempo medio necessario alla generazione di 4096 job, dato il tasso di arrivo considerato ($\lambda = 1.2$ req/s). Perciò, a ciascun batch viene associata una stima delle metriche di interesse calcolata sui dati raccolti all'interno di tale intervallo temporale. Queste stime sono trattate come medie calcolate batch per batch, analogamente a quanto previsto dal metodo delle Batch Means, e utilizzate per il calcolo degli intervalli di confidenza sulle stesse.

In conclusione, la scelta finale dei parametri $(b, k) = (4096, 878)$ rappresenta un compromesso tra il rispetto delle condizioni di indipendenza dei batch e la necessità di ottenere una varianza inter-batch sufficientemente elevata.

Alla luce di queste considerazioni, la coerenza complessiva dei risultati conferma quindi la validità del modello adottato e l'affidabilità delle conclusioni tratte.

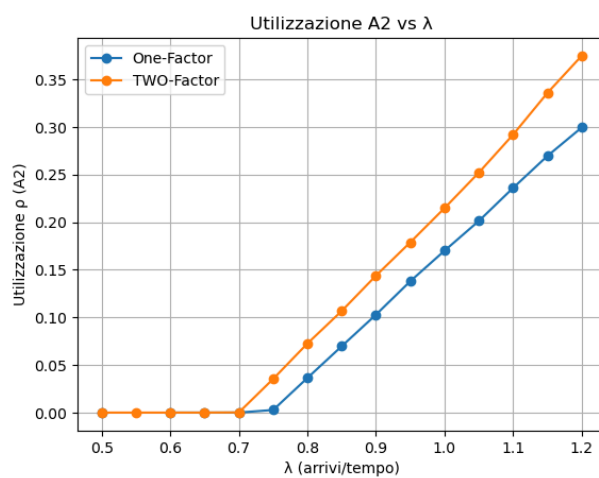
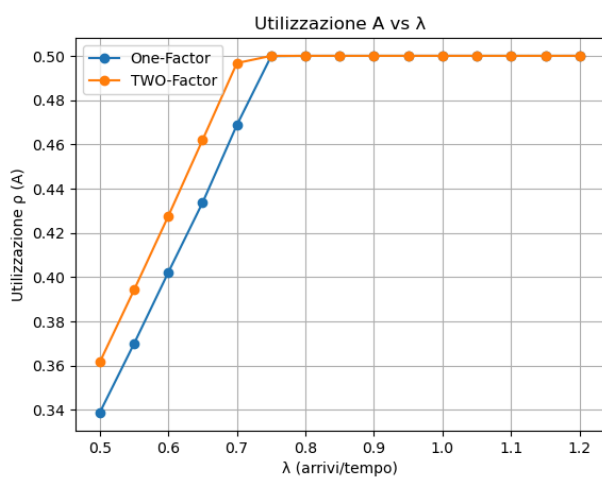
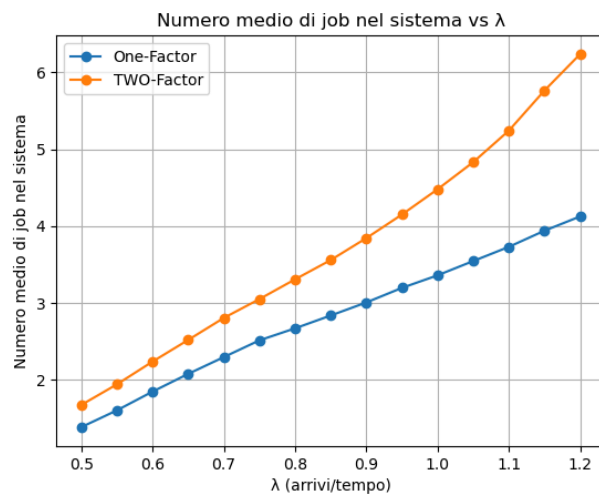
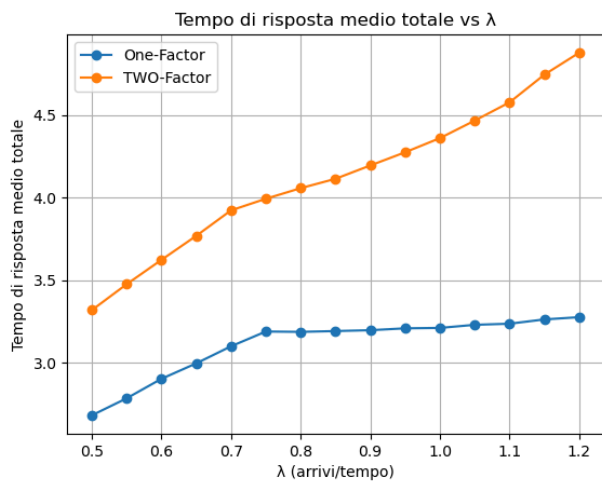
Validazione

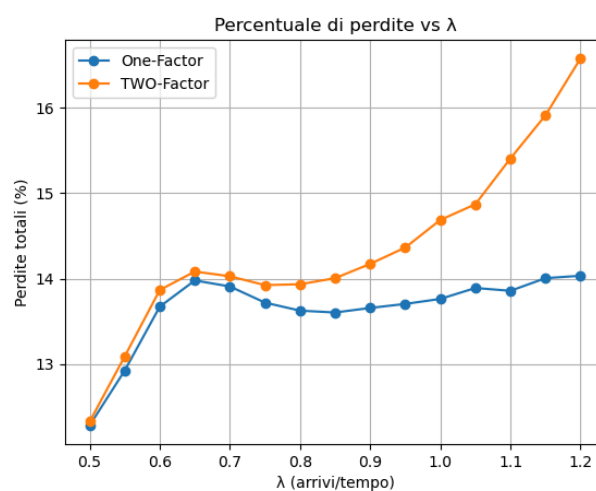
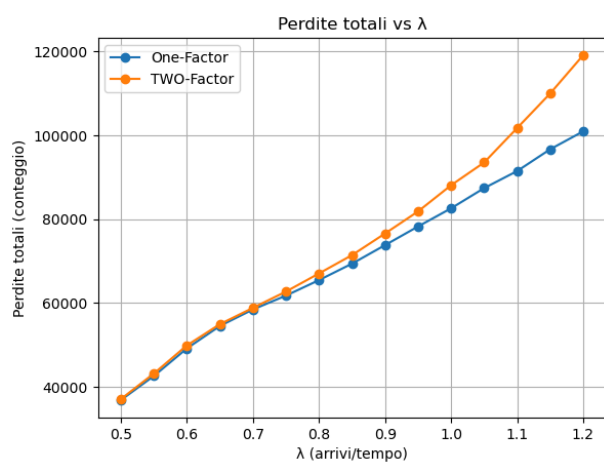
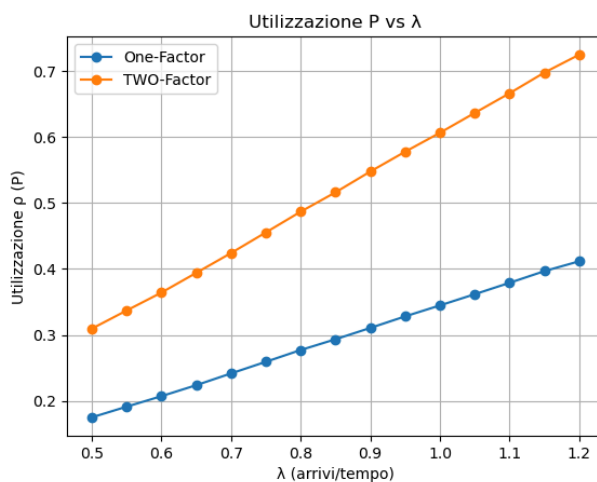
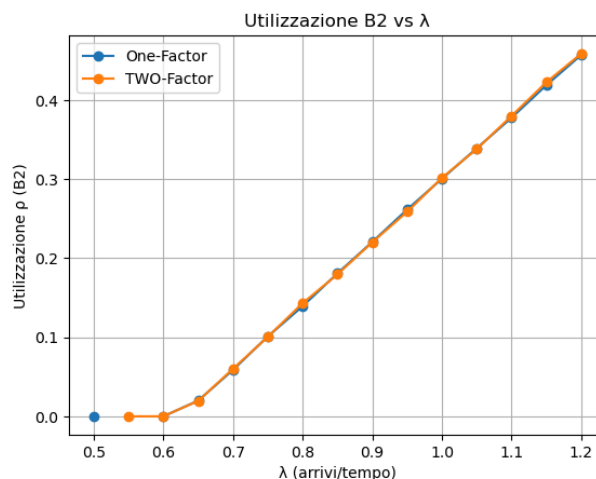
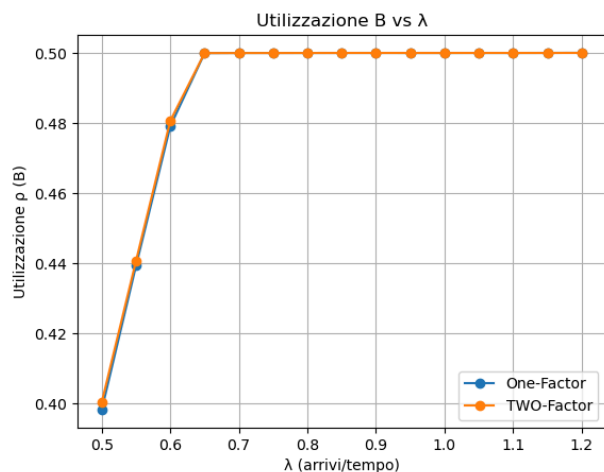
Poiché la specifica di riferimento non prevedeva alcun modello migliorativo, non erano disponibili dati reali al di fuori di quelli ottenuti mediante una verifica analitica. In particolare, sono state simulate due configurazioni anche in questo caso: **One-Factor Authentication** e **Two-Factor Authentication**, analizzando i **tempi di risposta globali del sistema**, il **numero medio di job presenti nel sistema throughput** e in aggiunta **al modello base anche le probabilità di perdita**. La simulazione è stata condotta variando il **tasso di arrivo λ** da **0.5** a **1.2** req/s.

I parametri stabiliti per le threshold di attivazione sono:

- $k_threshold_A_up=0.5$
- $k_threshold_A_down=0.2$
- $k_threshold_B_up=0.5$
- $k_threshold_B_down=0.2$

Di seguito vengono riportati i grafici relativi alla simulazione.





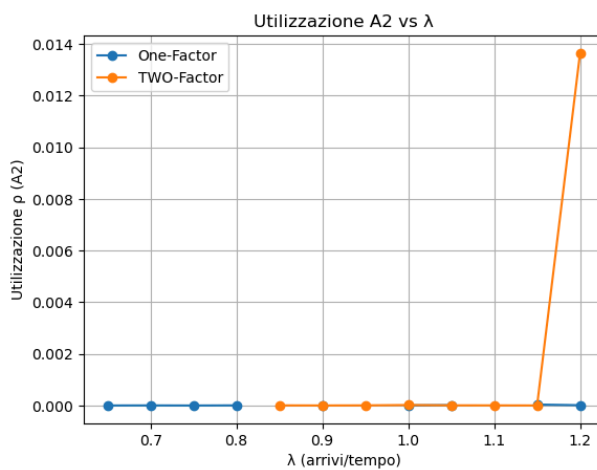
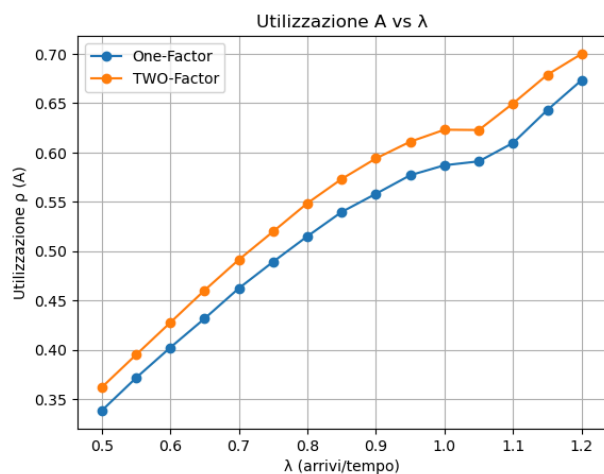
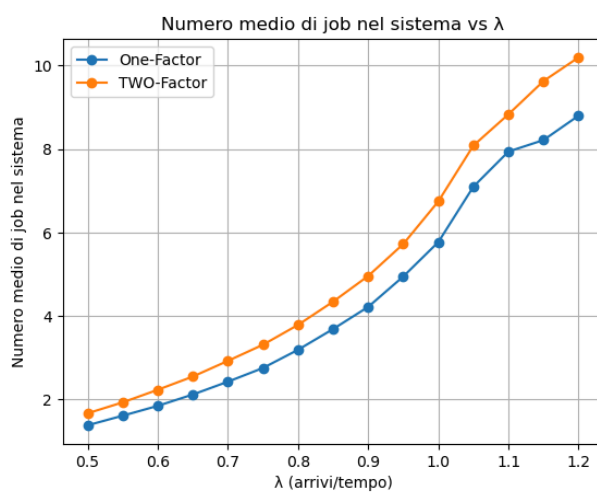
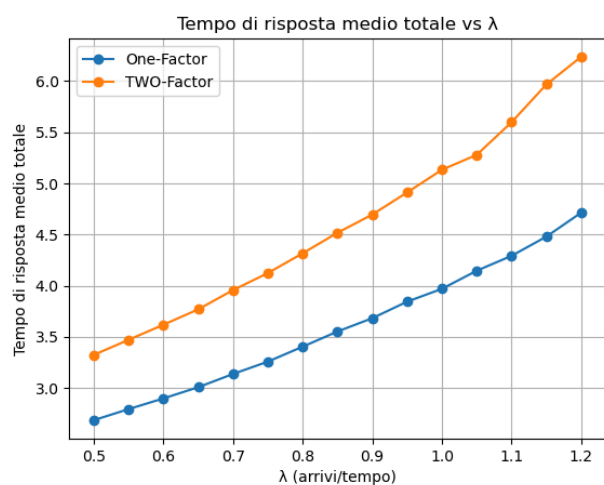
Come notiamo anche se il sistema si trova in sovraccarico con $\lambda=1.2$ comunque abbiamo:

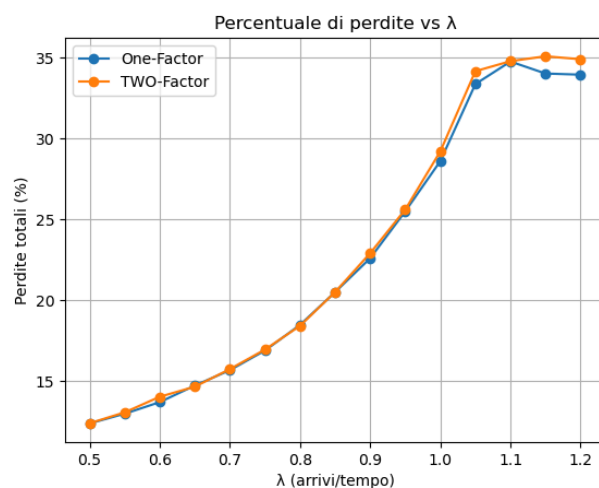
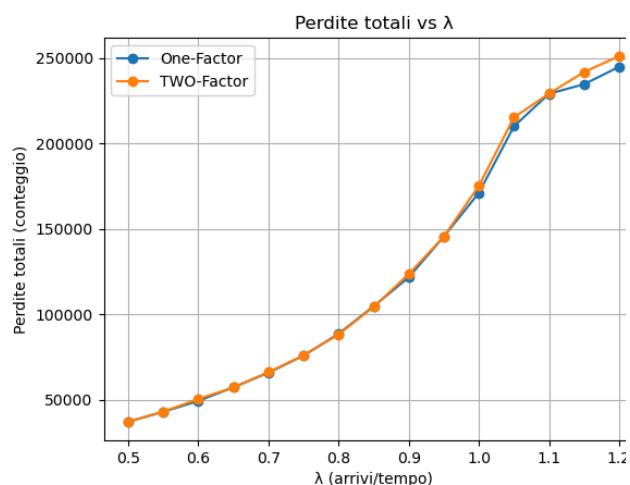
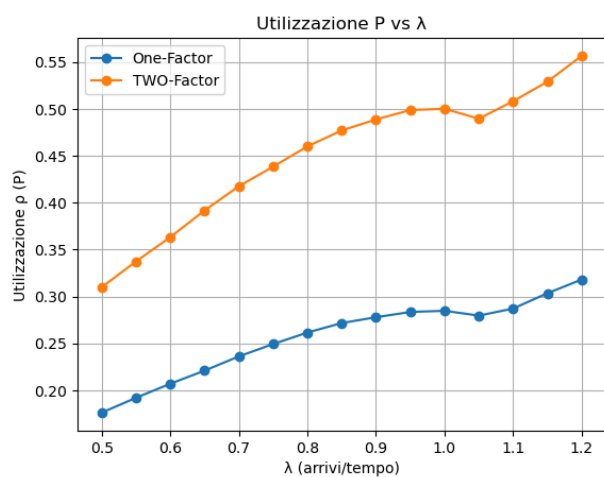
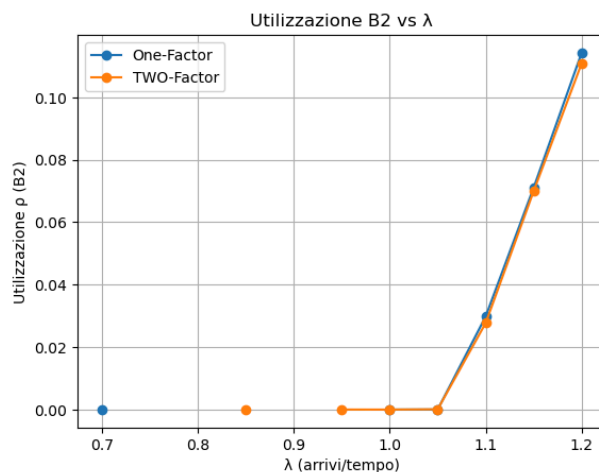
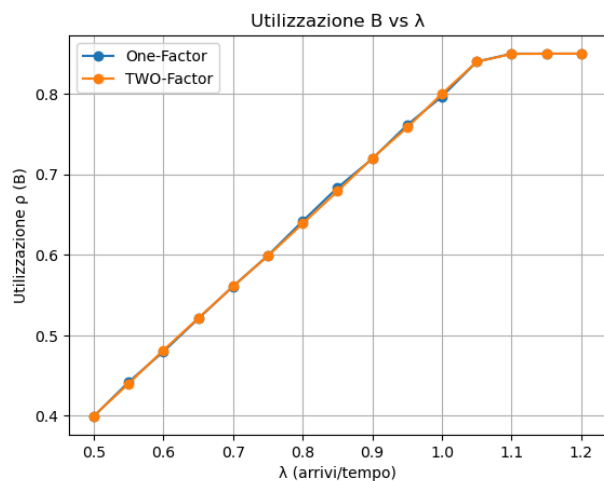
Arrivi totali: 718343 | Persi totali: 119086 | % persi: 16.58%

Risulta rispettare i Qos del nostro sistema.

Ora, andiamo a ripetere la stessa simulazione impostando delle soglie di scale up in modo più alto. Come mostrato dai grafici, si ha un peggioramento generale delle metriche misurate. I parametri stabiliti per le threshold di attivazione sono:

- $k_threshold_A_up=0.7$
- $k_threshold_A_down=0.4$
- $k_threshold_B_up=0.85$
- $k_threshold_B_down=0.4$





Dai grafici si osserva che, con questa configurazione, le metriche peggiorano: lo scaling posticipato del secondo server (sia nel ramo A2 sia in B2) riduce la capacità effettiva e comporta un aumento complessivo delle perdite.

In conclusione, il comportamento del sistema è coerente con le attese e conferma, con buona robustezza statistica, la validità del modello implementato.

Progettazione degli esperimenti

Analisi del collo di bottiglia

Nel modello migliorativo, l'identificazione del collo di bottiglia non può essere effettuata considerando unicamente il massimo tempo medio di servizio, come fatto nell'analisi del bottleneck del modello base, a causa della presenza di server paralleli. In questo contesto, il bottleneck è determinato dal livello che presenta la maggiore domanda di servizio complessiva per job, rapportata alla capacità disponibile.

Per una rete di code con Processor Sharing, il throughput massimo sostenibile dal sistema è limitato dal livello i che minimizza la capacità disponibile rispetto alla domanda di servizio.

Per definizione, il **throughput X** è il numero di job completati nell'unità di tempo. Inoltre, sempre per definizione, il tempo totale di servizio richiesto da un job al centro i -esimo è

$$D_i = V_i * E[S_i]$$

dove V_i = **numero medio di visite** al centro i -esimo **per job**.

Il **tasso di arrivo** al centro i -esimo è $\lambda_i = X * V_i$

L'utilizzazione **per server** considerato il centro i -esimo è:

$$\rho_i = \frac{\lambda_i * E[S_i]}{m_i} = \frac{X V_i E[S_i]}{m_i} = \frac{X D_i}{m_i}$$

Per non divergere, chiaramente tale utilizzazione deve essere minore di 1, cioè $\rho_i = \frac{X D_i}{m_i} < 1$

Da cui

$$X < \frac{m_i}{D_i}$$

Per ciascun livello i , il throughput massimo teorico è quindi:

$$X_{max, i} = \frac{m_i}{D_i}$$

dove m_i è il numero di server paralleli nell' i -esimo livello e D_i è la richiesta di servizio per job sul livello i . Il throughput massimo del sistema è quindi:

$$X_{max} = \min_i X_{max,i}$$

Il livello i che realizza questo minimo è il bottleneck.

Livello A / A2

Ogni job visita il livello A per tre volte, ogni volta con tempi di servizio medi diversi.

$$E[S_{A,1}] = 0.2 \text{ s}$$

$$E[S_{A,2}] = 0.4 \text{ s}$$

$$E[S_{A,3}] = 0.1 \text{ s} - \text{ caso 1 factor Auth}$$

$$E[S_{A,3}] = 0.15 \text{ s} - \text{ caso 2 factor Auth}$$

La Demand di servizio complessiva per job sul livello A è

$$D_A = \sum_{k=1}^3 E[S_{A,k}] = 0.2 + 0.4 + 0.1 = 0.7 \text{ s} - \text{ caso 1 factor Auth}$$

$$D_A = \sum_{k=1}^3 E[S_{A,k}] = 0.2 + 0.4 + 0.15 = 0.75 \text{ s} - \text{ caso 2 factor Auth}$$

Livello B / B2

Ogni job visita il livello B una sola volta.

$$E[S_B] = 0.8 \text{ s}$$

La Demand di servizio per job sul livello B è

$$D_B = 0.8 \text{ s}$$

Numero di server paralleli:

$$m_B = 2$$

Livello P

Ogni job visita il livello P una sola volta.

$$E[S_P] = 0.4 \text{ s} - \text{ caso 1 factor Auth}$$

$$E[S_p] = 0.7 s - \text{caso 2 factor Auth}$$

La Demand di servizio per job sul livello P è

$$D_p = 0.4 s - \text{caso 1 factor Auth}$$

$$D_p = 0.7 s - \text{caso 2 factor Auth}$$

Numero di server paralleli:

$$m_p = 1$$

Throughput massimo per livello

Livello A / A2

$$X_{max,A} = \frac{m_A}{D_A} = \frac{2}{0.70} = 2.857142 \text{ job/s} - \text{caso 1 fact Auth}$$

$$X_{max,A} = \frac{m_A}{D_A} = \frac{2}{0.75} = 2.666666 \text{ job/s} - \text{caso 2 fact Auth}$$

Livello B / B2

$$X_{max,B} = \frac{m_B}{D_B} = \frac{2}{0.8} = 2.5 \text{ job/s}$$

Livello P

$$X_{max,P} = \frac{m_p}{D_p} = \frac{1}{0.4} = 2.5 \text{ job/s} - \text{caso 1 fact Auth}$$

$$X_{max,P} = \frac{m_p}{D_p} = \frac{1}{0.7} = 1.428571 \text{ job/s} - \text{caso 2 fact Auth}$$

Identificazione del bottleneck

Confrontando i throughput massimi:

$$X_{max} = 2.5 \text{ job/s} - \text{caso 1 factor Auth}$$

$$X_{max} = 1.428571 \text{ job/s} - \text{caso 2 factor Auth}$$

I livelli **B / B2** e **P** e il **livello P** rispettivamente, costituiscono il bottleneck teorico in termini di capacità nel caso di autenticazione ad uno e a due fattori.

Esecuzione delle Simulazioni

Analisi del transitorio

Protocollo di simulazione

Sono state eseguite **5 repliche** della simulazione. Ogni replica ha una **soglia temporale** pari a **200.000 secondi**, così da valutare se e quando il sistema mostri **convergenza verso un regime stazionario**.

Raccolta delle metriche

Le metriche operative del sistema sono state **campionate ogni 100 secondi** per l'intera durata di ciascuna replica.

RNG multi-stream: inizializzazione e utilizzo

Per la generazione delle variabili aleatorie è stata adottata la libreria **RNG multi-stream** basata su generatori Lehmer, che mette a disposizione **256 stream indipendenti**.

Ogni stream è assegnato a **una specifica funzione del modello**, così da separare statisticamente:

- gli **arrivi**
- i **tempi di servizio** dei diversi stadi
- le **decisioni di load balancing**,
- le **politiche di spillover**,
- gli **eventi di perdita**.

Questa separazione garantisce **riproducibilità**, **assenza di correlazioni spurie** tra componenti del sistema e **coerenza statistica** nelle repliche.

La **mappa degli stream** utilizzati è la seguente:

Stream ID	Nome costante	Ruolo nel modello
0	STREAM_ARR	Generazione dei processi di arrivo

1	STREAM_A_1 e STREAM_A1_A2	Tempi di servizio nel server A e sul server A2 (prima visita)
2	STREAM_B1 e STREAM_B_2	Tempi di servizio nel server B e B2
3	STREAM_A_2 e STREAM_A2_A2	Tempi di servizio nel server A e server A2 (seconda visita)
4	STREAM_P	Tempi di servizio al nodo P (stadio di post-processing)
5	STREAM_A_3 STREAM_A3_A2	Tempi di servizio nel server A e server A2 (terza visita)
6	STREAM_PROBE	Decisioni di probing/adattamento (feedback sul sistema)
7	STREAM_LB_A	Scelte di load-balancing tra A e A2
8	STREAM_LB_B	Scelte di load-balancing tra B e B2

9	<code>STREAM_LOSS_B</code> / <code>STREAM_LOSS_B2</code>	Estrazioni per eventi di perdita sul server B e B2
10	<code>STREAM_LOSS_P</code>	Estrazioni per perdite a valle del nodo P

Politica di inizializzazione nelle repliche

La chiamata:

```
plantSeeds(MASTER_SEED)
```

viene eseguita **una sola volta prima del ciclo delle 40 repliche**.

In questo modo tutti gli stream partono da **stati deterministici ma indipendenti**.

Durante ciascuna replica:

- la simulazione richiama `selectStream(...)` ogni volta che deve generare un variate,
- gli stream avanzano **in modo indipendente**,
- lo stato del modello (code, server, contatori) viene **re-inizializzato**, ma **lo stato degli stream prosegue**.

Questo fa sì che:

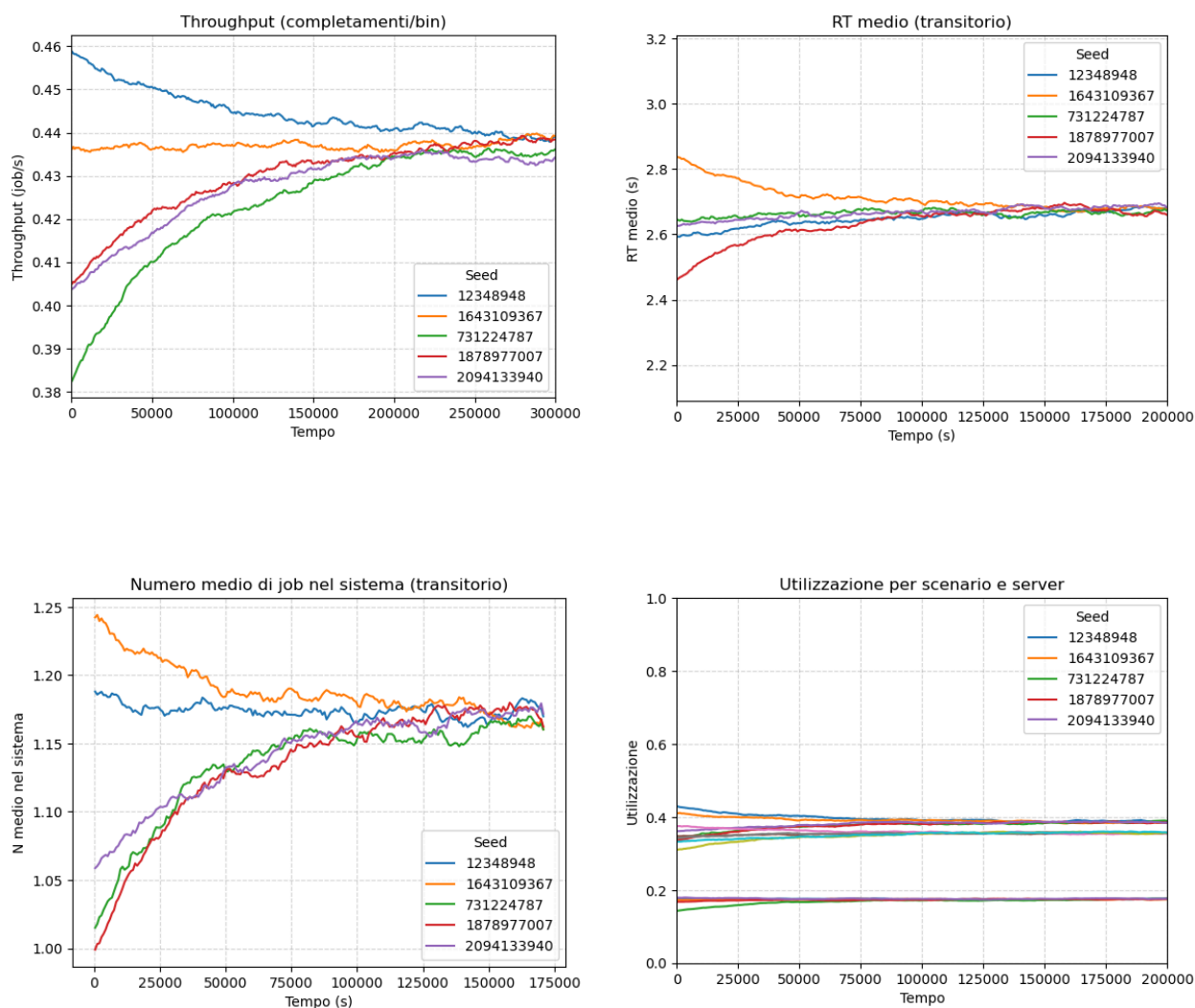
- ogni replica **usi porzioni disgiunte** delle sequenze casuali → **niente sovrapposizione**,
- le repliche siano **i.i.d.** (indipendenti e identicamente distribuite) dal punto di vista statistico,
- è possibile **replicare esattamente** una run salvando e ripristinando i seed degli stream utilizzati.

Replicabilità

Al termine di ogni replica **salviamo lo stato/seed di ciascuno stream** in `seedMigliorativo.csv`. Questo salvataggio **non è necessario** per far “ripartire” la replica successiva (che continua comunque dagli

stati interni mantenuti dalla libreria), ma consente la **replicazione esatta** degli esperimenti, il **ripristino** dopo eventuali interruzioni e la **tracciabilità** delle varie run.

Perveniamo ai seguenti grafici:



Dai risultati ottenuti vediamo che il throughput del sistema raggiunge un regime stazionario dopo circa 200.000 secondi, mentre il tempo di risposta medio di tutto il sistema e utilizzazione per ogni server vengono raggiunti intorno ai 100.000 s.

Simulazioni orizzonte finito

One-factor authentication orizzonte finito

Anche in questo caso è stata adottata una **simulazione a orizzonte finito**, con l’obiettivo di analizzare l’evoluzione del sistema su un intervallo temporale di **172 000 secondi**, corrispondente alla durata del periodo di osservazione del **weekend**.

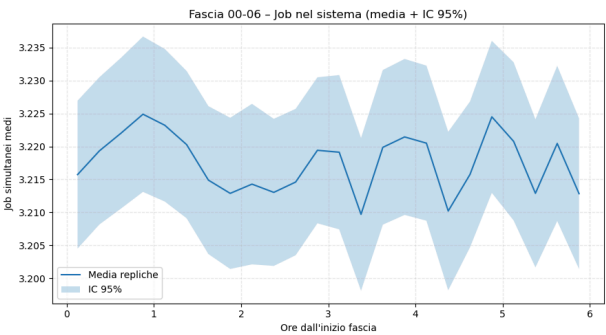
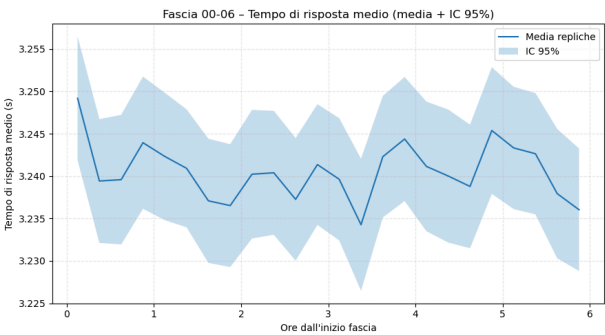
Tale configurazione permette di valutare il comportamento del sistema in **condizioni di carico elevato (stress)**, poiché il weekend rappresenta il periodo caratterizzato dal **massimo tasso di arrivi**.

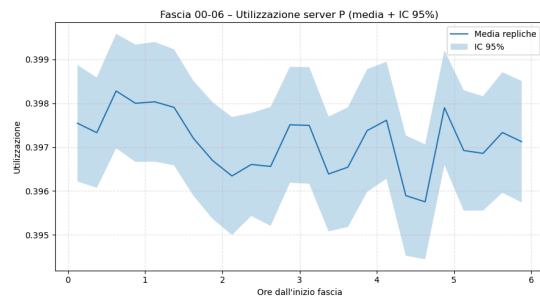
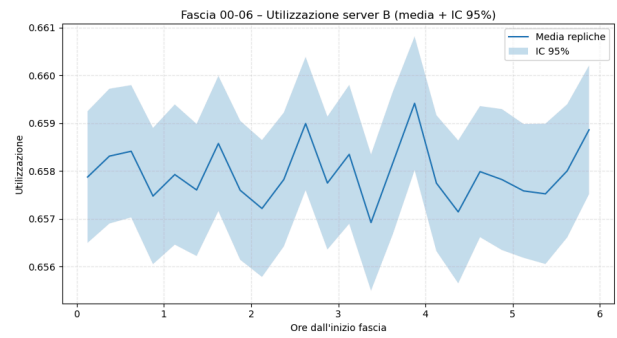
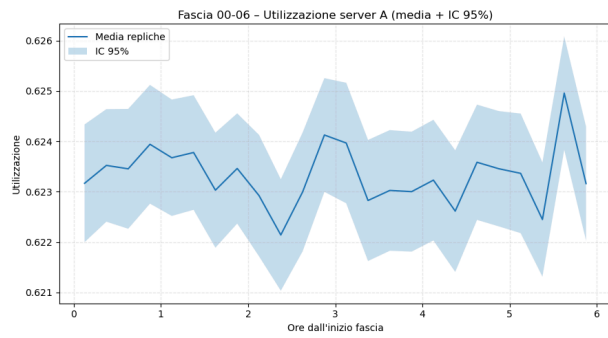
Sempre secondo la seguente tabella estrapolata dal paper sopra elencato.

	00-06	06-09	09-12	12-17	17-21	21-24
Sabato	1.176	1.224	1.23	1.351	1.251	1.126
Domenica	1.13	1.24	1.287	1.379	1.244	1.166

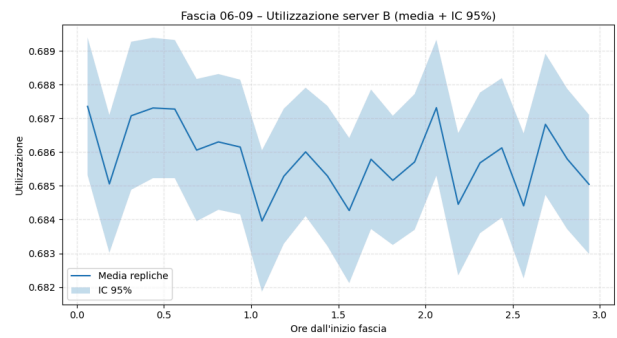
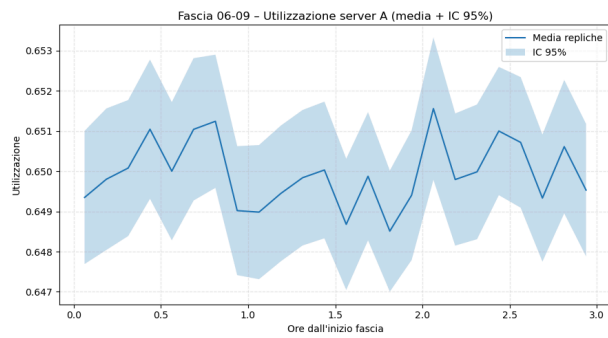
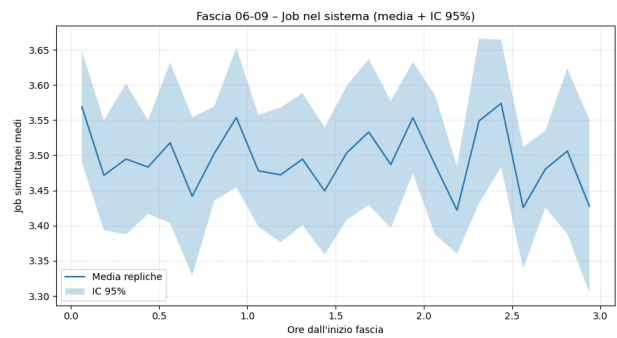
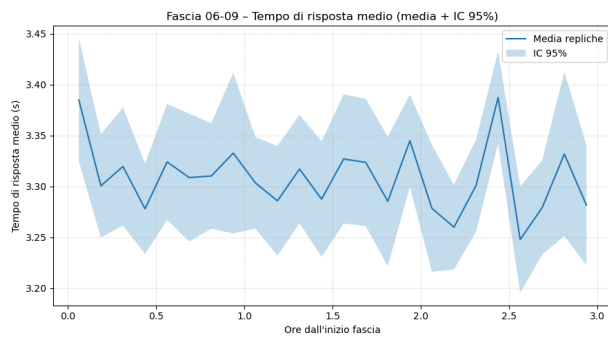
Gli **intervalli di confidenza** delle metriche di interesse sono stati calcolati utilizzando **la stessa metodologia adottata per la simulazione a orizzonte finito del modello base**. In particolare, al fine di ottenere stime statisticamente affidabili, la simulazione è stata eseguita su **385 repliche indipendenti**, e gli intervalli di confidenza sono stati costruiti sulla media delle repliche.

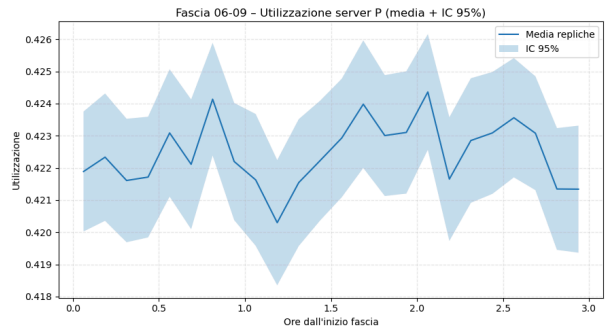
Fascia Oraria 00:00-06:00



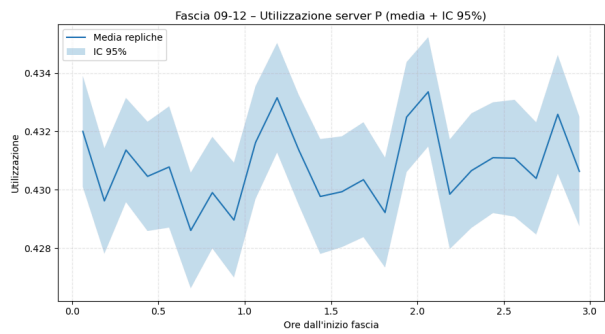
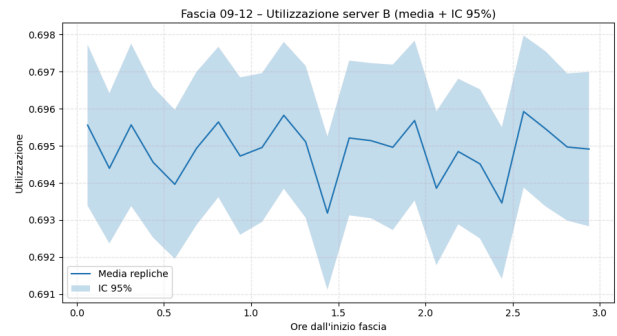
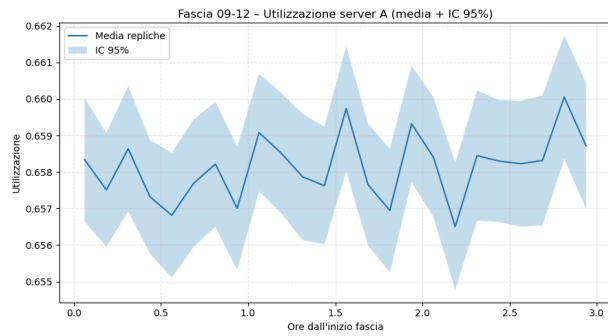
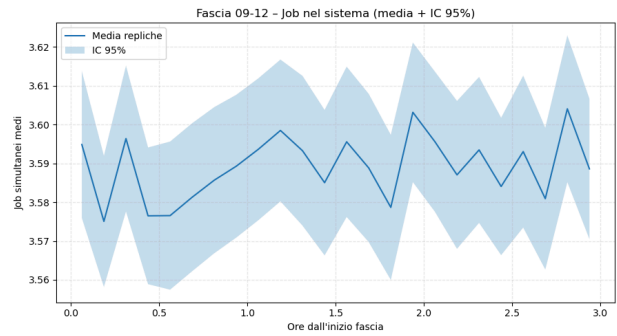
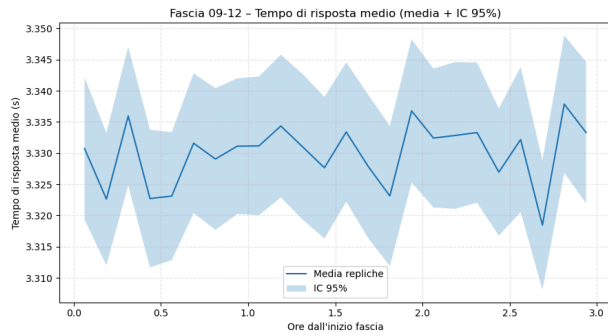


Fascia Oraria 06:00-09:00

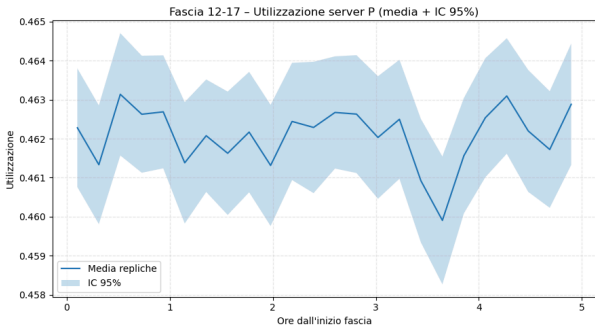
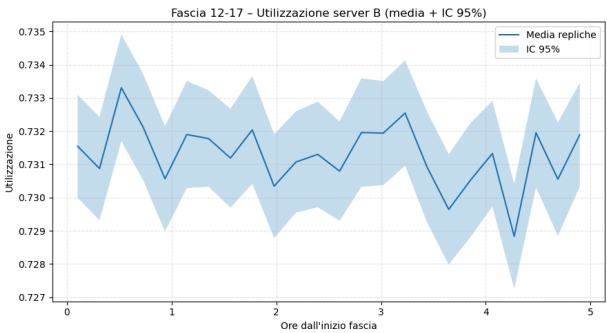
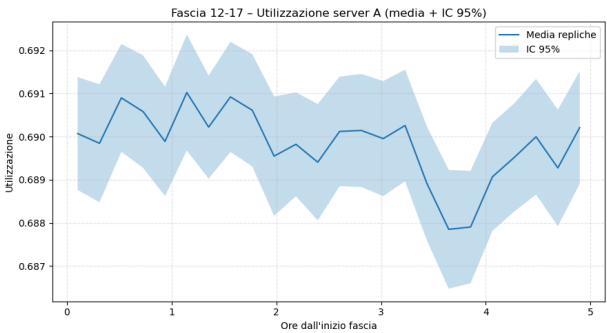
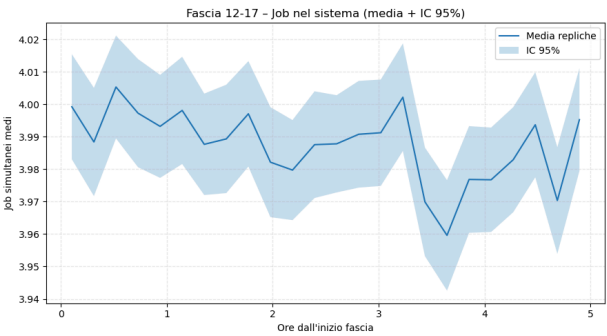
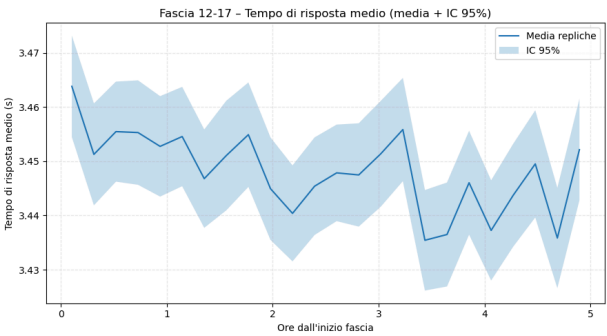




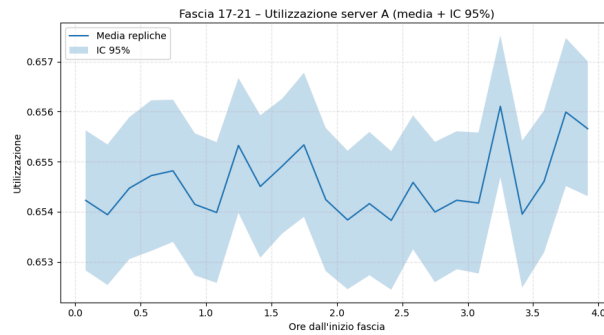
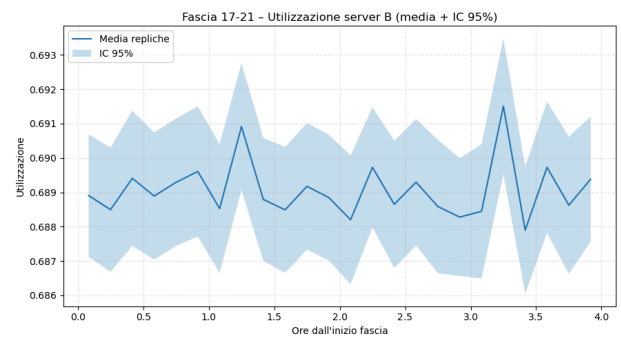
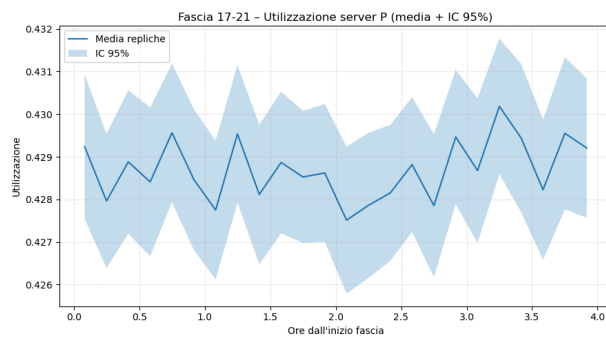
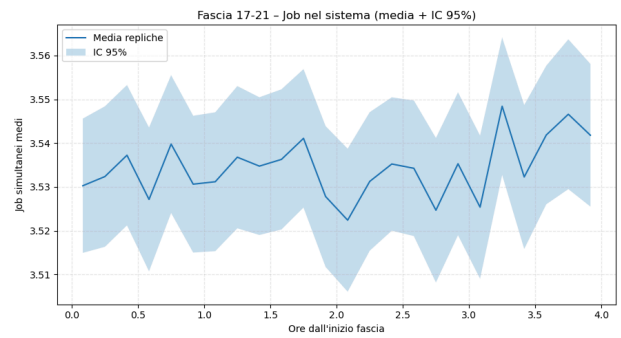
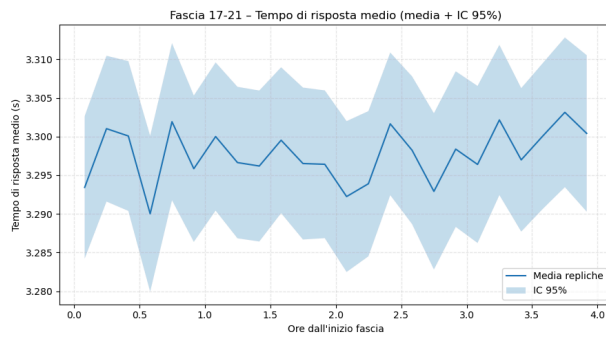
Fascia Oraria 09:00-12:00



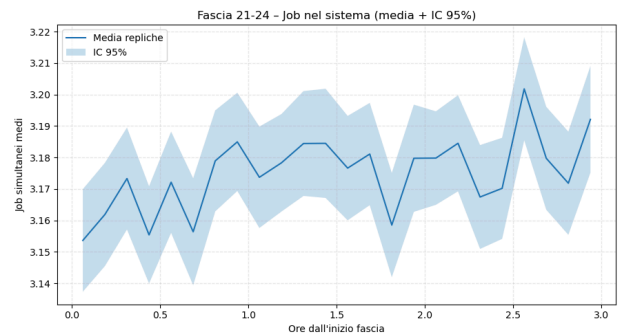
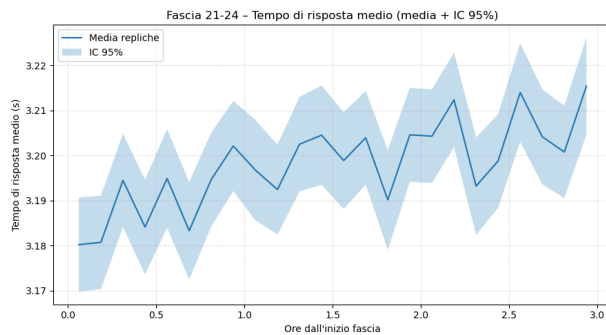
Fascia Oraria 12:00-17:00

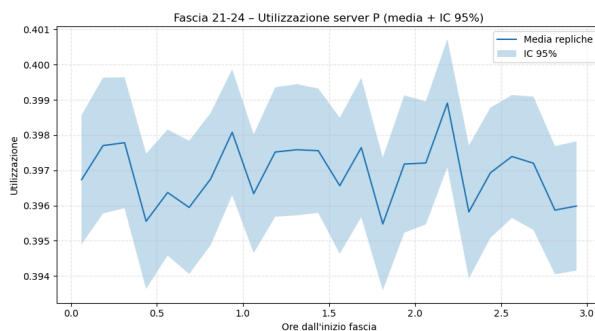
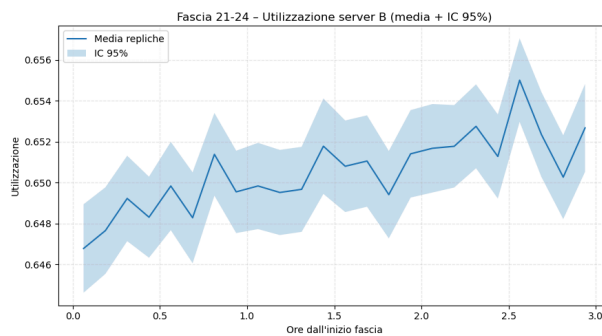
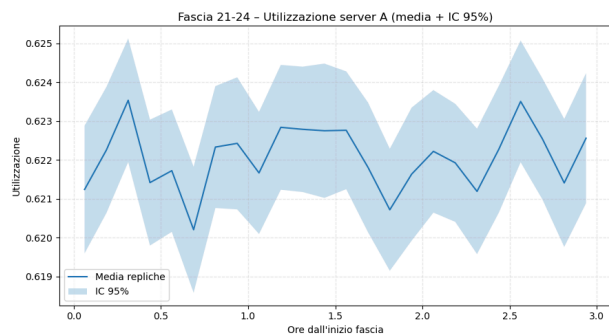


Fascia Oraria 17:00-21:00



Fascia Oraria 21:00-24:00

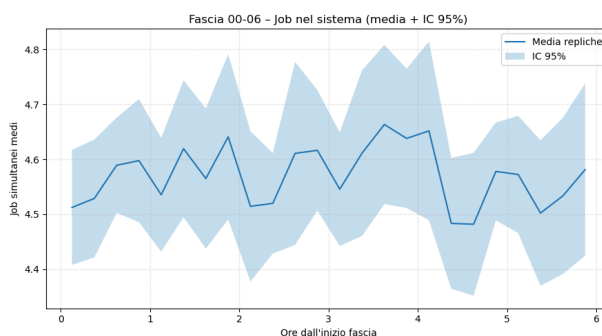
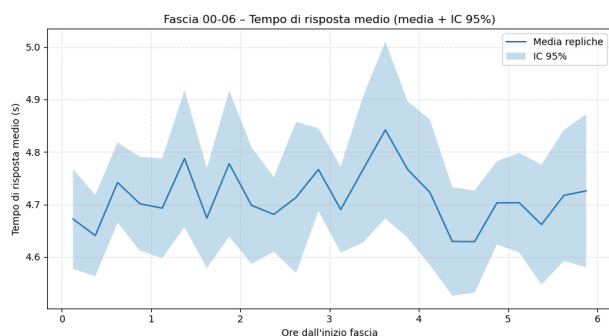


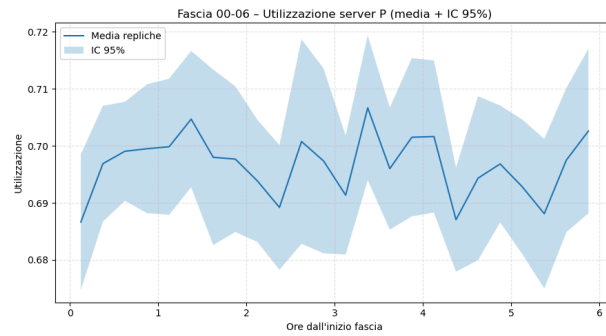
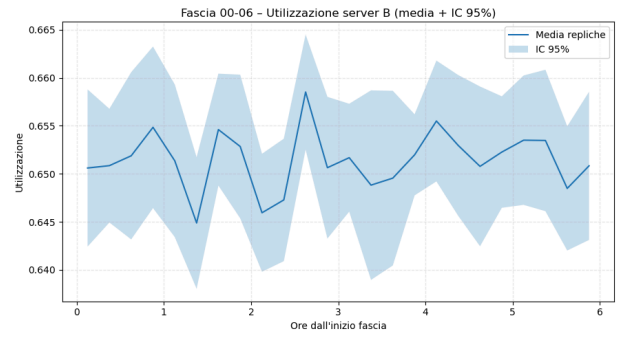
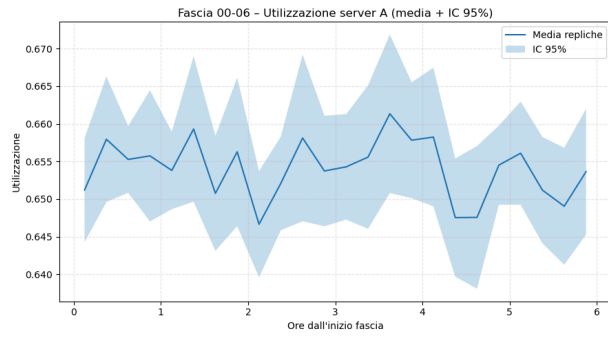


Two-factor authentication orizzonte finito

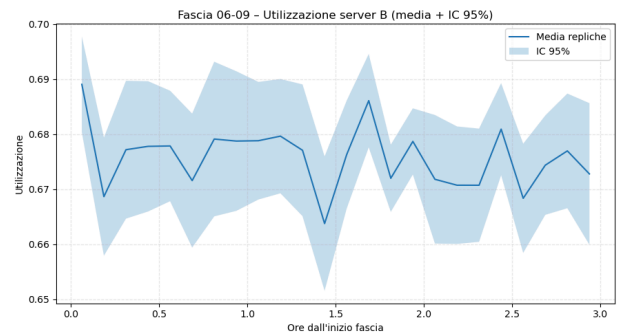
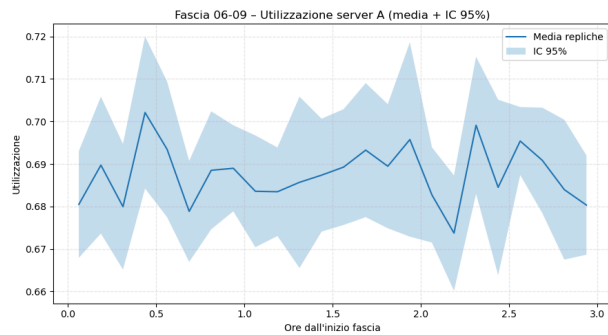
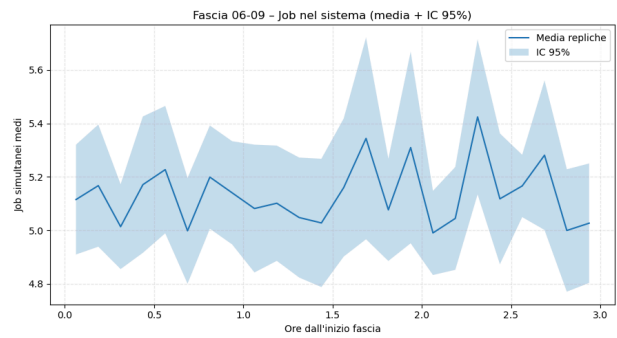
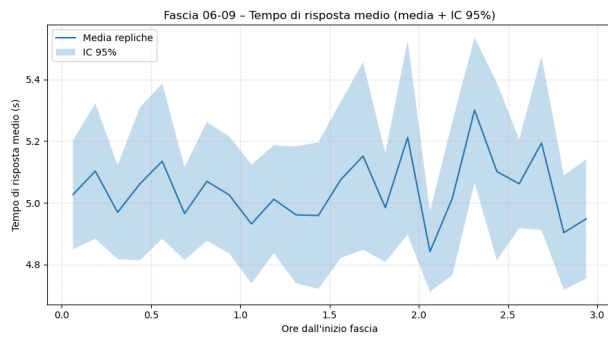
La stessa simulazione è stata eseguita anche per il two factor, ottenendo i seguenti grafici:

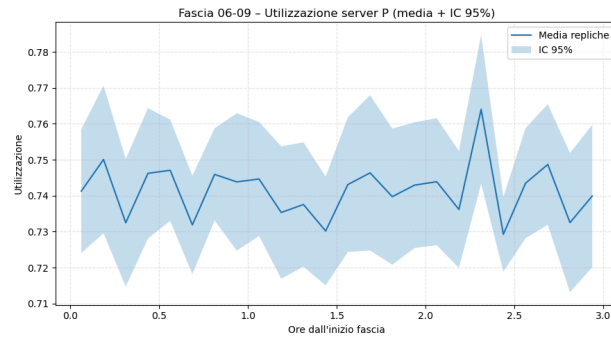
Fascia Oraria 00:00-06:00



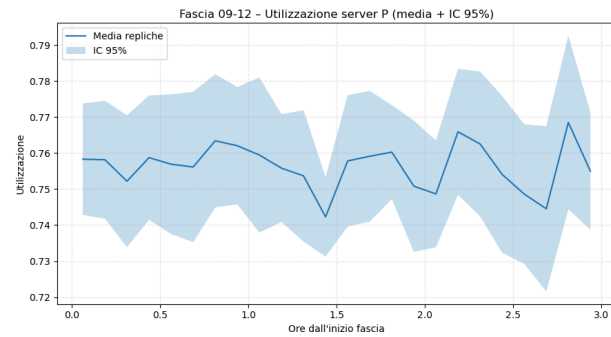
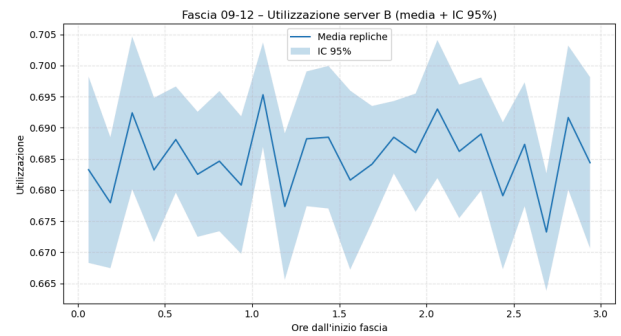
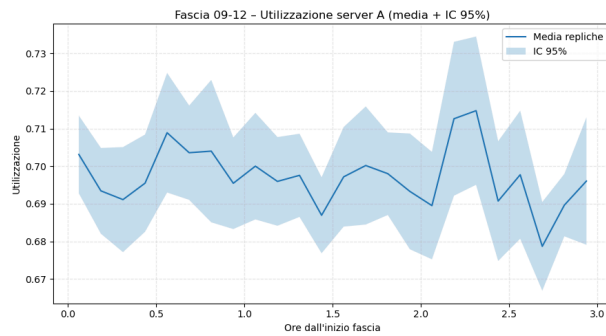
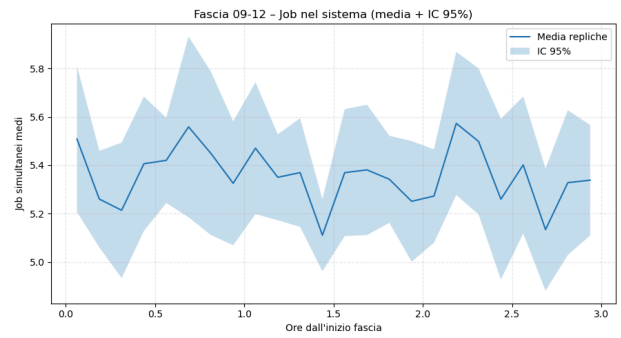
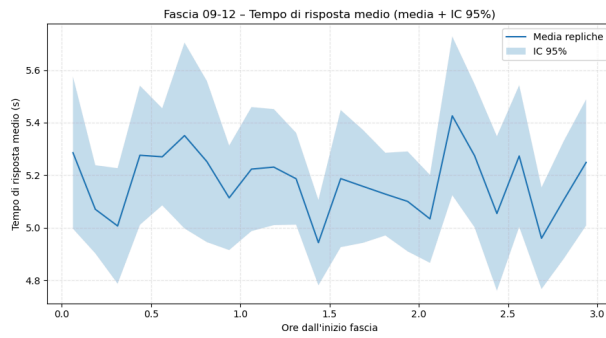


Fascia Oraria 06:00-09:00

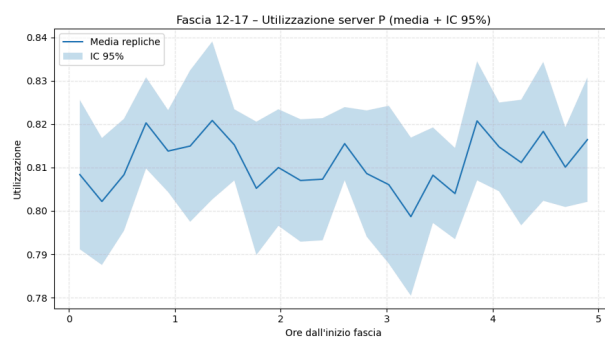
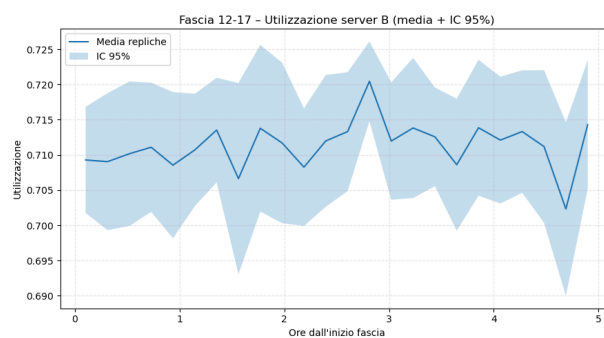
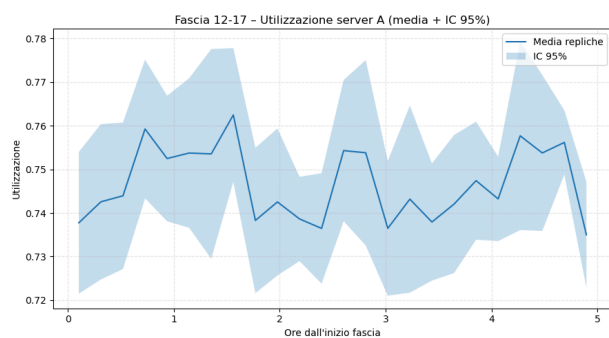
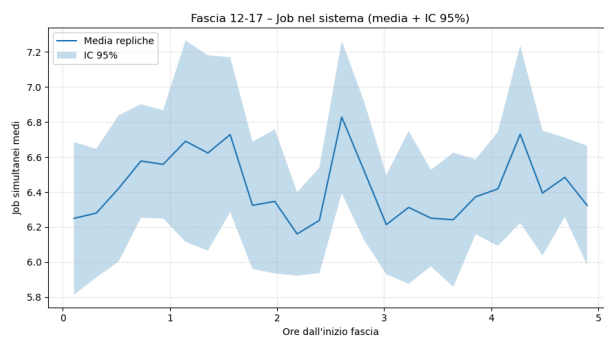
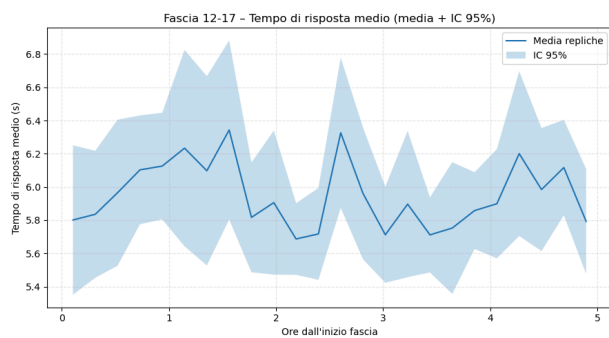




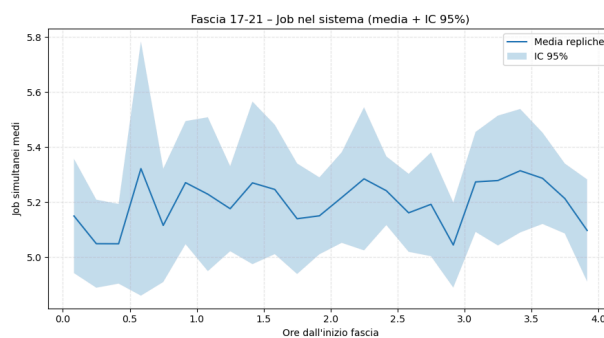
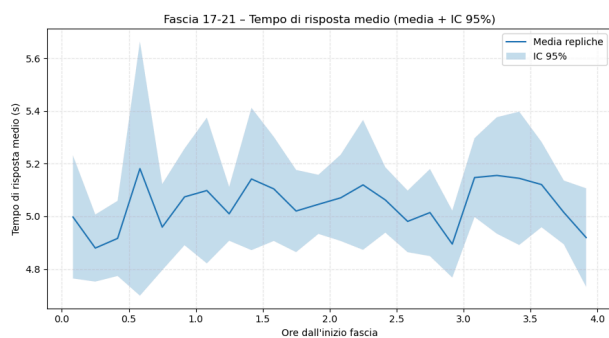
Fascia Oraria 9:00-12:00

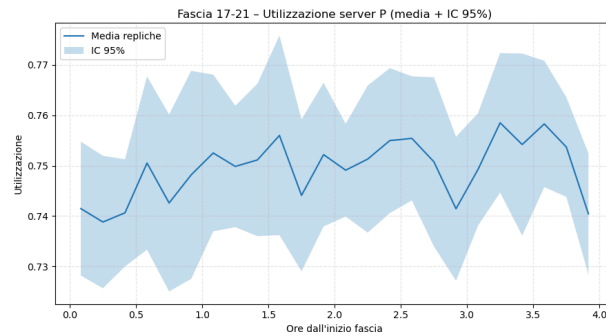
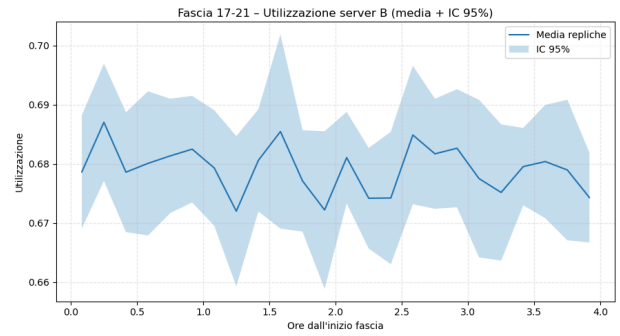
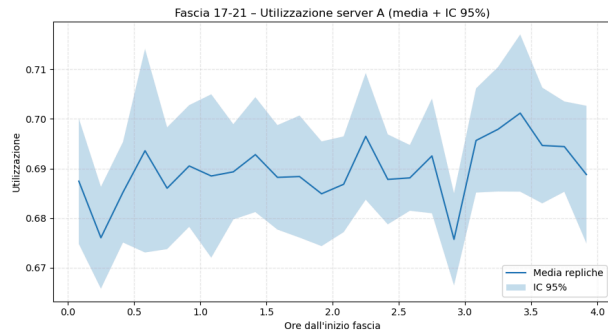


Fascia Oraria 12:00-17:00

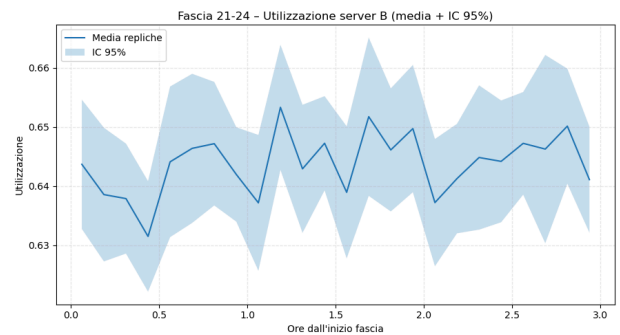
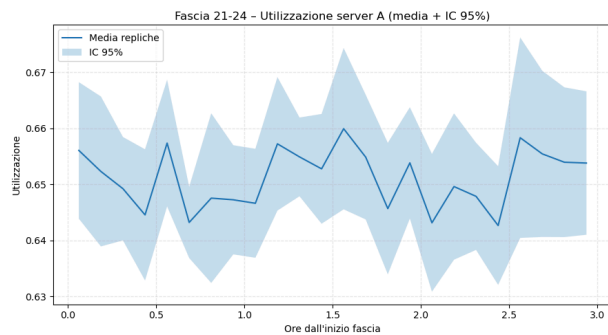
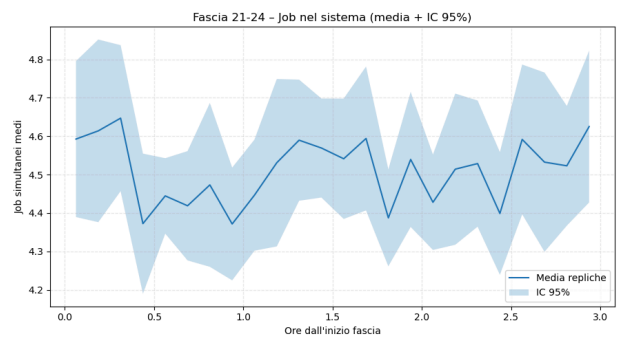
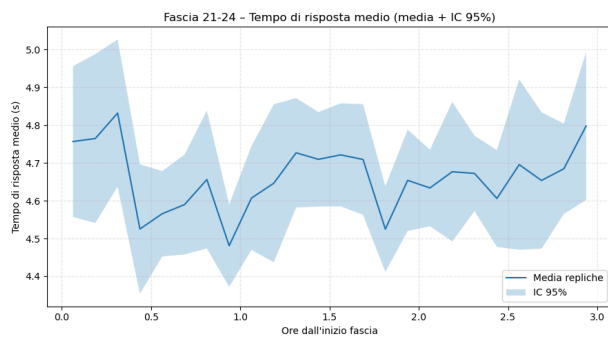


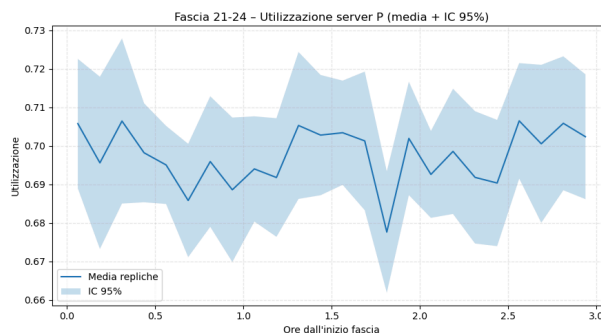
Fascia Oraria 17:00-21:00





Fascia Oraria 21:00-24:00





Conclusioni

Sulla base della configurazione ottimale, gli obiettivi prefissati sono stati raggiunti. Il *bottleneck* sul server è in grado di sostenere picchi di carico più elevati senza che le metriche operative divergano in modo significativo. Entrambe le configurazioni del sistema sono state analizzate e valutate. Inoltre, i requisiti di Quality of Service (QoS) – ovvero la minimizzazione dei tempi di servizio e delle perdite – sono stati soddisfatti grazie alla configurazione di *scaling* individuata e alle metriche misurate tramite simulazione.

In particolare, si possono fare le seguenti osservazioni:

- **Server B / B2**

Grazie all'impiego congiunto di meccanismi di scale up/scale down e di load balancing, configurati con soglie ottimizzate per attivarsi in prossimità dei picchi di carico, è possibile contenere le perdite e garantire tempi di risposta adeguati. In particolare, nel best case scenario il tempo medio di risposta è di circa 2 secondi, ampiamente inferiore al limite imposto dal QoS (6 secondi). Le perdite registrate si attestano intorno al 6%, valore considerato accettabile rispetto alle condizioni operative del sistema.

- **Server A/ A2**

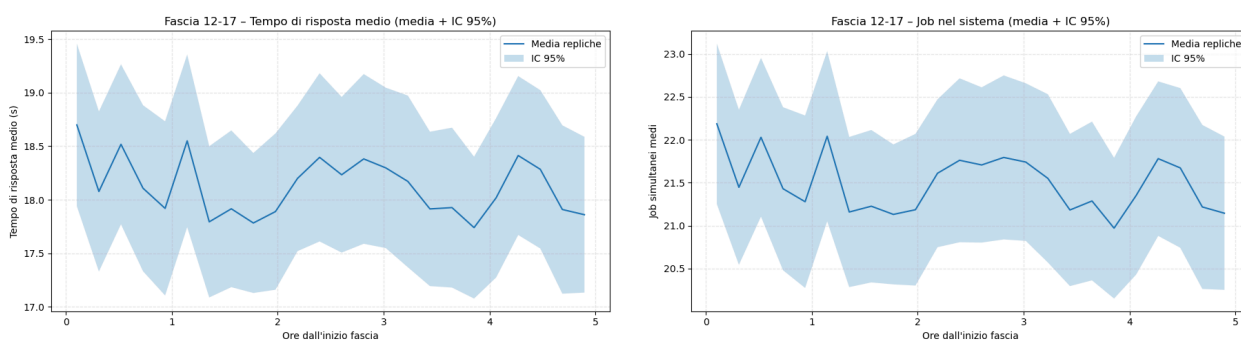
Grazie all'impiego congiunto di meccanismi di scale up/scale down e di load balancing, configurati con soglie ottimizzate per attivarsi in prossimità dei picchi di carico come nel server B, vengono garantiti tempi di risposta adeguati.

- **Server P**

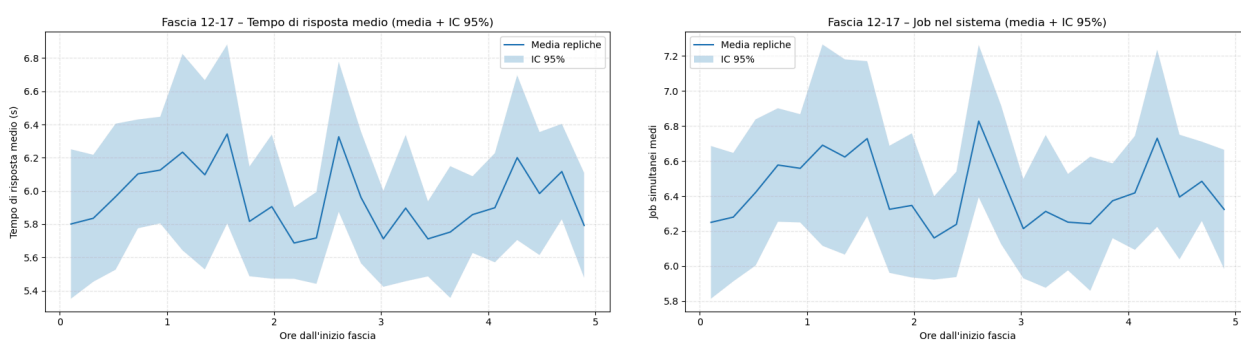
In seguito all'introduzione dei meccanismi sopra descritti, il Server P si configura come il nuovo collo di bottiglia del sistema. Tuttavia, come già indicato anche dalla verifica analitica, i risultati della simulazione del modello finale mostrano che i tempi medi di risposta del provider esterno rimangono inferiori ai limiti imposti dal QoS. Le perdite riscontrate in questa fase si attestano tra il 6,5% e il 7% dei job in ingresso, valore che, pur evidenziando un punto critico, rientra nei margini di accettabilità.

Per concludere, confrontiamo la simulazione a orizzonte finito dei modelli, base e migliorativo, concentrandoci sulla fascia oraria più congestionata, dalle 12:00 alle 17:00.

Modello base (One factor):



Modello migliorativo(One factor):



Si evidenzia un chiaro miglioramento tra il modello base e quello migliorativo. Di conseguenza, i requisiti di QoS inizialmente definiti possono considerarsi soddisfatti. Inoltre, anche in condizioni di elevato carico, come durante il weekend, il modello migliorativo riesce a mantenere un tempo medio di risposta eccellente, contenendo le perdite complessive entro un range accettabile pari al 14,37%.

